



UNIVERSITÀ DI PARMA

DIPARTIMENTO DI SCIENZE MATEMATICHE, FISICHE E INFORMATICHE

Corso di Laurea Magistrale in Scienze Informatiche

Simulazione tridimensionale della stabilità del carico basata su tecniche di object detection e video tracking

*Three-Dimensional Simulation of Cargo Stability Based on
Object Detection and Video Tracking*

CANDIDATO:
Marco Corazzini

RELATORE:
Prof.ssa Eleonora Iotti

MATRICOLA:
380472

*“La realtà che possiamo esprimere a parole
non è mai la realtà stessa.”*

Werner Karl Heisenberg

Per le presenze che il tempo non cancella.

Indice

Introduzione	1
1 Stato dell'arte e fondamenti teorici	3
1.1 Standard europei per il trasporto pallettizzato	3
1.1.1 Criteri di ottimizzazione e stabilità del carico	5
1.2 Fondamenti di Computer Vision	6
1.2.1 Rappresentazione delle immagini e feature extraction	7
1.2.2 Object Detection	8
1.2.3 Approcci tradizionali nella Computer Vision	9
1.2.4 Reti neurali convoluzionali	14
1.2.5 YOLO (You Only Look Once)	19
1.2.6 Object Tracking nelle sequenze video	25
1.3 Motori di Simulazione Fisica	30
1.3.1 Evoluzione dei motori fisici	30
1.3.2 Motori fisici moderni	31
1.3.3 Bullet Physics Engine	32
1.3.4 PyBullet	33
1.4 Fondamenti di analisi statistica e probabilistica	34
1.4.1 Distribuzione normale e momenti statistici	34
1.4.2 Analisi di sensibilità	35
1.4.3 Correlazioni	36
1.4.4 ANOVA ad una e due vie	36
1.4.5 p-value e significatività statistica	37
1.4.6 Test di Kruskal–Wallis	37
1.4.7 Analisi di convergenza	37
2 Costruzione del dataset e sviluppo dei modelli YOLO	39
2.1 Primi approcci al problema di object detection	39
2.1.1 Background subtraction	39
2.1.2 U ² -Net	40
2.1.3 YOLOv8	41
2.2 Costituzione del dataset	42

2.2.1	Selezione dei video per l'addestramento	42
2.2.2	Estrazione dei frame dai video	43
2.2.3	Annotazione delle immagini	45
2.3	Addestramento dei modelli YOLO	47
2.3.1	Addestramento con bounding box orizzontali	50
2.3.2	Evoluzione verso bounding box orientate	55
3	Dall'analisi video alla simulazione fisica	61
3.1	Pipeline generale del sistema	61
3.2	Analisi video tramite YOLO	63
3.2.1	Tracking e stabilizzazione delle detections	64
3.2.2	Analisi delle traiettorie e generazione della reportistica grafica	66
3.3	Ricostruzione della simulazione fisica	71
3.3.1	Creazione dello scenario simulato	71
3.3.2	Modellazione cinematica della slitta	77
3.4	Modellazione della pellicola estensibile mediante modello a parametri concentrati	79
3.4.1	Scelta del paradigma di modellazione: parametri concentrati o analisi agli elementi finiti	79
3.4.2	Modello viscoelastico di Kelvin–Voigt	80
3.4.3	Implementazione della rete di molle	82
3.4.4	Parametri del modello	87
3.5	Ottimizzazione dei parametri	89
3.5.1	Struttura della grid search	89
3.5.2	Adattamento dinamico della griglia: rilevamento della stabilità	90
3.5.3	Funzione di costo	91
3.5.4	Esecuzione parallela e integrazione cinematica	92
3.5.5	Processo di fine-tuning in batch	93
3.5.6	Risultati e selezione dei parametri ottimali	94
4	Ottimizzazione dei parametri e risultati	95
4.1	Descrizione del protocollo sperimentale	95
4.2	Distribuzione globale della funzione di perdita	96
4.2.1	Statistica descrittiva sull'intero spazio di ricerca	96
4.2.2	Distribuzione delle configurazioni ottimali	97
4.3	Analisi per gruppi: massa e accelerazione	100
4.3.1	Effetto della massa del pallet	100
4.3.2	Effetto del livello di accelerazione	101
4.4	Distribuzione dei parametri ottimali	102
4.4.1	Numero di avvolgimenti (N_w)	102

4.4.2	Moltiplicatore di rigidezza (k_s)	102
4.4.3	Moltiplicatore di smorzamento (k_d)	102
4.4.4	Moltiplicatore di rottura (k_b)	103
4.4.5	Moltiplicatore di attrito (k_f)	103
4.5	Analisi di sensibilità	103
4.6	Analisi delle correlazioni	105
4.7	Test non parametrico di Kruskal-Wallis	106
4.8	Analisi delle interazioni tra parametri	107
4.9	Analisi di convergenza	109
4.10	Variabilità cross-video e generalizzazione	110
4.11	Sintesi dei risultati	111
Conclusione		113
Bibliografia		115

Elenco delle figure

1.1	Frame della piattaforma di test dinamico ESTL, utilizzata per l'esecuzione delle prove secondo lo standard EUMOS 40509. . . .	4
1.2	Esempio di disposizione di carico su pallet all'interno di un camion, nello specifico sorretto da un sistema di cinghie di fissaggio.	6
1.3	Esempio di processo di feature extraction di un'immagine. . . .	7
1.4	Esempio di processo di object detection in un'immagine. [5] . .	9
1.5	Diagramma dello spazio colore CIE 1931 che rappresenta lo spazio colore Rec. 601 (SDTV) all'interno del triangolo cromatico. Il nero indica i sistemi a 625 linee, mentre il bianco i sistemi a 525 linee. Lo standard Rec. 601 adotta l'illuminante D65 come punto di bianco di riferimento. [15]	10
1.6	Immagine originale (sinistra) e filtro gaussiano (destra).	11
1.7	Immagine originale (sinistra) e thresholding binario (destra). . .	11
1.8	Esempio di background subtraction.	12
1.9	Esempi di segmentazione ottenuti con U ² -Net. [24]	13
1.10	Applicazione di un kernel di convoluzione su un'immagine di input per generare una feature map.	15
1.11	Grafico della funzione di attivazione ReLU.	16
1.12	Esempio di Max Pooling e Average Pooling. [30]	17
1.13	Rilevamento di droni in tempo reale mediante un modello YOLO. [8]	19
1.14	Suddivisione in celle e predizione di bounding boxes in YOLO. [2]	20
1.15	Architettura della rete YOLOv8 per il rilevamento degli oggetti.	23
1.16	Tracking delle traiettorie percorse da veicoli all'interno di una scena video.	26
1.17	Predizione del movimento di pedoni tramite Kalman Filter. . . .	28
1.18	Differenze di rilevamenti tra SORT, ByteTrack e DeepSORT in un contesto di tracking multi-oggetto.	29

1.19	Esempio di simulazione fisica realizzata con Open Dynamics Engine (ODE), uno dei primi motori fisici open source utilizzati in ambito accademico e robotico per la simulazione di corpi rigidi e vincoli articolari.	31
1.20	Architettura semplificata del Bullet Physics Engine e pipeline di simulazione fisica.	33
1.21	Analisi del processo di convergenza della metrica di errore x_n in funzione del numero di iterazioni o campioni n . Vengono messe a confronto tre diverse configurazioni parametriche (A, B e C) rispetto al vero limite di convergenza asintotico $L = 0.5$ (indicato dalla linea tratteggiata orizzontale). L'area ombreggiata rappresenta l'intervallo di confidenza associato alla Configurazione A ($\pm\sigma$), evidenziando la riduzione dell'incertezza all'aumentare delle iterazioni.	38
2.1	Frame elaborato tramite background subtraction	40
2.2	Frame elaborato YOLOv8m pre-addestrato su COCO	41
2.3	Esempi di frame selezionati per l'addestramento dei modelli	43
2.4	Esempi di frame etichettati tramite Label Studio, prima e dopo	46
2.5	Grafici di training e validation del modello YOLOv8n addestrato con bounding box orizzontali	51
2.6	Grafici di training e validation del modello YOLOv8s addestrato con bounding box orizzontali	52
2.7	Grafici di training e validation del modello YOLOv8m addestrato con bounding box orizzontali	53
2.8	Grafici comparativi delle metriche di validazione tra i modelli YOLOv8 Nano, Small e Medium (addestramento HBB)	54
2.9	Esempio di frame elaborato con bounding box orizzontali, evidenziando i limiti nella localizzazione di oggetti inclinati	55
2.10	Grafici di training e validation del modello YOLOv8s-obbb addestrato con bounding box orientate	56
2.11	Grafici di training e validation del modello YOLOv8m-obbb addestrato con bounding box orientate	57
2.12	Grafici di training e validation del modello YOLOv8l-obbb addestrato con bounding box orientate	58
2.13	Grafici comparativi delle metriche di validazione tra i modelli YOLOv8 Small, Medium e Large (addestramento OBB)	60
2.14	Esempio di frame elaborato con bounding box orientate	60
3.1	Pipeline generale del software sviluppato.	62
3.2	Confronto tra i rilevamenti del modello YOLO pre-addestrato su COCO e il modello custom	63

3.3	Confronto tra i rilevamenti grezzi (a) e quelli post-processati (b) con il sistema di tracking e stabilizzazione implementato.	66
3.4	Esempio di grafico generato dal modulo di reportistica, con segmentazione cromatica degli stati di stabilità.	70
3.5	Esempio di tre simulazioni dello stesso video con diverse profondità di colli	73
3.6	Creazione dello scenario simulato a partire dalle informazioni geometriche estratte dal sistema di tracking.	75
3.7	Comparazione tra rilevamento YOLO e Replay in PyBullet . . .	76
3.8	Rappresentazione del modello a parametri concentrati di Kelvin–Voigt adottato per la pellicola estensibile.	80
3.9	Schema della registrazione di un package e generazione della molla virtuale del film estensibile.	82
3.10	Esempi di lavoro della pellicola in diverse fasi della simulazione .	86
4.1	Istogramma della funzione di perdita $\mathcal{L}$ su tutti i 34 328 run (scala logaritmica sull’asse x). Le linee verticali tratteggiate indicano il minimo (rosso), la mediana (arancio) e la media (grigio).	97
4.2	Istogramma di $\mathcal{L}$ per le configurazioni best-per-video ($n = 103$, scala logaritmica). In alto a destra sono riportati i valori di asimmetria e curtosi in eccesso.	98
4.3	Confronto tra la distribuzione di $\mathcal{L}$ su tutti i run e sulle configurazioni best-per-video (scala logaritmica). Il diamante rosso indica la media aritmetica.	99
4.4	Funzione di ripartizione empirica (ECDF) di $\mathcal{L}$ per le configurazioni best-per-video. Le linee tratteggiate evidenziano i quantili Q1, mediana e Q3 derivati dalla Tabella 4.2.	99
4.5	Distribuzione di $\mathcal{L}^*$ per classe di massa del pallet. I box-plot mostrano Q1, mediana e Q3; i punti rossi indicano le singole osservazioni (scala logaritmica).	101
4.6	Ranking di sensibilità dei parametri fisici. L’asse orizzontale riporta Δ_j (equazione (4.2)); la barra evidenziata in blu corrisponde al parametro più influente.	104
4.7	Distribuzione di $\mathcal{L}$ per ciascun valore discreto dei cinque parametri (violin plot, scala logaritmica). La linea rossa indica la mediana di ciascun gruppo.	105
4.8	Mappe di calore della $\mathcal{L}$ media per le cinque coppie parametriche selezionate. La scala cromatica va dal verde (minimo MSE) al rosso (massimo MSE).	108

4.9 Valori di $\mathcal{L}^*$ (best-per-video) ordinati in modo crescente. La linea arancione tratteggiata indica la mediana (0.158), quella rossa la media aritmetica (34.28). Scala logaritmica sull'asse y . . 110

Elenco degli algoritmi

1	Estrazione di frame dai video per la costruzione del dataset . . .	44
2	Costruzione del dataset di addestramento da immagini e label .	48
3	Procedura di training per object detection basata su YOLO . .	49
4	Registrazione di un package e creazione della molla di film . . .	83
5	Applicazione della forza elastica-viscosa della film spring	84

Elenco delle tabelle

4.1	Parametri fisici oggetto di fine-tuning e relativi spazi di ricerca.	96
4.2	Statistica descrittiva di $\mathcal{L}$ su tutti i run e sul sottoinsieme delle configurazioni migliori per video.	96
4.3	Distribuzione dell'errore MSE ($\mathcal{L}$) per le configurazioni <i>best-per-video</i> .	98
4.4	Statistica della funzione di perdita best-per-video per classe di massa del singolo oggetto caricato sul pallet.	100
4.5	Statistica della funzione di perdita best-per-video per livello di accelerazione.	101
4.6	Distribuzione dei parametri nelle configurazioni ottimali (best-per-video, $n = 103$). La moda è evidenziata in grassetto.	102
4.7	Ranking di sensibilità dei parametri fisici. La colonna <i>Valore ottimo marginale</i> indica il livello con la perdita media minima nella griglia di ricerca globale.	104
4.8	Correlazioni parametro $\rightarrow \mathcal{L}$ ($n = 34\,328$). Significatività: *** $p < 0.001$, * $p < 0.05$.	105
4.9	Test di Kruskal-Wallis: effetto di ciascun parametro sulla distribuzione di $\mathcal{L}$ ($n = 34\,328$).	106
4.10	Interazione Stiffness Mult $\times$ Break Mult: $\mathcal{L}$ media per cella della griglia. Il minimo globale è evidenziato in grassetto.	107
4.11	Convergenza media per soglia ϵ (media e dev. std. su 103 video).	109

Introduzione

Nel trasporto di merci su pallet, garantire la stabilità del carico è una sfida concreta e quotidiana: scatole mal posizionate, pellicole di avvolgimento insufficienti o configurazioni di impilamento sbilanciate possono causare il cedimento del pallet durante la movimentazione, con conseguenti danni ai prodotti, rischi per la sicurezza degli operatori e costi economici rilevanti. Valutare preventivamente se una determinata configurazione di imballaggio supererà le prove dinamiche previste dagli standard europei richiede oggi la realizzazione fisica di ogni singolo prototipo, un processo iterativo costoso e in termini di tempo.

Questa tesi affronta il problema proponendo un sistema integrato che combina tecniche di *computer vision* e simulazione fisica per analizzare, ricostruire e predire il comportamento dinamico di carichi pallettizzati, riducendo la necessità di test fisici ripetuti.

Il Capitolo 1 introduce il contesto applicativo: vengono presentati gli standard europei per il trasporto pallettizzato, con particolare riferimento alla normativa EUMOS 40509 e ai criteri di stabilità del carico, seguiti dai fondamenti teorici delle tecniche di computer vision impiegate nel progetto: dall'object detection ai modelli YOLO e dai principi della simulazione fisica tramite motori come PyBullet.

Il Capitolo 2 descrive la costruzione del dataset e lo sviluppo dei modelli di rilevamento. A partire da circa 450 video di crash test reali forniti dall'azienda collaboratrice ACMI S.p.A. con sede a Fornovo di Taro (Parma), vengono illustrati i diversi approcci esplorati: dalla background subtraction alle reti convoluzionali di segmentazione fino alla scelta definitiva di YOLOv8 con bounding box orientate, addestrato su un dataset custom per il rilevamento dei singoli colli sul pallet.

Il Capitolo 3 presenta la metodologia complessiva del sistema: la pipeline sequenziale che trasforma i frame video in un gemello digitale interattivo all'interno di PyBullet, la modellazione della pellicola estensibile tramite un approccio a parametri concentrati, e la procedura di ottimizzazione automatica dei parametri fisici tramite grid search parallela.

Il Capitolo 4 riporta i risultati della campagna sperimentale di fine-tuning condotta su 103 video, analizzando la distribuzione della funzione di perdita, la sensibilità dei parametri fisici e la capacità del framework di riprodurre fedelmente il comportamento dinamico reale dei carichi pallettizzati.

Capitolo 1

Stato dell'arte e fondamenti teorici

1.1 Standard europei per il trasporto pallettizzato

Per l'analisi della stabilità dei carichi pallettizzati durante il trasporto, uno dei riferimenti più utilizzati in ambito europeo è lo standard EUMOS 40509 [11], sviluppato nel contesto del quadro normativo collegato alla European Road Worthiness Directive [10]. Tale standard definisce le procedure di prova per la valutazione del comportamento dinamico delle unità di carico, con l'obiettivo di verificare che il pallet mantenga stabilità e integrità strutturale in condizioni assimilabili a quelle che si verificano durante il trasporto su strada.

La metodologia di test prevede l'applicazione di un impulso di accelerazione della durata dell'ordine di 0.5 s, seguito da una fase di decelerazione controllata fino all'arresto del sistema. Il valore tipico di riferimento è pari a 0.5 g, anche se, nella pratica sperimentale, le prove vengono spesso ripetute su un intervallo più ampio, generalmente compreso tra 0.3 g e 0.8 g, con step di incremento pari a 0.1 g, al fine di osservare il comportamento del carico sotto diverse intensità di sollecitazione.

Durante il test si distinguono due tipologie di risposta del sistema: le deformazioni elastiche, che si manifestano esclusivamente durante l'applicazione del carico dinamico e tendono a scomparire al termine della sollecitazione, e le deformazioni permanenti, che invece rimangono visibili anche dopo il rilascio delle forze applicate e rappresentano lo spostamento residuo della struttura.

La conformità allo standard viene infine determinata sulla base di una serie di criteri quantitativi. In particolare, il test è considerato superato se:

- lo spostamento permanente di qualsiasi punto del carico non eccede il 5% dell'altezza totale dell'unità pallettizzata;
- nella regione inferiore del carico, entro i primi 20 cm dal piano del pallet, lo spostamento residuo non supera i 4 cm;
- lo spostamento elastico massimo osservato durante la prova rimane inferiore al 10% dell'altezza complessiva del carico;
- non si verificano danni strutturali evidenti né perdite di prodotto.

Per l'esecuzione di tali prove viene tipicamente utilizzata una piattaforma di test dinamico, nota come ESTL (Enhanced Safety Test Laboratory 1.1), costituita da una slitta mobile sulla quale il pallet viene vincolato e sottoposto ai profili di accelerazione richiesti. L'analisi del comportamento del carico è supportata da sistemi di acquisizione video ad alta velocità, i cui dati vengono successivamente elaborati tramite software dedicati per la stima delle deformazioni elastiche e permanenti, consentendo così la verifica quantitativa della conformità allo standard EUMOS 40509.

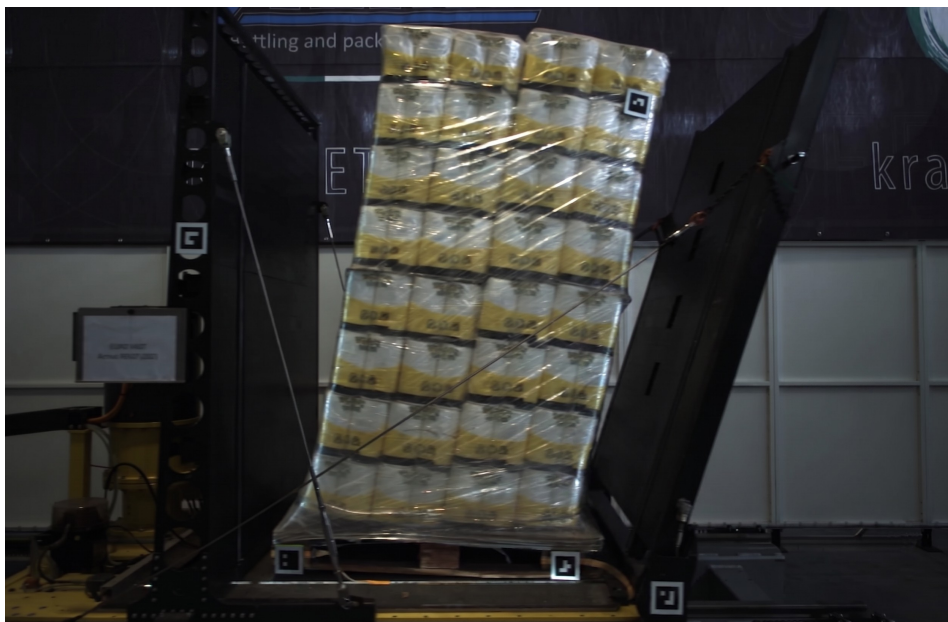


Figura 1.1: Frame della piattaforma di test dinamico ESTL, utilizzata per l'esecuzione delle prove secondo lo standard EUMOS 40509.

1.1.1 Criteri di ottimizzazione e stabilità del carico

Nel trasporto di merci su pallet, la disposizione del carico è quasi tanto importante quanto il pallet stesso. Non basta rispettare gli standard dimensionali: se il carico è organizzato male, lo spazio disponibile nel mezzo viene sfruttato solo in parte e la stabilità generale peggiora.

In pratica, la prima cosa che si cerca di fare è riempire il volume del veicolo nel modo più efficiente possibile. Questo significa ridurre gli spazi vuoti tra pallet e scegliere una disposizione che permetta di trasportare più unità senza superare i vincoli di peso o di ingombro. Anche la distribuzione del carico lungo il mezzo è importante, perché influisce direttamente sull'equilibrio del veicolo.

Un altro punto riguarda il pallet stesso. Non tutti i carichi si comportano allo stesso modo durante il trasporto: cambia molto in base a come sono disposte le scatole, a quanto è alto il pallet e a dove si trova il peso principale. Se il centro di massa è troppo alto, ad esempio, il carico tende a diventare più instabile quando il mezzo frena o cambia direzione.

Per limitare questi problemi si usano normalmente film estensibile, reggette o altri sistemi di fissaggio. In alcuni casi si lavora anche sulla disposizione interna delle scatole, cercando configurazioni che aumentino l'attrito o la compattezza del carico.

Infine, va considerato che durante il trasporto reale il pallet non è mai completamente statico: vibrazioni, accelerazioni e piccoli urti sono sempre presenti e possono influenzare il comportamento complessivo del carico. Per questo motivo, nelle fasi di test e validazione, diventa utile osservare come l'unità pallettizzata si deforma e si sposta nel tempo, così da identificare eventuali criticità nella configurazione adottata.

Questa esigenza di analisi temporale e spaziale del movimento ha portato allo sviluppo di strumenti di misura sempre più automatici, basati sull'elaborazione di dati visivi. In questo contesto, le tecniche di computer vision permettono di monitorare il comportamento del carico durante le prove dinamiche, fornendo informazioni quantitative sulle deformazioni e sugli spostamenti dell'unità pallettizzata.



Figura 1.2: Esempio di disposizione di carico su pallet all'interno di un camion, nello specifico sorretto da un sistema di cinghie di fissaggio.

1.2 Fondamenti di Computer Vision

La *Computer Vision* è un ambito dell'intelligenza artificiale che si occupa dello sviluppo di metodi per l'acquisizione, l'elaborazione e l'analisi di dati visivi, con l'obiettivo di estrarre informazioni utili da immagini o sequenze video.

In generale, i sistemi di Computer Vision operano su rappresentazioni digitali dell'informazione visiva e producono in output descrizioni strutturate della scena, che possono includere proprietà geometriche, statistiche o semantiche degli oggetti presenti.

1.2.1 Rappresentazione delle immagini e feature extraction

Un'immagine digitale in scala di grigi può essere modellata come una funzione discreta bidimensionale:

$$I(x, y)$$

dove x e y rappresentano le coordinate spaziali del pixel e $I(x, y)$ ne rappresenta l'intensità luminosa.

Nel caso di immagini a colori, la rappresentazione viene estesa a più canali, tipicamente nello spazio RGB:

$$I(x, y) = \{R(x, y), G(x, y), B(x, y)\}$$

Ogni canale contiene l'informazione relativa all'intensità della componente cromatica corrispondente.

Prima dell'elaborazione ad alto livello, le immagini vengono generalmente sottoposte a operazioni di preprocessing, tra cui filtraggio spaziale, riduzione del rumore, normalizzazione e miglioramento del contrasto.

L'estrazione di caratteristiche (*feature extraction*, 1.3) consiste nel trasformare i dati grezzi dell'immagine in un insieme di descrittori numerici. Le feature possono essere classificate in:

- **feature di basso livello:** bordi, angoli, gradienti, texture;
- **feature di medio livello:** forme, regioni, pattern locali;
- **feature di alto livello:** rappresentazioni semantiche associate a oggetti.

Nei metodi tradizionali, le feature venivano progettate manualmente (ad esempio SIFT [22], SURF [3], HOG [9]), mentre nei modelli basati su deep learning l'estrazione delle feature viene appresa automaticamente attraverso reti neurali, in particolare convoluzionali.

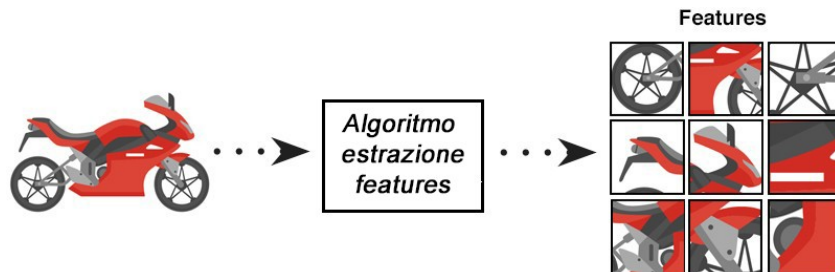


Figura 1.3: Esempio di processo di feature extraction di un'immagine.

1.2.2 Object Detection

L'*Object Detection* è il problema di identificare e localizzare oggetti appartenenti a classi predefinite all'interno di un'immagine o di un frame video. [12]

Dato un'immagine I , l'output del sistema è un insieme di oggetti:

$$O = \{o_1, o_2, \dots, o_n\}$$

dove ciascun oggetto o_i è definito da:

- una classe c_i ;
- una bounding box b_i .

La bounding box è tipicamente rappresentata come:

$$b_i = (x_i, y_i, w_i, h_i)$$

dove (x_i, y_i) rappresentano le coordinate del vertice superiore sinistro e w_i, h_i rispettivamente larghezza e altezza.

Gli approcci all'object detection possono essere suddivisi in due categorie principali:

- **two-stage detectors**: il problema viene decomposto in generazione di regioni candidate e successiva classificazione (es. R-CNN, Fast R-CNN, Faster R-CNN);
- **one-stage detectors**: la predizione di classi e bounding box avviene in un singolo passaggio (es. YOLO, SSD), con maggiore efficienza computazionale.

La valutazione delle prestazioni dei sistemi di object detection si basa comunemente su metriche standardizzate, tra cui l'Intersection over Union (IoU) e la mean Average Precision (mAP). L'IoU misura il grado di sovrapposizione tra la bounding box predetta e quella ground truth, ed è definita come il rapporto tra l'area di intersezione e l'area di unione delle due regioni. La mAP, invece, rappresenta la media delle precisioni calcolate su tutte le classi del dataset e costituisce uno degli indicatori principali per confrontare diversi modelli.

L'introduzione delle reti neurali convoluzionali ha permesso un miglioramento significativo delle prestazioni rispetto ai metodi basati su feature manuali, rendendo possibile l'utilizzo di sistemi di object detection in applicazioni real-time.

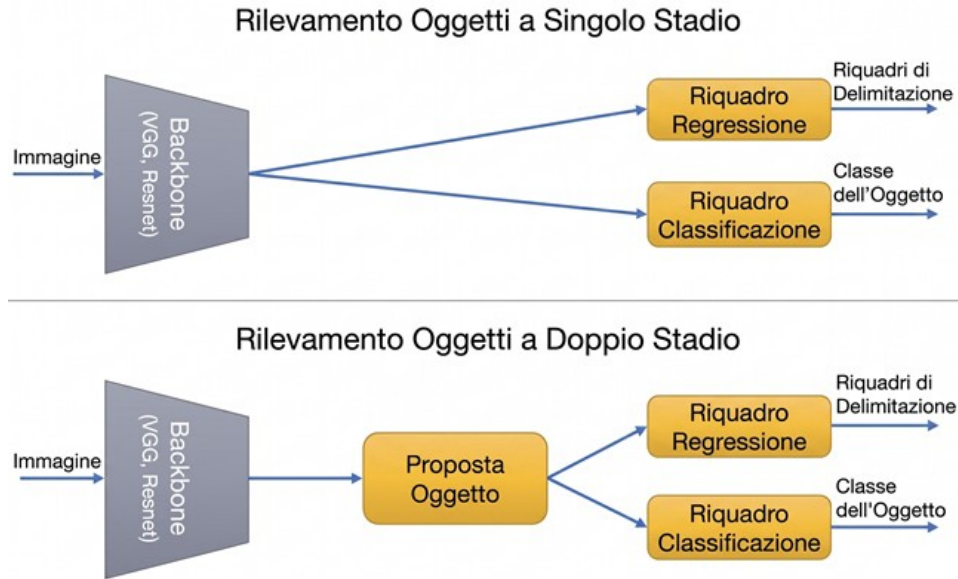


Figura 1.4: Esempio di processo di object detection in un'immagine. [5]

1.2.3 Approcci tradizionali nella Computer Vision

Prima della diffusione delle reti neurali profonde, i sistemi di Computer Vision si basavano principalmente su tecniche di elaborazione classica delle immagini. In questo contesto, l'identificazione degli oggetti avveniva attraverso pipeline composte da più fasi consecutive: acquisizione dell'immagine, preprocessing, estrazione manuale delle feature e classificazione finale.

L'obiettivo era individuare caratteristiche significative dell'immagine, come contorni, variazioni di intensità, movimento o regioni di interesse, utilizzando algoritmi deterministici e regole progettate manualmente.

Uno dei principali vantaggi degli approcci tradizionali consisteva nel basso costo computazionale. Tali metodi potevano essere eseguiti anche su hardware limitato e risultavano particolarmente efficaci in scenari controllati, caratterizzati da illuminazione stabile, sfondi statici e ridotta variabilità della scena. Tuttavia, queste tecniche presentavano notevoli limitazioni in ambienti dinamici o complessi, dove la presenza di rumore, ombre, occlusioni o variazioni prospettiche riduceva drasticamente l'affidabilità del sistema. [20]

1.2.3.1 Tecniche di preprocessing

Le operazioni di preprocessing rappresentano una fase preliminare fondamentale nei sistemi di visione artificiale tradizionali. Il loro obiettivo è miglio-

rare la qualità delle immagini acquisite e ridurre la presenza di informazioni indesiderate prima delle successive fasi di analisi ed elaborazione.

A seconda delle caratteristiche del problema affrontato, possono essere applicate diverse tecniche di preprocessing. Tra le più comuni vi sono la conversione dell'immagine in scala di grigi, utilizzata per semplificare la rappresentazione dei dati eliminando l'informazione cromatica, il filtraggio spaziale e le procedure di riduzione del rumore, impiegate per attenuare disturbi e artefatti presenti nell'immagine. Altre operazioni frequentemente adottate includono l'equalizzazione dell'istogramma, che migliora il contrasto e rende maggiormente evidenti i dettagli nelle diverse regioni dell'immagine, e il thresholding binario, utilizzato per separare gli oggetti di interesse dallo sfondo mediante la definizione di una soglia di intensità.

La conversione in scala di grigi permette di ridurre la dimensionalità dell'immagine eliminando l'informazione cromatica. Un pixel RGB può essere convertito mediante la relazione:

$$I = 0.299R + 0.587G + 0.114B \quad (1.1)$$

dove R , G e B rappresentano le componenti cromatiche del pixel originale. I coefficienti associati alle tre componenti cromatiche derivano dal modello percettivo umano, secondo lo standard ITU-R BT.601 [15].

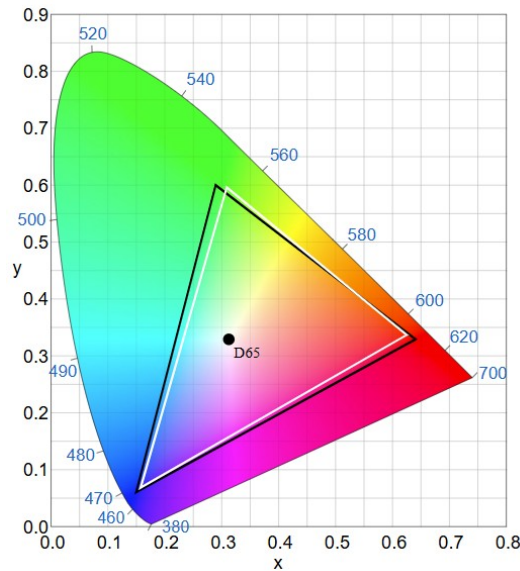


Figura 1.5: Diagramma dello spazio colore CIE 1931 che rappresenta lo spazio colore Rec. 601 (SDTV) all'interno del triangolo cromatico. Il nero indica i sistemi a 625 linee, mentre il bianco i sistemi a 525 linee. Lo standard Rec. 601 adotta l'illuminante D65 come punto di bianco di riferimento. [15]

Per la riduzione del rumore vengono frequentemente impiegati filtri convoluzionali. Nel caso del filtro Gaussiano, l'operazione di smoothing viene effettuata tramite una convoluzione con una funzione gaussiana bidimensionale:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (1.2)$$

dove σ controlla il grado di sfocatura applicato all'immagine.



Figura 1.6: Immagine originale (sinistra) e filtro gaussiano (destra).

Il thresholding binario consente invece di segmentare l'immagine separando foreground e background. Data un'intensità $I(x, y)$, la trasformazione binaria può essere definita come:

$$B(x, y) = \begin{cases} 1 & \text{se } I(x, y) > T \\ 0 & \text{se } I(x, y) \leq T \end{cases} \quad (1.3)$$

dove T rappresenta una soglia prefissata.



Figura 1.7: Immagine originale (sinistra) e thresholding binario (destra).

1.2.3.2 Background subtraction

Una delle tecniche più diffuse nei sistemi di visione tradizionali per il rilevamento di oggetti in movimento è la background subtraction. Tale approccio si basa sull'ipotesi che lo sfondo della scena rimanga relativamente statico nel tempo, mentre gli oggetti di interesse producano variazioni significative tra frame consecutivi. [4]

Il principio di funzionamento consiste nel confrontare il frame corrente con un modello di background precedentemente acquisito. Indicando con $I_t(x, y)$ l'immagine al tempo t e con $B(x, y)$ il background stimato, la differenza assoluta viene calcolata come:

$$D(x, y) = |I_t(x, y) - B(x, y)| \quad (1.4)$$

Successivamente viene applicata una soglia per ottenere una maschera binaria delle regioni considerate in movimento:

$$M(x, y) = \begin{cases} 1 & \text{se } D(x, y) > T \\ 0 & \text{se } D(x, y) \leq T \end{cases} \quad (1.5)$$

Le aree identificate come foreground possono quindi essere analizzate tramite operazioni morfologiche, come erosione e dilatazione, utili per eliminare rumore e migliorare la continuità degli oggetti rilevati.

Nonostante la semplicità implementativa e il ridotto costo computazionale, la background subtraction presenta numerose criticità. Il metodo risulta particolarmente sensibile a variazioni di illuminazione, ombre, riflessi, movimenti della telecamera, cambiamenti dello sfondo e occlusioni parziali.

In scenari industriali reali, caratterizzati da condizioni ambientali variabili e presenza simultanea di più oggetti, tali limitazioni riducono significativamente l'affidabilità del sistema.

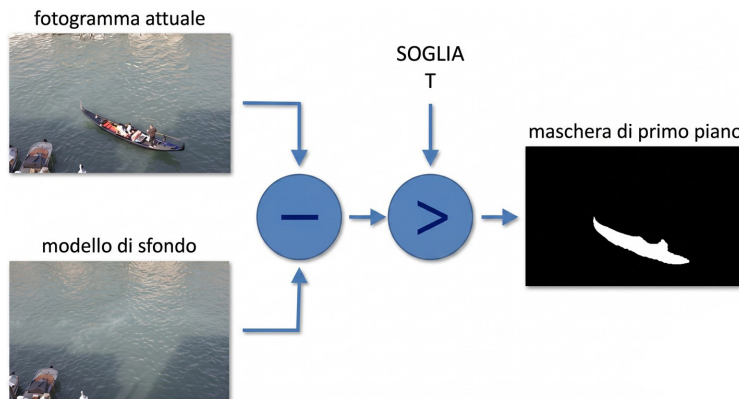


Figura 1.8: Esempio di background subtraction.

1.2.3.3 Segmentazione e saliency detection

Un ulteriore approccio tradizionalmente impiegato nella Computer Vision consiste nella segmentazione dell'immagine, ovvero nella suddivisione della scena in regioni omogenee associate a specifici oggetti o aree di interesse. Questa operazione permette di isolare gli elementi rilevanti e di semplificare le successive fasi di elaborazione.

Tra le tecniche di segmentazione maggiormente diffuse rientrano gli approcci basati sul rilevamento dei bordi, utilizzati per individuare i contorni degli oggetti, gli algoritmi di region growing, che aggregano pixel con caratteristiche simili, le tecniche di clustering per il raggruppamento automatico dei pixel e la watershed transformation, che interpreta l'immagine come una superficie topografica per identificare regioni distinte.

Con l'evoluzione delle reti neurali profonde, la segmentazione è stata progressivamente sostituita da modelli basati su CNN (*Convolutional Neural Network*), architettura la quale verrà approfondita nel capitolo 1.2.4, in grado di apprendere automaticamente rappresentazioni spaziali molto più robuste.

Tra le architetture maggiormente utilizzate per la segmentazione saliente si colloca U²-Net [24], una rete progettata per identificare automaticamente gli oggetti principali presenti nella scena attraverso la generazione di mappe di saliency.

L'idea fondamentale consiste nel produrre una maschera che assegni ad ogni pixel una probabilità di appartenenza all'oggetto di interesse. Questo approccio permette di separare foreground e background in maniera significativamente più accurata rispetto ai metodi classici basati su soglie o differenze temporali.

Nonostante i miglioramenti introdotti dalle reti di segmentazione, tali approcci non risultano ottimali nei problemi di object detection real-time, poiché non forniscono direttamente informazioni strutturate come bounding box e classificazione simultanea degli oggetti presenti nella scena.



Figura 1.9: Esempi di segmentazione ottenuti con U²-Net. [24]

1.2.4 Reti neurali convoluzionali

Le reti neurali convoluzionali, comunemente indicate con l'acronimo CNN (*Convolutional Neural Network*), rappresentano una delle principali evoluzioni nel campo della Computer Vision moderna. A differenza degli approcci tradizionali, le CNN non richiedono la progettazione manuale delle feature, ma apprendono automaticamente rappresentazioni gerarchiche direttamente dai dati durante la fase di addestramento [17].

L'introduzione delle CNN ha consentito di ottenere significativi miglioramenti in numerosi compiti di visione artificiale, tra cui la classificazione delle immagini, la segmentazione semantica, l'object detection, il riconoscimento facciale e il tracciamento degli oggetti.

L'idea alla base di queste architetture consiste nell'utilizzo di operazioni di convoluzione per estrarre progressivamente caratteristiche sempre più complesse dall'immagine. I primi livelli della rete apprendono tipicamente pattern semplici, come bordi e texture, mentre gli strati più profondi sono in grado di rappresentare forme, strutture e concetti semantici di livello superiore.

1.2.4.1 Operazione di convoluzione

La convoluzione rappresenta il nucleo fondamentale delle CNN e consiste nell'applicazione di un kernel (o filtro) sull'immagine di input per generare una feature map, in grado di descrivere localmente la presenza di determinati pattern visivi e strutture spaziali rilevanti.

Data un'immagine I e un kernel K , la convoluzione discreta bidimensionale è definita come:

$$S(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n) \quad (1.6)$$

Il kernel scorre sull'immagine eseguendo prodotti elemento per elemento e sommando i risultati, permettendo di estrarre pattern locali rilevanti e ridurre progressivamente la complessità della rappresentazione.

Questa operazione consente di evidenziare caratteristiche visive come bordi orizzontali e verticali, texture e variazioni di intensità. Le feature map ottenute possono essere interpretate come descrizioni intermedie dell'immagine, utilizzate dagli strati successivi per apprendere informazioni di livello sempre più elevato. Ogni filtro apprende automaticamente tali caratteristiche durante la fase di training.

1.2.4.2 Feature maps e gerarchia delle feature

L'output di ciascuna operazione di convoluzione prende il nome di feature map. Ogni feature map rappresenta una trasformazione dell'immagine di input che evidenzia la risposta di uno specifico filtro rispetto a determinate caratteristiche locali della scena. Rispetto ai pixel grezzi dell'immagine originale, le feature map forniscono quindi una rappresentazione intermedia più strutturata e informativa.

Nei primi layer della rete vengono apprese caratteristiche semplici e locali, come bordi, gradienti e variazioni di intensità. Queste rappresentazioni costituiscono la base su cui si sviluppa l'intero processo di astrazione e, procedendo verso gli strati più profondi, tali informazioni vengono combinate per formare pattern progressivamente più complessi.

La gerarchia delle feature segue una transizione da informazioni low-level a rappresentazioni high-level: nei layer intermedi emergono texture e forme semplici, mentre nei layer profondi le feature diventano sempre più astratte e semanticamente significative, fino a rappresentare oggetti o parti di essi.

Questa struttura gerarchica permette alla rete di apprendere automaticamente rappresentazioni sempre più discriminative dell'immagine, senza la necessità di progettare manualmente le feature. Inoltre, la combinazione di più feature map consente di catturare informazioni complementari, rendendo il modello più robusto a variazioni di scala, illuminazione e posizione degli oggetti.

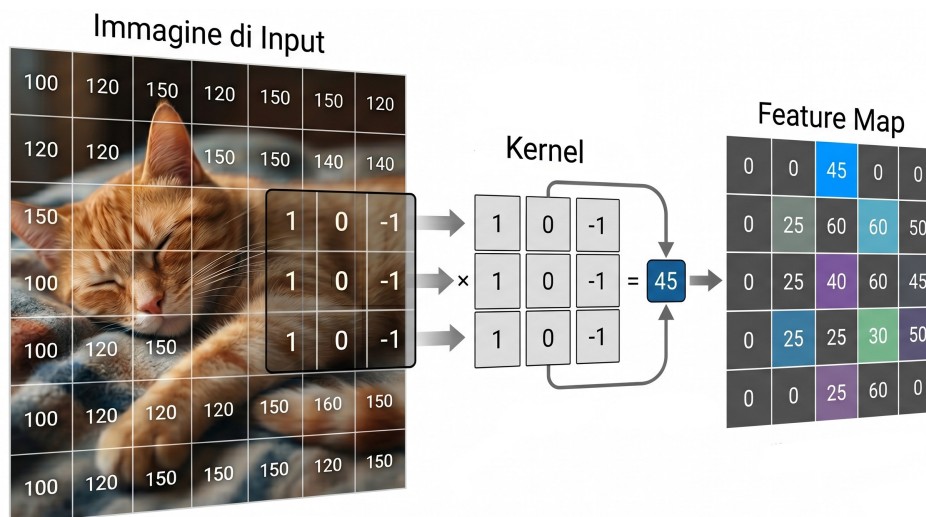


Figura 1.10: Applicazione di un kernel di convoluzione su un'immagine di input per generare una feature map.

1.2.4.3 Funzioni di attivazione

Dopo ogni convoluzione viene normalmente applicata una funzione di attivazione non lineare, necessaria per introdurre capacità rappresentative più complesse.

La funzione maggiormente utilizzata nelle CNN moderne è la ReLU (*Rectified Linear Unit*):

$$f(x) = \max(0, x) \quad (1.7)$$

La ReLU consente di ottenere diversi vantaggi rilevanti: riduce il problema del vanishing gradient [13], facilitando la propagazione del gradiente nelle reti profonde; velocizza il processo di addestramento grazie alla sua formulazione semplice e non saturante; e introduce una non linearità essenziale che permette alla rete di modellare funzioni complesse e dati ad alta dimensionalità.

A differenza di funzioni di attivazione tradizionali come la sigmoide, la ReLU non risulta derivabile nel punto $x = 0$. Nonostante ciò, questa caratteristica non rappresenta un problema significativo durante l'addestramento delle reti neurali, poiché i framework di deep learning utilizzano convenzionalmente una derivata definita nel punto di discontinuità.

La ReLU viene comunque preferita nelle CNN moderne perché presenta una struttura matematica molto semplice e richiede operazioni computazionali meno costose rispetto alla sigmoide o alla tangente iperbolica, che implicano il calcolo di funzioni esponenziali. Questo consente di ridurre i tempi di elaborazione e rendere più efficiente il training delle reti profonde.

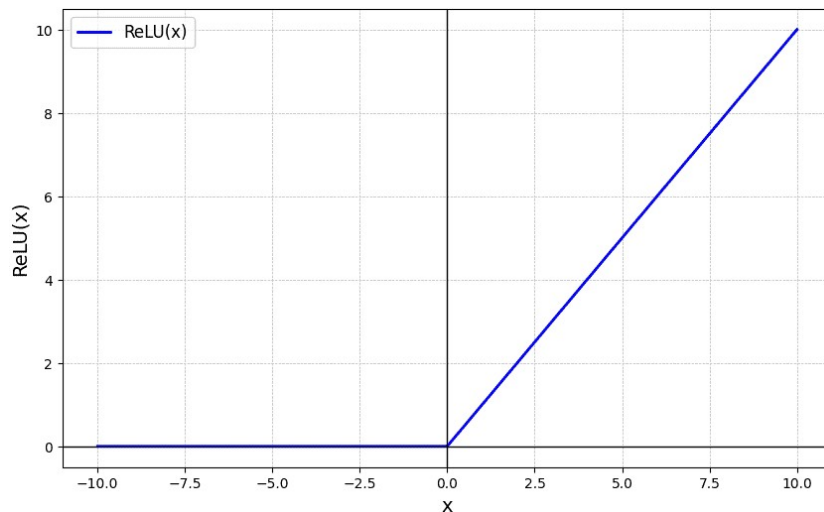


Figura 1.11: Grafico della funzione di attivazione ReLU.

1.2.4.4 Pooling

Le operazioni di pooling vengono utilizzate per ridurre la dimensionalità spaziale delle feature maps mantenendo le informazioni più rilevanti.

Nel *max pooling*, il valore massimo all'interno di una finestra locale viene selezionato come output:

$$y = \max(x_1, x_2, \dots, x_n) \quad (1.8)$$

Questo approccio consente di preservare le caratteristiche più attivate, come bordi o pattern particolarmente evidenti all'interno della feature map.

Un'altra tecnica diffusa è l'*average pooling*, nel quale viene calcolata la media dei valori presenti nella finestra locale:

$$y = \frac{1}{n} \sum_{i=1}^n x_i \quad (1.9)$$

A differenza del max pooling, l'average pooling produce una rappresentazione più smussata delle informazioni, mantenendo il contributo medio delle attivazioni invece dei soli valori massimi. Questa tecnica viene spesso utilizzata nelle fasi finali delle CNN, ad esempio tramite *Global Average Pooling*, per ridurre il numero di parametri e limitare l'overfitting.

La riduzione dimensionale introdotta dalle operazioni di pooling comporta una serie di vantaggi rilevanti per l'efficienza e la generalizzazione del modello: la diminuzione del costo computazionale, dovuta alla compressione delle rappresentazioni intermedie; una maggiore robustezza alle traslazioni e piccoli spostamenti degli oggetti all'interno dell'immagine; e infine una riduzione del rischio di overfitting, grazie alla minore complessità delle feature estratte.

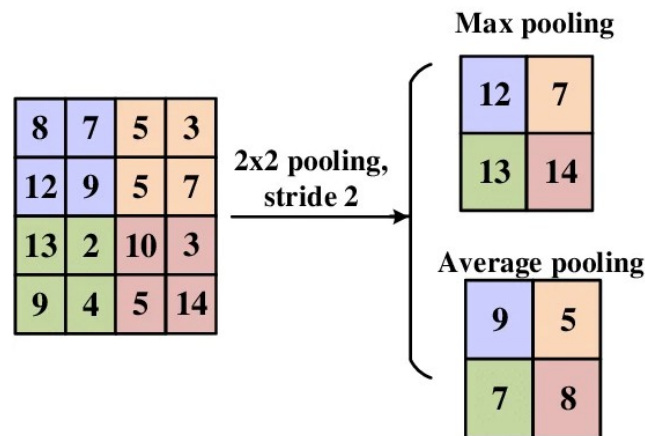


Figura 1.12: Esempio di Max Pooling e Average Pooling. [30]

1.2.4.5 Forward propagation e training

Durante la forward propagation, l'immagine attraversa sequenzialmente tutti i layer della rete fino alla produzione dell'output finale.

L'errore tra output predetto e valore reale viene calcolato tramite una loss function. Nel caso della classificazione multiclasse viene frequentemente utilizzata la cross-entropy:

$$L = - \sum_i y_i \log(\hat{y}_i) \quad (1.10)$$

dove:

- y_i rappresenta l'etichetta reale;
- $\hat{y}_i$ la probabilità predetta dal modello.

L'ottimizzazione dei pesi avviene tramite backpropagation e gradient descent:

$$w_{t+1} = w_t - \eta \frac{\partial L}{\partial w} \quad (1.11)$$

dove:

- w rappresenta il peso della rete;
- η il learning rate;
- L la loss function.

1.2.4.6 Limiti delle CNN classiche

Nonostante l'elevata efficacia, le CNN tradizionali nate per la classificazione delle immagini presentano alcune limitazioni nei problemi di object detection. In particolare, tali modelli producono un'unica etichetta globale per l'intera immagine, senza fornire informazioni sulla posizione o sull'estensione degli oggetti presenti.

Questo comporta l'impossibilità di gestire scenari con più istanze contemporanee, dove è necessario non solo riconoscere cosa è presente, ma anche localizzare ogni oggetto nella scena. Per superare questo limite sono state sviluppate architetture dedicate all'object detection, in grado di combinare classificazione e localizzazione all'interno dello stesso modello.

Tali approcci permettono quindi di identificare simultaneamente più oggetti e di determinarne la posizione spaziale, rendendo possibile l'analisi di scene più complesse. Tra le soluzioni più diffuse rientrano le architetture della famiglia YOLO, apprezzate per la capacità di eseguire inferenza in tempo reale mantenendo elevate prestazioni di accuratezza.

1.2.5 YOLO (You Only Look Once)

L'approccio *You Only Look Once* (YOLO) rappresenta una delle evoluzioni più significative nell'ambito dell'*Object Detection* in tempo reale. Introdotto nel 2016 da Joseph Redmon et al. [26], YOLO ha modificato radicalmente il paradigma tradizionale della rilevazione di oggetti, proponendo un modello unico in grado di eseguire simultaneamente classificazione e localizzazione degli oggetti presenti nell'immagine.

Uno degli obiettivi principali alla base dello sviluppo di YOLO era la realizzazione di sistemi di rilevamento estremamente rapidi per contesti critici, nei quali la velocità di elaborazione rappresenta un requisito fondamentale. Tra gli scenari applicativi iniziali vi erano anche contesti militari e di sorveglianza, nei quali risulta necessario identificare rapidamente bersagli o mezzi in movimento, come aerei, veicoli o droni, direttamente da flussi video in tempo reale. Le architetture tradizionali basate su region proposals risultavano troppo lente per tali applicazioni, mentre YOLO venne progettato proprio con l'obiettivo di ottenere inferenze ad alta velocità mantenendo una accuratezza sufficientemente elevata.

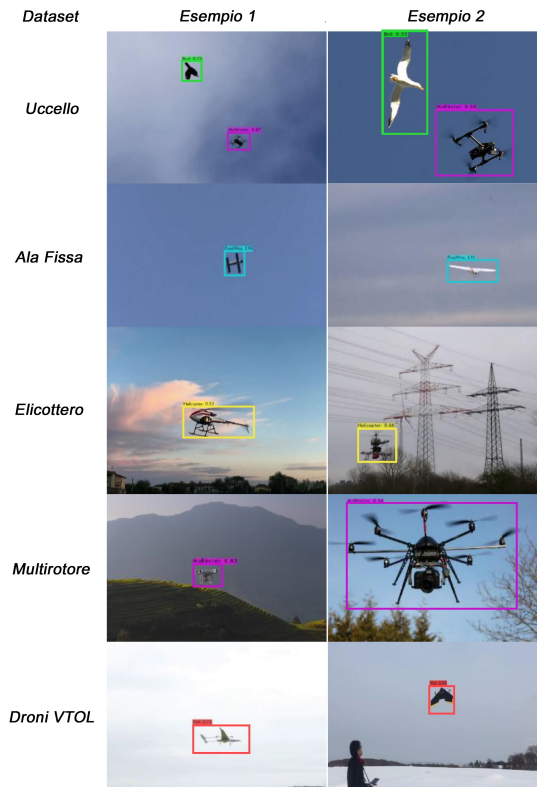


Figura 1.13: Rilevamento di droni in tempo reale mediante un modello YOLO. [8]

Prima dell'introduzione di YOLO, i principali algoritmi di *Object Detection*, come R-CNN e le sue evoluzioni (Fast R-CNN e Faster R-CNN), adottavano un approccio a più stadi: in una prima fase venivano generate possibili regioni candidate (*region proposals*), mentre in una seconda fase ciascuna regione veniva classificata separatamente mediante una rete neurale convoluzionale, come mostrato in figura 1.4. Sebbene tali approcci garantissero un'elevata accuratezza, il costo computazionale risultava particolarmente elevato, rendendo difficile l'utilizzo in applicazioni real-time. [12]

YOLO introduce invece un approccio *single-stage detector*, nel quale l'intera immagine viene elaborata una sola volta dalla rete. Da qui deriva il nome "You Only Look Once", che evidenzia la capacità del modello di effettuare la predizione in un'unica passata forward senza pipeline multi-stadio.

L'obiettivo principale è trasformare il problema della rilevazione di oggetti in un problema di regressione diretta, in cui la rete apprende a predire simultaneamente più informazioni a partire dall'immagine di input.

In particolare, il modello stima contemporaneamente la posizione degli oggetti tramite *bounding boxes*, la probabilità che un oggetto sia presente e la classe associata a ciascun oggetto rilevato.

Nella versione originale, l'immagine viene suddivisa in una griglia di dimensione $S \times S$. Ogni cella della griglia è responsabile della predizione di un certo numero di *bounding boxes* e delle relative probabilità di classe, come mostrato in figura 1.14.

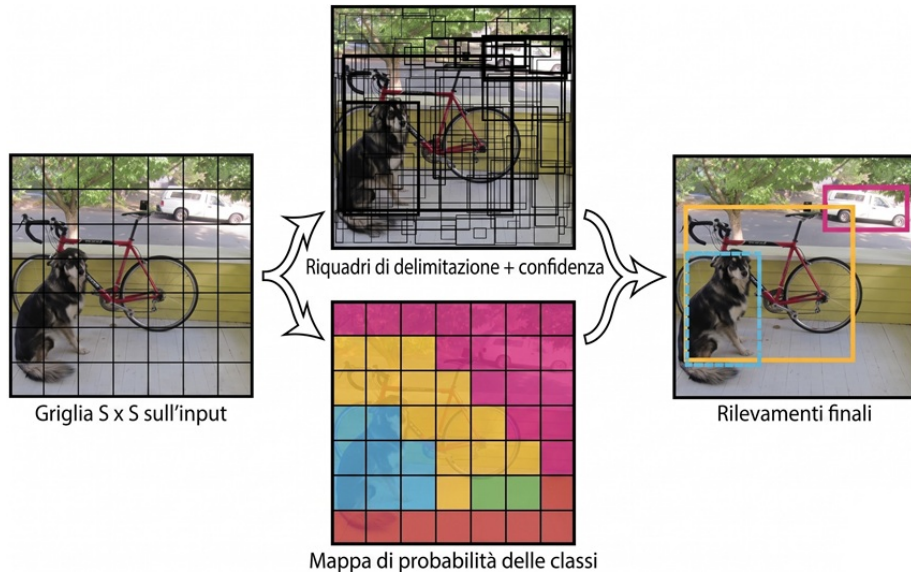


Figura 1.14: Suddivisione in celle e predizione di bounding boxes in YOLO. [2]

Una bounding box viene generalmente rappresentata tramite:

$$(x, y, w, h)$$

dove:

- x e y rappresentano le coordinate del centro;
- w e h rappresentano larghezza e altezza.

Per ogni bounding box, il modello produce inoltre un valore di *confidence score*, che misura sia la probabilità che un oggetto sia presente sia la qualità della localizzazione.

La metrica utilizzata per valutare la sovrapposizione tra bounding box predetta e reale è l'*Intersection over Union* (IoU):

$$\text{IoU} = \frac{|B_p \cap B_{gt}|}{|B_p \cup B_{gt}|} \quad (1.12)$$

dove B_p rappresenta la bounding box predetta e B_{gt} la bounding box reale (*ground truth*).

Un valore di IoU pari a 1 indica una sovrapposizione perfetta, mentre valori vicini a 0 indicano una scarsa corrispondenza.

Uno degli aspetti fondamentali di YOLO è la sua capacità di apprendere caratteristiche globali dell'immagine. A differenza dei metodi basati su region proposals, che analizzano separatamente porzioni locali, YOLO osserva l'intero contesto visivo durante la fase di inferenza. Questo consente di ridurre significativamente i falsi positivi e di ottenere velocità di elaborazione molto elevate.

Nel corso degli anni sono state sviluppate numerose evoluzioni dell'architettura originale:

- YOLOv2 introduce l'utilizzo delle *anchor boxes*, del *batch normalization* e di tecniche di training più avanzate;
- YOLOv3 migliora la rilevazione multi-scala mediante *Feature Pyramid Networks* (FPN);
- YOLOv4 e YOLOv5 introducono ulteriori ottimizzazioni architetturali e strategie di data augmentation;
- YOLOv7 e YOLOv8 migliorano ulteriormente accuratezza, velocità e semplicità di utilizzo.

Nel progetto presentato in questa tesi viene utilizzata la versione YOLOv8, sviluppata da Ultralytics [29], in quanto rappresenta uno degli stati dell'arte per applicazioni di object detection in tempo reale.

YOLOv8 introduce una serie di miglioramenti significativi rispetto alle versioni precedenti, ottimizzando l'equilibrio tra accuratezza descrittiva ed efficienza computazionale. Dal punto di vista architetturale, il modello abbandona le classiche strutture monolitiche a favore di una configurazione flessibile suddivisa in tre componenti macroscopiche principali, ciascuna ingegnerizzata con blocchi funzionali specifici, come mostrato in figura 1.15:

- **Backbone:** Ha il compito fondamentale di estrarre le mappe delle caratteristiche (*feature maps*) dall'immagine di input attraverso strati convoluzionali progressivamente più profondi. In YOLOv8, l'efficienza di questa fase è garantita dalla sinergia di tre moduli chiave:
 - **Conv (Convolutional Block):** Costituisce l'unità d'istanza elementare della rete. Non si limita alla sola operazione matematica di convoluzione, ma integra in sequenza uno strato di Convoluzione (per l'estrazione dei pattern visivi), un blocco di *Batch Normalization* (per la regolarizzazione e la stabilizzazione dei gradienti durante l'addestramento) e la funzione di attivazione non lineare *SiLU* (*Sigmoid Linear Unit*).
 - **C2f (Cross Stage Partial Bottleneck with 2 convolutions):** Rappresenta la principale innovazione architetturale del backbone di YOLOv8, evoluzione del modulo C3 presente nelle versioni precedenti. Questo blocco divide il flusso dei canali in due percorsi paralleli, combinando le caratteristiche *high-level* con i gradienti fluiti dai blocchi passati tramite connessioni di tipo *split* e *residual*. Tale approccio incrementa drasticamente il flusso di informazione del gradiente e arricchisce la rappresentazione semantica delle feature, mantenendo il modello computazionalmente leggero.
 - **SPPF (Spatial Pyramid Pooling - Fast):** Posizionato strategicamente al termine del backbone, questo modulo estrae caratteristiche multi-scala applicando in sequenza operazioni di *max-pooling* con finestre di dimensionamento fisso, per poi concatenarne i risultati. Il suo scopo primario è l'estensione del campo ricettivo (*receptive field*) della rete, permettendo ai neuroni più profondi di catturare il contesto globale dell'immagine e agevolando la corretta identificazione di oggetti di scale molto diverse tra loro senza penalizzare la velocità di calcolo.

- **Neck:** Agisce come una struttura di aggregazione e fusione delle caratteristiche geometriche e semantiche estratte a vari livelli del backbone. Sfruttando un'architettura ispirata alle reti *FPN* (*Feature Pyramid Network*) e *PANet* (*Path Aggregation Network*), il Neck combina i dettagli spaziali ad alta risoluzione provenienti dagli strati iniziali con le forti componenti semantiche astratte degli strati profondi. Questa operazione si articola mediante:
 - **Upsample (Sovracampionamento):** Un blocco che incrementa le dimensioni spaziali (altezza e larghezza) delle feature map più profonde e concentrate, raddoppiandone la risoluzione geometrica per renderle dimensionalmente compatibili con le mappe estratte nei livelli superiori del backbone.
 - **Concat (Concatenation):** Esegue l'unione fisica lungo la dimensione dei canali delle mappe geometriche native e di quelle appena sovracampionate. Questo processo di fusione multi-scala è essenziale per non perdere la precisione della localizzazione spaziale dell'oggetto, parametro critico per la successiva regressione delle coordinate.
- **Head (o Prediction):** Rappresenta lo strato deputato alla formulazione dell'output finale. YOLOv8 adotta una configurazione di tipo *Decoupled Head* (testa disaccoppiata), separando nettamente i rami decisionali: un sotto-percorso è specializzato esclusivamente nell'estrazione delle probabilità di classe (*Classification*), mentre l'altro è dedicato alla predizione geometrica dei *bounding boxes* (*Regression*). I moduli **Detect** operano in parallelo su tre scale differenti per garantire un'elevata sensibilità sia verso elementi microscopici sia verso porzioni macroscopiche della scena analizzata.

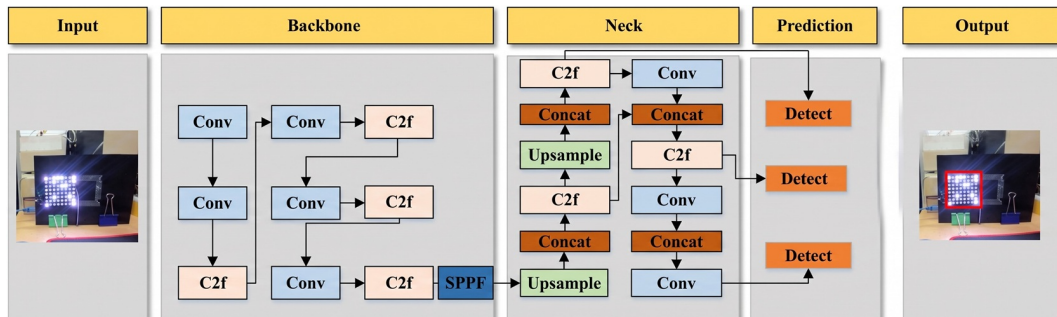


Figura 1.15: Architettura della rete YOLOv8 per il rilevamento degli oggetti.

Una delle principali innovazioni di YOLOv8 consiste nell'utilizzo di una detection head *anchor-free*. Nelle versioni precedenti, le predizioni venivano generate a partire da anchor boxes predefinite. YOLOv8 elimina tale dipendenza, rendendo il modello più semplice, più stabile durante l'addestramento e maggiormente adattabile a differenti dataset.

Il modello utilizza inoltre meccanismi avanzati di *feature aggregation* e strategie di training ottimizzate, ottenendo un migliore compromesso tra accuratezza e velocità.

Dal punto di vista operativo, YOLOv8 supporta diversi task di computer vision, tra cui object detection, instance segmentation, image classification, pose estimation e object tracking, mostrando una notevole versatilità applicativa all'interno di scenari eterogenei.

Per la valutazione delle prestazioni vengono generalmente utilizzate metriche quali precision, recall e mAP (*mean Average Precision*). La precision misura la proporzione di predizioni corrette rispetto al totale delle predizioni effettuate e si definisce come:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (1.13)$$

dove TP rappresenta i veri positivi e FP i falsi positivi. Questa metrica è utile per valutare quanto il modello sia affidabile nelle sue predizioni, penalizzando le rilevazioni errate.

La recall, invece, misura la capacità del modello di individuare tutti gli oggetti presenti e si definisce come:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (1.14)$$

dove FN indica i falsi negativi. Essa è fondamentale per comprendere quanto il sistema riesca a non perdere oggetti rilevanti all'interno della scena.

Infine, la mAP rappresenta una misura complessiva delle prestazioni del modello e viene calcolata come media dell'Average Precision su tutte le classi. L'AP deriva dall'area sotto la curva precision-recall, e la mAP può essere espressa come:

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N AP_i \quad (1.15)$$

dove N è il numero di classi e AP_i rappresenta l'Average Precision per la classe i .

Nel contesto della object detection vengono comunemente utilizzate due varianti della mAP: la $\text{mAP}@50$ e la $\text{mAP}@50-95$.

La $mAP@50$ calcola l'Average Precision considerando corrette le predizioni che presentano un valore di Intersection over Union (IoU) maggiore o uguale a 0.50. Questa metrica risulta meno restrittiva e fornisce una misura generale della capacità del modello di individuare correttamente gli oggetti.

La $mAP@50-95$, introdotta nel protocollo COCO, rappresenta invece una valutazione più rigorosa. In questo caso la mAP viene calcolata mediando l'Average Precision su più soglie di IoU comprese tra 0.50 e 0.95, con incrementi di 0.05. Formalmente:

$$mAP@50-95 = \frac{1}{10} \sum_{t=0.50}^{0.95} AP_t \quad (1.16)$$

Questa metrica penalizza maggiormente bounding box poco accurate e consente di valutare non solo la capacità del modello di rilevare un oggetto, ma anche la precisione della sua localizzazione.

Come metodologia di tracking è stato quindi scelto il modello *YOLOv8* per la sua elevata efficienza nel rilevamento e nel tracciamento degli oggetti appartenenti al dataset considerato. La rete garantisce infatti prestazioni accurate anche in scenari caratterizzati da variazioni di illuminazione e occlusioni parziali, mantenendo al contempo costi computazionali e tempi di inferenza estremamente ridotti. Tali caratteristiche lo rendono particolarmente adatto ad applicazioni *real-time*, nelle quali rapidità di elaborazione e affidabilità del tracking rappresentano requisiti fondamentali.

1.2.6 Object Tracking nelle sequenze video

L'*Object Tracking* è il processo che consente di seguire nel tempo uno o più oggetti all'interno di una sequenza video, mantenendone l'identità tra frame consecutivi.

A differenza dell'*Object Detection*, che opera su singole immagini, il tracking introduce una componente temporale: oltre alla rilevazione degli oggetti, il sistema deve stimarne il movimento e la continuità nel tempo.

Considerando un video come una sequenza di frame:

$$V = \{F_1, F_2, \dots, F_n\}$$

il sistema assegna ad ogni oggetto rilevato un identificatore univoco (*ID*), mantenuto nei frame successivi.

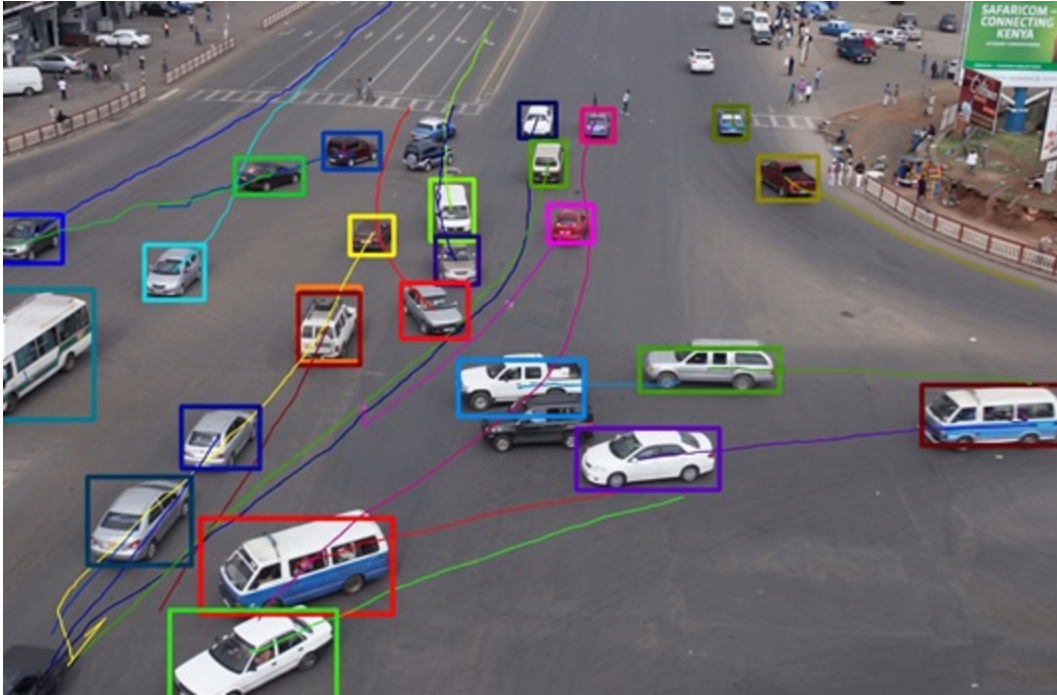


Figura 1.16: Tracking delle traiettorie percorse da veicoli all'interno di una scena video.

1.2.6.1 Problema dell'associazione temporale

Il compito principale dell'object tracking consiste nel determinare la corrispondenza tra le detections ottenute in frame differenti. Per ogni nuovo frame, il sistema deve stabilire se una detection appartenga a un oggetto già osservato, rappresenti un nuovo oggetto oppure corrisponda a un oggetto che è uscito dalla scena. Questa fase è fondamentale per garantire la continuità delle traiettorie nel tempo e mantenere un'identità coerente per ciascun oggetto tracciato.

Il problema dell'associazione temporale risulta particolarmente complesso in presenza di condizioni non ideali, che possono compromettere l'affidabilità delle corrispondenze tra frame consecutivi. Tra i principali fattori critici rientrano occlusioni parziali o totali, variazioni di illuminazione, cambiamenti di scala e prospettiva, movimenti rapidi degli oggetti e sovrapposizioni tra elementi differenti all'interno della scena. Tali condizioni aumentano l'ambiguità del matching e rendono più difficile mantenere un'identità stabile nel tempo.

1.2.6.2 Approccio Tracking-by-Detection

I moderni sistemi di tracking adottano generalmente il paradigma *tracking-by-detection*, che rappresenta uno degli approcci più diffusi nell'object tracking

contemporaneo. L'idea di base consiste nel ridurre il problema del tracking alla combinazione di due sotto-problemi più semplici e ben definiti, sfruttando l'affidabilità dei moderni detector basati su deep learning.

In questo approccio, il sistema non mantiene una modellazione esplicita continua del movimento, ma si basa su rilevazioni indipendenti effettuate frame per frame. Il problema viene quindi suddiviso in due fasi principali:

1. il detector individua gli oggetti presenti in ogni frame;
2. un algoritmo di associazione collega le detections nel tempo, mantenendo la coerenza delle identità tra frame consecutivi.

La fase di associazione temporale può essere supportata da diverse strategie basate su similarità spaziale, apparizione o modelli predittivi del movimento. La qualità complessiva del tracking dipende in modo diretto dall'accuratezza e dalla stabilità delle detections prodotte dal detector, poiché errori o inconsistenze in questa fase si propagano all'intero sistema.

1.2.6.3 Kalman Filter

Per stimare il movimento degli oggetti viene frequentemente utilizzato il *Kalman Filter*, un filtro ricorsivo capace di prevedere lo stato futuro di un oggetto a partire dalle osservazioni precedenti. [16]

Lo stato dinamico può essere rappresentato come:

$$x_k = [x, y, v_x, v_y]^T$$

dove:

- x, y rappresentano la posizione;
- v_x, v_y rappresentano la velocità.

Il Kalman Filter esegue iterativamente due operazioni principali:

- **predizione:** stima la posizione futura dell'oggetto;
- **correzione:** aggiorna la stima utilizzando le nuove osservazioni.



Figura 1.17: Predizione del movimento di pedoni tramite Kalman Filter.

1.2.6.4 Associazione delle detections

L'associazione tra detections e traiettorie esistenti viene spesso risolta mediante l'*Hungarian Algorithm*, un algoritmo di ottimizzazione che minimizza il costo globale di assegnazione.[18]

La similarità tra due bounding boxes può essere valutata tramite la metrica *Intersection over Union* (IoU) oppure mediante caratteristiche visive estratte tramite reti neurali profonde.

1.2.6.5 Principali algoritmi di tracking

Tra gli algoritmi di tracking più diffusi rientra **SORT** (*Simple Online Realtime Tracking*), un approccio che combina Kalman Filter e Hungarian Algorithm per ottenere elevate prestazioni in tempo reale con una ridotta complessità computazionale. Questo metodo si basa principalmente su informazioni geometriche e di movimento, risultando efficace in scenari semplici ma meno robusto in presenza di occlusioni o sovrapposizioni tra oggetti.

Una sua estensione diretta è **DeepSORT**, che introduce informazioni di apparenza estratte tramite reti neurali profonde. L'aggiunta di un modello di re-identificazione consente di migliorare significativamente la robustezza del tracking, soprattutto in presenza di occlusioni, poiché il sistema non si basa esclusivamente sulla distanza spaziale tra le detections.

Tra gli approcci più recenti ed efficaci si colloca **ByteTrack**, che introduce una strategia innovativa nella fase di associazione, includendo anche detections a bassa confidenza. Questa scelta permette di ridurre le perdite di tracking e di mantenere una maggiore continuità temporale delle traiettorie, anche in scenari complessi o rumorosi.

Infine, **BoT-SORT** (*Bag of Tricks SORT*) rappresenta un ulteriore miglioramento delle famiglie basate su SORT, integrando tecniche avanzate di associazione, raffinamento delle traiettorie e una combinazione di informazioni sia di movimento che di apparenza. Grazie a queste estensioni, il metodo risulta particolarmente robusto in contesti caratterizzati da elevata densità di oggetti e frequenti occlusioni.

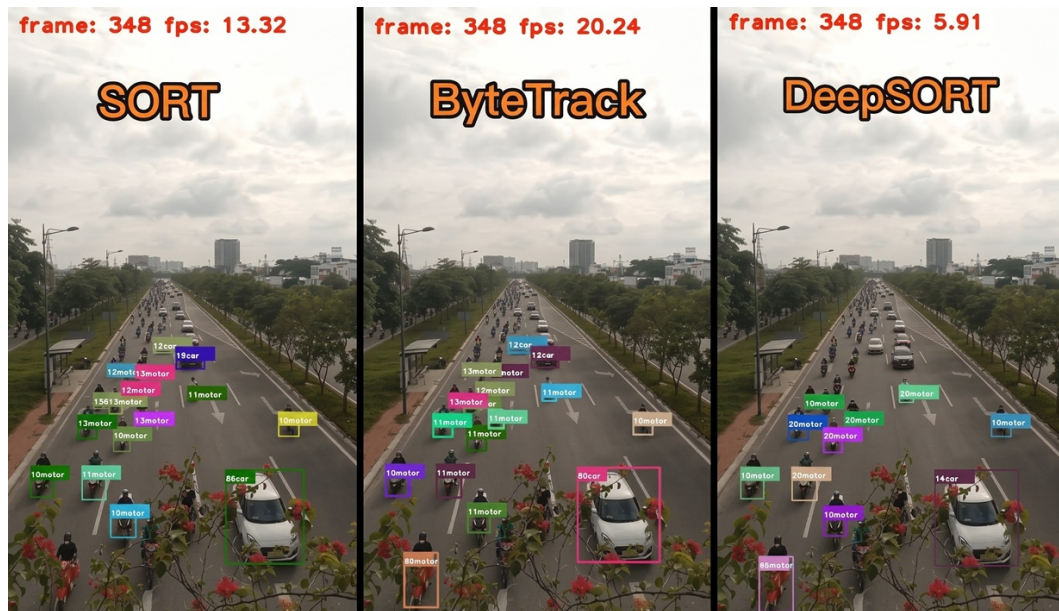


Figura 1.18: Differenze di rilevamenti tra SORT, ByteTrack e DeepSORT in un contesto di tracking multi-oggetto.

1.2.6.6 Integrazione con YOLOv8

Nel framework YOLOv8, il tracking può essere integrato direttamente all'interno della pipeline di inferenza. Il detector produce le bounding boxes per ciascun frame, mentre algoritmi come ByteTrack o BoT-SORT mantengono la continuità temporale degli oggetti.

Nel progetto sviluppato in questa tesi, l'object tracking viene utilizzato per monitorare la posizione e il movimento degli oggetti rilevati all'interno della scena video, consentendo un'analisi continua e coerente della sequenza acquisita.

1.3 Motori di Simulazione Fisica

La simulazione fisica in ambienti virtuali rappresenta uno dei pilastri fondamentali della computer grafica moderna e dei sistemi interattivi. L'obiettivo di un motore di simulazione fisica è quello di riprodurre, con un livello di approssimazione controllato, il comportamento dei corpi nello spazio tridimensionale, gestendo dinamica, collisioni, forze e vincoli.

Più che una replica fedele delle leggi fisiche reali, questi sistemi mirano a garantire stabilità numerica, coerenza e prestazioni in tempo reale, rendendoli adatti a contesti interattivi come videogiochi, robotica e simulazione ingegneristica.

1.3.1 Evoluzione dei motori fisici

I primi sistemi tridimensionali non disponevano di motori fisici general-purpose. La gestione della fisica era infatti integrata direttamente all'interno delle singole applicazioni, spesso attraverso soluzioni ad-hoc fortemente dipendenti dal contesto specifico e difficilmente riutilizzabili in altri progetti.

Questo approccio presentava diversi limiti che ne riducevano l'efficacia in scenari complessi o in continua evoluzione. In particolare, si osservavano una scarsa riusabilità del codice, difficoltà di scalabilità al crescere del numero di oggetti simulati, una manutenzione più complessa e un comportamento fisico non uniforme tra applicazioni diverse, con conseguenti differenze nei risultati ottenuti a parità di condizioni iniziali.

Con l'aumento della complessità delle scene 3D e dell'interazione tra oggetti, è quindi emersa la necessità di sistemi più generali, modulari e indipendenti dalle singole applicazioni. Questo ha portato alla nascita dei primi *physics engine* general-purpose, progettati per essere riutilizzabili, estendibili e facilmente integrabili in contesti differenti, standardizzando la simulazione fisica all'interno dei sistemi grafici moderni.

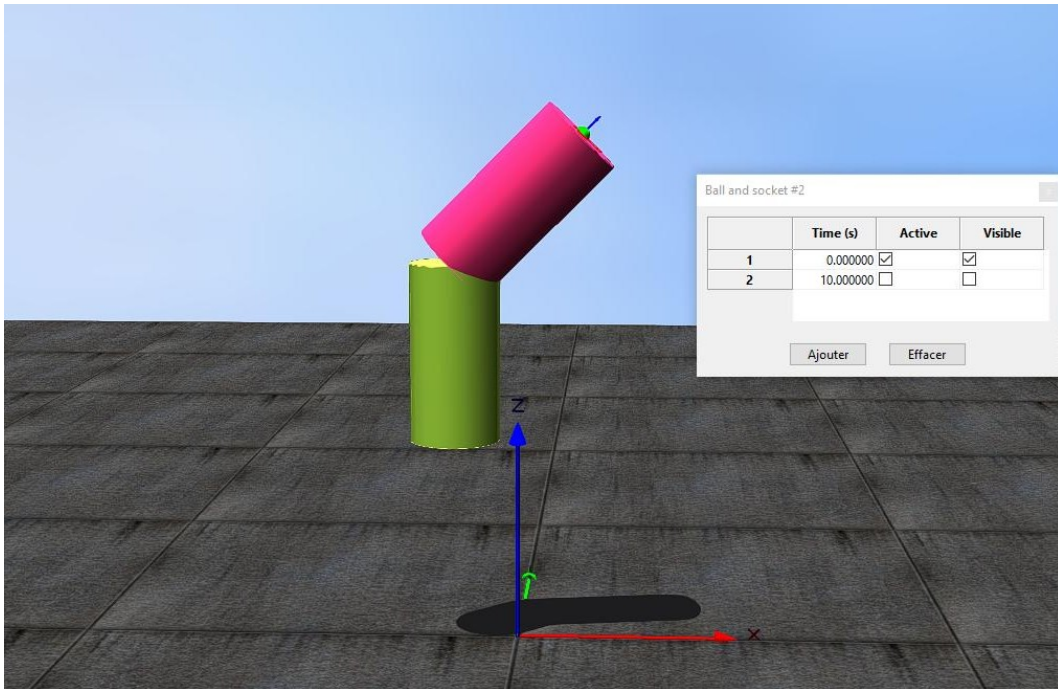


Figura 1.19: Esempio di simulazione fisica realizzata con Open Dynamics Engine (ODE), uno dei primi motori fisici open source utilizzati in ambito accademico e robotico per la simulazione di corpi rigidi e vincoli articolari.

1.3.2 Motori fisici moderni

I moderni motori di simulazione fisica sono progettati come librerie indipendenti e general-purpose, in grado di gestire in modo centralizzato l'interazione fisica tra oggetti all'interno di ambienti tridimensionali complessi. Essi forniscono un'astrazione ad alto livello che consente agli sviluppatori di integrare facilmente la simulazione fisica senza dover implementare manualmente i modelli matematici sottostanti.

Tali sistemi si occupano della gestione della dinamica dei corpi rigidi e deformabili, del rilevamento e della risoluzione delle collisioni tra oggetti, della definizione e del mantenimento di vincoli articolari (*joint constraints*) e dell'integrazione numerica delle equazioni del moto, necessaria per aggiornare lo stato del sistema nel tempo.

Tra gli aspetti più rilevanti nella progettazione di un motore fisico vi è il compromesso tra accuratezza fisica e prestazioni computazionali. In molti contesti real-time, come videogiochi o simulazioni interattive, la priorità non è ottenere una precisione fisica assoluta, ma garantire stabilità numerica, coerenza della simulazione e tempi di calcolo ridotti, anche a fronte di approssimazioni del modello fisico.

1.3.3 Bullet Physics Engine

In questo scenario si colloca Bullet Physics [6], un motore di simulazione fisica open source progettato per essere efficiente, modulare e adatto a contesti real-time. Grazie alla sua architettura flessibile, è ampiamente utilizzato sia in ambito accademico sia in applicazioni industriali, in particolare quando è necessario bilanciare realismo fisico e prestazioni.

Bullet è stato inizialmente sviluppato con un focus particolare sulla simulazione di corpi rigidi e sulla gestione efficiente delle collisioni, due aspetti fondamentali per garantire stabilità numerica e velocità di esecuzione. Nel tempo, il motore si è evoluto includendo funzionalità aggiuntive che ne hanno ampliato il campo di applicazione.

Tra queste rientrano la simulazione di *soft body*, utile per modellare materiali deformabili, sistemi di vincoli articolari per la gestione di strutture complesse e connesse, tecniche avanzate di collision detection e il supporto a scenari multi-corpo ad alta densità.

Uno dei punti di forza principali di Bullet è la capacità di mantenere buone prestazioni anche con un numero elevato di oggetti simulati simultaneamente, rendendolo particolarmente adatto a contesti in cui la fisica deve coesistere con altri processi computazionali, come il rendering in tempo reale o algoritmi di controllo e simulazione integrata.

Sebbene nato in ambito videoludico, Bullet è oggi ampiamente utilizzato anche in robotica, simulazione e ricerca scientifica, grazie alla sua natura open source e alla possibilità di integrazione in sistemi complessi.

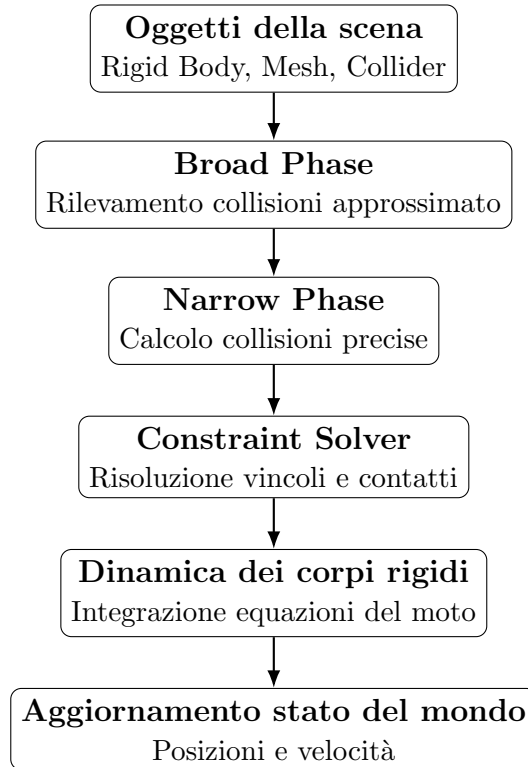


Figura 1.20: Architettura semplificata del Bullet Physics Engine e pipeline di simulazione fisica.

1.3.4 PyBullet

Nel contesto di questo progetto è stata utilizzata l'interfaccia Python di Bullet, nota come PyBullet [7]. PyBullet rappresenta un binding diretto del motore Bullet che consente di accedere alle funzionalità di simulazione fisica tramite Python, eliminando la necessità di interazione diretta con il codice C++. Questo lo rende particolarmente adatto a scenari di prototipazione rapida e a pipeline sperimentali, dove è fondamentale ridurre il tempo di sviluppo e facilitare l'integrazione con altri moduli software.

Uno degli aspetti più rilevanti di PyBullet è la sua capacità di integrarsi in modo naturale con ecosistemi basati su Python, in particolare con librerie di computer vision, machine learning e analisi dati. Questa caratteristica ha permesso di costruire una pipeline coerente, in cui la simulazione fisica e l'elaborazione delle informazioni visive possono coesistere all'interno dello stesso ambiente di esecuzione, riducendo la complessità architetturale del sistema.

La scelta di PyBullet si è quindi rivelata particolarmente favorevole nel contesto del progetto, poiché ha consentito di mantenere un flusso di lavoro uniforme tra le diverse componenti, evitando la necessità di bridge comples-

si tra linguaggi o framework differenti. Inoltre, la possibilità di interagire dinamicamente con la simulazione ha facilitato la sperimentazione di diversi scenari fisici in modo iterativo, migliorando l'efficienza del processo di sviluppo e validazione.

I principali vantaggi riscontrati nell'utilizzo di PyBullet possono essere riassunti nella semplicità di integrazione con Python e con librerie esterne, nella rapidità di sviluppo e testing grazie all'interfaccia ad alto livello, nella possibilità di iterazione veloce sui parametri fisici e sulle condizioni della simulazione, e nella buona documentazione accompagnata da una vasta comunità di supporto.

Oltre a questi vantaggi, PyBullet offre anche strumenti utili per la visualizzazione della simulazione e per il debug del comportamento fisico, permettendo di analizzare in modo intuitivo le interazioni tra oggetti e la coerenza del modello dinamico.

Ai fini del progetto, PyBullet è stato utilizzato per la costruzione dell'ambiente simulato e per l'analisi del comportamento dinamico degli oggetti, con particolare attenzione alla valutazione della stabilità del carico in condizioni controllate e riproducibili. L'utilizzo di un ambiente simulato ha inoltre permesso di effettuare test ripetibili e di controllare con precisione le variabili fisiche coinvolte, facilitando la validazione dei risultati ottenuti.

1.4 Fondamenti di analisi statistica e probabilistica

L'analisi quantitativa dei risultati sperimentali richiede strumenti statistici e probabilistici in grado di descrivere la variabilità dei dati, identificare relazioni tra variabili e valutare la significatività delle differenze osservate. Nel contesto della presente tesi, tali strumenti sono utilizzati per interpretare i risultati delle simulazioni fisiche, confrontare differenti configurazioni parametriche e stimare l'influenza dei parametri del modello sulle metriche di errore adottate.

1.4.1 Distribuzione normale e momenti statistici

Una distribuzione statistica descrive il modo in cui i valori osservati si distribuiscono all'interno di un insieme di dati. Tra le distribuzioni più importanti vi è la distribuzione normale, o gaussiana, ampiamente utilizzata per modellare fenomeni fisici e misure sperimentali grazie alla sua forma simmetrica attorno al valore medio.

La funzione di densità di probabilità della distribuzione normale è definita come:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \quad (1.17)$$

dove μ rappresenta la media della distribuzione e σ la deviazione standard.

Per caratterizzare quantitativamente una distribuzione vengono utilizzati i momenti statistici.

Il primo momento corrisponde alla media aritmetica:

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i \quad (1.18)$$

che rappresenta il valore medio delle osservazioni.

Il secondo momento centrato definisce invece la varianza:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2 \quad (1.19)$$

la quale misura la dispersione dei dati rispetto alla media. La deviazione standard $\sigma = \sqrt{\sigma^2}$ fornisce una misura diretta della variabilità dei dati nella stessa unità di misura delle osservazioni.

Ulteriori proprietà della distribuzione possono essere descritte tramite l'asimmetria (*skewness*) e la curtosi (*kurtosis*), che quantificano rispettivamente la simmetria della distribuzione e la concentrazione dei valori attorno alla media.

1.4.2 Analisi di sensibilità

L'analisi di sensibilità studia la variazione dell'uscita di un modello al variare dei parametri di ingresso. Tale approccio consente di identificare i parametri maggiormente influenti sul comportamento del sistema simulato e di comprendere la robustezza del modello rispetto alle perturbazioni dei suoi coefficienti fisici.

In forma generale, una funzione di risposta può essere espressa come:

$$y = f(x_1, x_2, \dots, x_n) \quad (1.20)$$

dove $x_1, \dots, x_n$ rappresentano i parametri del modello e y la grandezza di interesse. La sensibilità locale rispetto al parametro x_i può essere descritta mediante la derivata parziale:

$$S_i = \frac{\partial y}{\partial x_i} \quad (1.21)$$

che misura la variazione dell'uscita in risposta a una variazione infinitesima del parametro considerato.

1.4.3 Correlazioni

La correlazione rappresenta una misura della dipendenza tra due variabili quantitative. Uno degli indicatori più utilizzati è il coefficiente di correlazione lineare di Pearson, definito come:

$$\rho_{X,Y} = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y} \quad (1.22)$$

dove $\text{cov}(X, Y)$ è la covarianza tra le variabili:

$$\text{cov}(X, Y) = \frac{1}{N} \sum_{i=1}^N (x_i - \mu_X)(y_i - \mu_Y) \quad (1.23)$$

e σ_X , σ_Y rappresentano le rispettive deviazioni standard.

Il coefficiente $\rho_{X,Y}$ assume valori compresi tra -1 e 1 . Valori positivi indicano una correlazione diretta tra le variabili, valori negativi una correlazione inversa, mentre valori prossimi a zero suggeriscono assenza di correlazione lineare significativa.

1.4.4 ANOVA ad una e due vie

L'analisi della varianza (ANOVA, *Analysis of Variance*) è una tecnica statistica utilizzata per confrontare le medie di più gruppi e verificare se le differenze osservate siano attribuibili al caso oppure all'effetto di uno o più fattori sperimentali.

Nel caso dell'ANOVA a una via viene analizzato un singolo fattore, mentre l'ANOVA a due vie permette di studiare simultaneamente due fattori e la loro eventuale interazione. Il metodo si basa sul confronto tra la variabilità interna ai gruppi e quella tra gruppi differenti.

La statistica di test è definita come:

$$F = \frac{\text{varianza tra gruppi}}{\text{varianza intra-gruppo}} \quad (1.24)$$

Valori elevati della statistica F indicano che la variabilità tra gruppi è significativamente maggiore rispetto alla variabilità interna ai gruppi, suggerendo la presenza di un effetto statisticamente rilevante.

1.4.5 p-value e significatività statistica

Nell'ambito dei test statistici, il *p-value* rappresenta la probabilità di osservare risultati almeno altrettanto estremi rispetto a quelli ottenuti, assumendo vera l'ipotesi nulla H_0 .

Formalmente:

$$p = P(D \mid H_0) \quad (1.25)$$

dove D rappresenta l'insieme dei dati osservati. Un valore ridotto del p-value indica che i dati risultano poco compatibili con l'ipotesi nulla.

Convenzionalmente, si considera statisticamente significativo un risultato per cui:

$$p < 0.05 \quad (1.26)$$

In tal caso, l'ipotesi nulla viene rigettata a favore dell'ipotesi alternativa. È importante sottolineare che il p-value non rappresenta una misura dell'importanza pratica dell'effetto osservato, ma esclusivamente della sua significatività statistica.

1.4.6 Test di Kruskal–Wallis

Il test di Kruskal–Wallis costituisce un metodo non parametrico utilizzato per confrontare più gruppi indipendenti quando non risultano soddisfatte le ipotesi di normalità richieste dall'ANOVA.

A differenza dei test parametrici, il metodo opera sui ranghi dei dati anziché sui valori assoluti delle osservazioni. La statistica del test è definita come:

$$H = \frac{12}{N(N+1)} \sum_{i=1}^k \frac{R_i^2}{n_i} - 3(N+1) \quad (1.27)$$

dove N rappresenta il numero totale di osservazioni, k il numero di gruppi, R_i la somma dei ranghi del gruppo i e n_i il numero di osservazioni appartenenti al gruppo.

Valori elevati della statistica H indicano differenze significative tra i gruppi analizzati.

1.4.7 Analisi di convergenza

L'analisi di convergenza studia il comportamento di una sequenza di valori al crescere del numero di iterazioni o campioni considerati. In ambito numerico e statistico, un processo converge quando tende progressivamente verso un valore stabile.

Formalmente, una successione $\{x_n\}$ converge a un valore limite L se:

$$\lim_{n \rightarrow \infty} x_n = L \quad (1.28)$$

Nel contesto delle procedure di ottimizzazione parametrica, l'analisi di convergenza consente di verificare la stabilizzazione delle metriche di errore durante l'esplorazione dello spazio dei parametri e di valutare la robustezza delle soluzioni individuate.

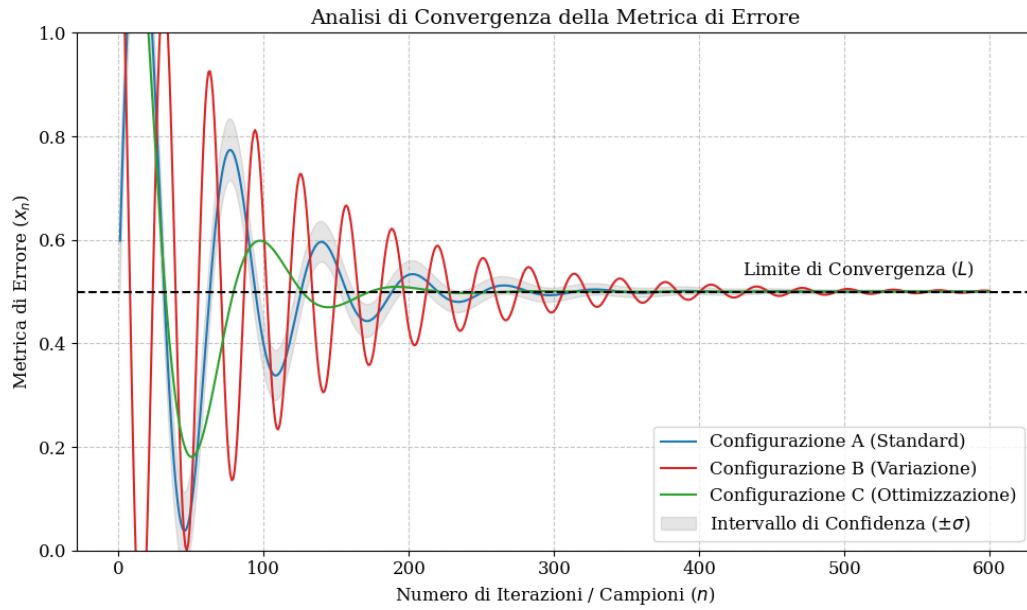


Figura 1.21: Analisi del processo di convergenza della metrica di errore x_n in funzione del numero di iterazioni o campioni n . Vengono messe a confronto tre diverse configurazioni parametriche (A, B e C) rispetto al vero limite di convergenza asintotico $L = 0.5$ (indicato dalla linea tratteggiata orizzontale).

L'area ombreggiata rappresenta l'intervallo di confidenza associato alla Configurazione A ($\pm\sigma$), evidenziando la riduzione dell'incertezza all'aumentare delle iterazioni.

Capitolo 2

Costruzione del dataset e sviluppo dei modelli YOLO

2.1 Primi approcci al problema di object detection

Come descritto nei capitoli precedenti, l'obiettivo principale del progetto consiste nello sviluppo di un sistema di object detection in grado di identificare con elevata precisione gli oggetti caricati sopra i pallet, così da poterne tracciare, per ogni frame, posizione, inclinazione e baricentro.

2.1.1 Background subtraction

Una prima soluzione esplorata durante le fasi iniziali del lavoro ha previsto l'utilizzo di tecniche classiche di computer vision basate sulla background subtraction, approfondita nel capitolo 1.2.3.2. Tale approccio si è però rivelato rapidamente inadatto al contesto analizzato. La scena presenta infatti un elevato livello di complessità visiva, caratterizzato da variazioni di illuminazione, presenza di texture eterogenee e continui cambiamenti nella disposizione degli oggetti.

Un ulteriore limite è rappresentato dall'assenza di un'immagine di sfondo pulita da utilizzare come riferimento per la sottrazione. Per cercare di compensare questa mancanza sono state introdotte diverse euristiche, combinate con tecniche di frame differencing, nel tentativo di isolare le regioni appartenenti agli oggetti in movimento. Nonostante ciò, i risultati ottenuti sono rimasti insufficienti ai fini del progetto.

Come mostrato in figura 2.1, il metodo produce un numero elevato di rilevazioni errate, generando bounding box associate a regioni prive di reale significato semantico. La precisione complessiva del sistema risulta quindi

CAPITOLO 2. COSTRUZIONE DEL DATASET E SVILUPPO DEI MODELLI YOLO

troppo bassa per consentire un tracciamento affidabile degli oggetti presenti sulla scena.

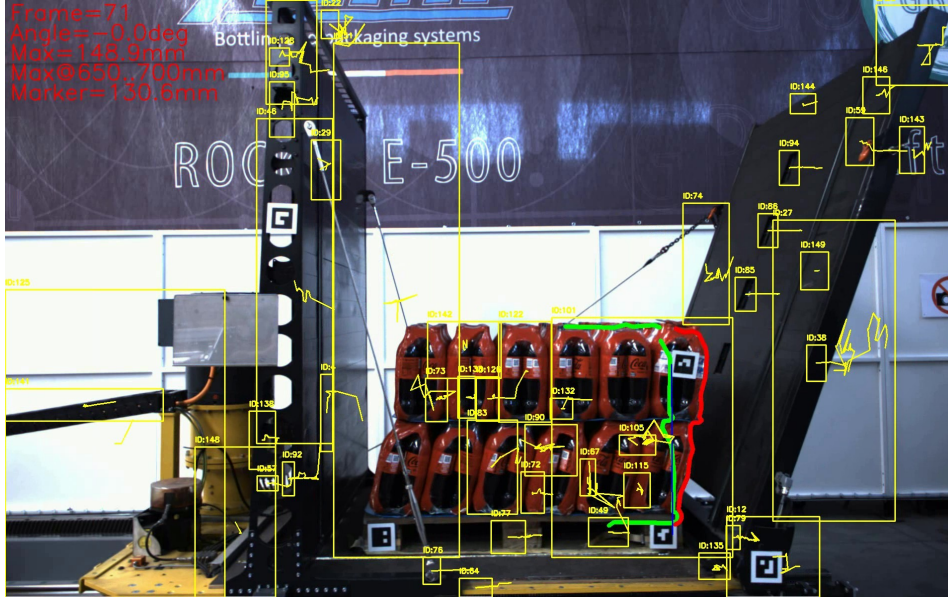


Figura 2.1: Frame elaborato tramite background subtraction

2.1.2 U²-Net

La prova successiva, a seguito del fallimento del metodo precedente, ha previsto l'utilizzo di una rete convoluzionale specializzata nella segmentazione semantica, nello specifico U²-Net, approfondita nel capitolo 1.2.3.3. Sebbene la rete sia nata con l'obiettivo di segmentare soggetti in primo piano, l'idea alla base di questo approccio è stata quella di sfruttarne la capacità di generare mappe di salienza, evidenziando le regioni più rilevanti all'interno dell'immagine.

Anche in questo caso i risultati ottenuti sono stati deludenti. La complessità del modello, unita all'elevata potenza di calcolo richiesta per la gestione delle operazioni di inferenza e delle elaborazioni intermedie, ha comportato una velocità di elaborazione inferiore al frame per secondo. Tale prestazione si è rivelata non accettabile per un'analisi video in tempo reale.

Anche qualora i due metodi precedentemente descritti avessero prodotto risultati soddisfacenti, sarebbe comunque rimasto un problema fondamentale da risolvere: la necessità di individuare ed etichettare gli oggetti presenti nella scena, e non soltanto rilevarne la presenza.

2.1.3 YOLOv8

A fronte dei risultati poco soddisfacenti ottenuti mediante gli approcci tradizionali, si è resa necessaria l'adozione di modelli appartenenti alla categoria dei one-stage detectors, considerata la necessità di individuare e localizzare gli oggetti presenti nella scena in maniera rapida ed efficiente.

La scelta è ricaduta su YOLOv8, ultima evoluzione della nota famiglia di modelli YOLO, descritta nel dettaglio nel capitolo 1.2.5. La prima fase di sperimentazione è stata condotta utilizzando un modello pre-addestrato sul dataset COCO [21], uno dei riferimenti principali nell'ambito dell'object detection, contenente numerose categorie di oggetti di uso comune.

Nello specifico, è stata impiegata la variante medium del modello, composta da circa 25 milioni di parametri, scelta in quanto rappresenta un buon compromesso tra accuratezza e velocità di inferenza.

I risultati ottenuti si sono dimostrati nettamente più promettenti rispetto a quelli prodotti dai metodi precedentemente analizzati, evidenziando un aumento significativo della precisione complessiva. Nonostante ciò, il modello standard addestrato su COCO non include classi realmente assimilabili agli oggetti di interesse del caso studio, costituiti principalmente da bottiglie, scatole e pallet. Di conseguenza, pur disponendo di una base iniziale solida, le prestazioni effettive risultano ancora limitate, come mostrato in figura 2.2.

Per questo motivo si è reso necessario procedere con una fase di fine-tuning del modello, effettuata tramite addestramento su un dataset specificamente costruito per il contesto applicativo considerato, così da adattare i pesi della rete alle caratteristiche visive degli oggetti da rilevare.

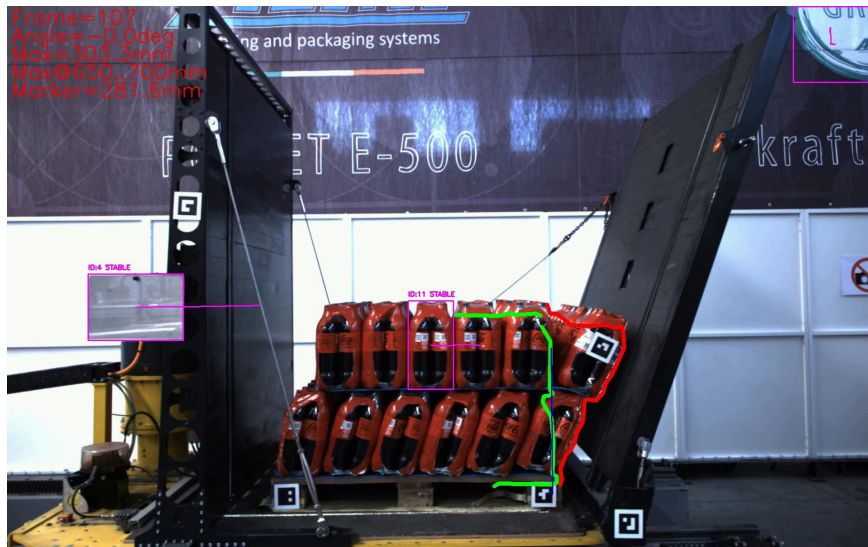


Figura 2.2: Frame elaborato YOLOv8m pre-addestrato su COCO

2.2 Costituzione del dataset

Il dataset è composto nella sua totalità da video privati registrati presso ACMI [1], azienda collaboratrice del progetto, la quale si occupa principalmente di fornire soluzioni per l'imbottigliamento e il packaging.

I video sono stati acquisiti interamente mediante una telecamera fissa, posizionata lateralmente rispetto alla slitta di crash test utilizzata per simulare le condizioni di movimentazione dei pallet all'interno di una stanza appositamente predisposta.

Il quantitativo complessivo di video raccolti ammonta a circa 450 test, effettuati con diverse soluzioni di imballaggio, differenti configurazioni del carico e vari livelli di accelerazione. I video relativi a prodotti differenti sono complessivamente più di 100.

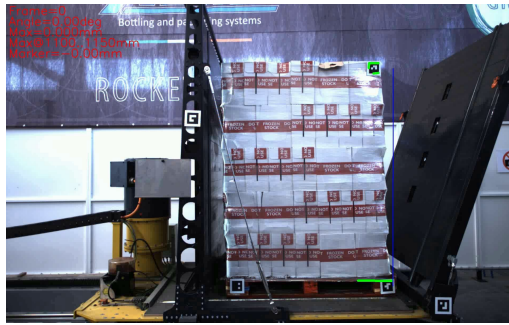
2.2.1 Selezione dei video per l'addestramento

Il processo di selezione dei video candidati per l'addestramento dei modelli è stato effettuato visionando manualmente l'intero dataset, al fine di identificare quali video possano contribuire maggiormente alla diversificazione del set di addestramento, e quindi al miglioramento delle prestazioni dei modelli cercando di favorire la generalizzazione di essi. Precisamente la tipologie di video esistenti possono essere ricondotte a due principali categorie: pallet avvolti con carta e pallet avvolti con pellicola film LDPE.

Va da sé che la soluzione di imballaggio con carta non risulti difficilmente analizzabile tramite tecniche di object detection, in quanto la carta avvolge completamente il carico, rendendo impossibile identificare i singoli oggetti presenti al suo interno. Per questo motivo, la maggioranza dei video selezionati per l'addestramento sono stati quelli relativi a pallet avvolti con pellicola film LDPE, in quanto questa soluzione di imballaggio consente di identificare con maggiore chiarezza i singoli oggetti presenti al suo interno, pur presentando comunque una certa complessità visiva dovuta alla trasparenza e alle riflessioni della pellicola.

Nel complesso, sono stati selezionati 66 video, caratterizzati da un'ampia varietà di prodotti, configurazioni di carico e condizioni di acquisizione. Questo insieme fornisce un dataset di addestramento sufficientemente diversificato e rappresentativo del contesto applicativo considerato, includendo comunque una quota limitata di esempi con imballaggio in carta, al fine di preservare una certa capacità di generalizzazione anche verso scenari più complessi.

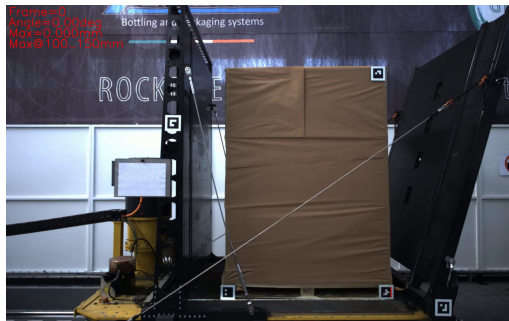
CAPITOLO 2. COSTRUZIONE DEL DATASET E SVILUPPO DEI MODELLI YOLO



(a) Esempio con pellicola film LDPE



(b) Esempio con pellicola film LDPE e due pallets uno sopra l'altro



(c) Esempio con imballaggio in carta



(d) Esempio con pellicola film LDPE e scarsa illuminazione

Figura 2.3: Esempi di frame selezionati per l'addestramento dei modelli

2.2.2 Estrazione dei frame dai video

Una volta selezionati i video da utilizzare per l'addestramento, è stato necessario procedere con l'estrazione dei singoli frame da ciascun video, al fine di creare un dataset di immagini su cui addestrare i modelli.

Il processo è stato automatizzato tramite l'utilizzo di uno script, il quale ha permesso di estrarre i frame a intervalli regolari, garantendo una copertura adeguata dell'intero video e una rappresentazione equilibrata delle diverse fasi del movimento dei pallet.

L'intero processo è descritto dettagliatamente dall'algoritmo 1, il quale prevede l'iterazione su ciascun video presente nella directory specificata, l'apertura del flusso video, la lettura dei frame e il salvataggio di quelli estratti in una directory di output dedicata per ogni video portando ad un totale di 735 frame estratti con una media di circa 12 frame per video.

Algoritmo 1 Estrazione di frame dai video per la costruzione del dataset

Require: D : directory dei video, $\Delta = 15$: intervallo di campionamento dei frame

Ensure: T : numero totale di frame estratti

```
1: procedure FRAMEEXTRACTION( $D, \Delta$ )
2:    $V \leftarrow$  file in  $D$  con estensione  $\{.avi, .mp4\}$ 
3:   if  $V = \emptyset$  then
4:     return
5:    $T \leftarrow 0$  ▷ conteggio globale dei frame salvati
6:   for  $v \in V$  do
7:     apertura dello stream video  $S$  da  $v$ 
8:     creazione della directory di output  $O_v$ 
9:      $i \leftarrow 0$  ▷ indice del frame corrente
10:     $k \leftarrow 0$  ▷ indice dei frame salvati per video
11:    while  $S$  ha frame successivo do
12:      lettura del frame  $f$  da  $S$ 
13:      if  $f = \emptyset$  then
14:        break
15:      if  $i \bmod \Delta = 0$  then
16:        salvataggio del frame  $f$  in  $O_v$ 
17:        con nome progressivo basato su  $k$ 
18:         $k \leftarrow k + 1$ 
19:         $T \leftarrow T + 1$ 
20:       $i \leftarrow i + 1$ 
21:    chiusura dello stream video  $S$ 
22:  return  $T$ 
```

2.2.3 Annotazione delle immagini

Una volta estratti i frame i quali costituiranno il dataset di addestramento, è stato necessario procedere con l'annotazione manuale di ciascuna immagine, al fine di fornire ai modelli le bounding box e le classi corrette per ogni oggetto presente nella scena.

Per scelta implementativa si è deciso di prevedere due classi distinte: una per il pallet e una per gli oggetti generici. Tale scelta è stata dettata dal fatto che l'obiettivo finale fosse quello di identificare il movimento degli oggetti durante il moto, piuttosto che classificare la tipologia di oggetto presente nella scena. Per questo motivo si è preferito adottare una classificazione più generica, così da favorire la capacità di generalizzazione dei modelli anche nei confronti di oggetti non presenti nel dataset di addestramento.

Il processo di annotazione è stato effettuato tramite Label Studio [28], uno strumento open-source che consente di creare e gestire dataset annotati per applicazioni di machine learning e computer vision.

Attraverso tale software è stato possibile associare manualmente a ciascun frame le relative bounding box, identificando gli oggetti presenti nella scena e assegnando la classe corrispondente. In particolare, per ogni immagine sono state annotate le coordinate spaziali degli oggetti, permettendo così ai modelli di apprendere sia la posizione sia la tipologia degli elementi presenti.

L'utilizzo di Label Studio ha inoltre semplificato la gestione del dataset, consentendo di esportare automaticamente le annotazioni nel formato richiesto da YOLOv8, riducendo il rischio di errori manuali durante la fase di preparazione dei dati.

Le annotazioni generate sono state salvate in due differenti formati, a seconda della tipologia di modello da addestrare: Horizontal Bounding Box (HBB) e Oriented Bounding Box (OBB).

Nel formato HBB, adottato dal formato standard YOLO, ogni bounding box è rappresentata tramite cinque valori normalizzati rispetto alle dimensioni dell'immagine: l'indice di classe, la coordinata del centro della bounding box lungo l'asse x , la coordinata del centro lungo l'asse y , la larghezza e l'altezza. In forma compatta, una annotazione HBB è quindi espressa come:

class_index x_center y_center width height

Un esempio di annotazione HBB è il seguente:

0 0.44 0.24 0.042 0.13

Nel formato OBB, invece, utilizzato nei modelli YOLO Oriented Bounding Box, ogni bounding box è rappresentata dai quattro vertici del rettangolo orientato, con coordinate normalizzate tra 0 e 1. In questo caso, l'annotazione

CAPITOLO 2. COSTRUZIONE DEL DATASET E SVILUPPO DEI MODELLI YOLO

è composta dall'indice di classe seguito dalle coordinate dei quattro punti del bounding box:

```
class_index x1 y1 x2 y2 x3 y3 x4 y4
```

Un esempio di annotazione OBB è il seguente:

```
0 0.42 0.17 0.46 0.17 0.46 0.31 0.42 0.31
```

L'utilizzo del formato OBB consente di rappresentare in modo più accurato oggetti inclinati o non allineati agli assi dell'immagine, migliorando la precisione della localizzazione rispetto alle bounding box tradizionali.

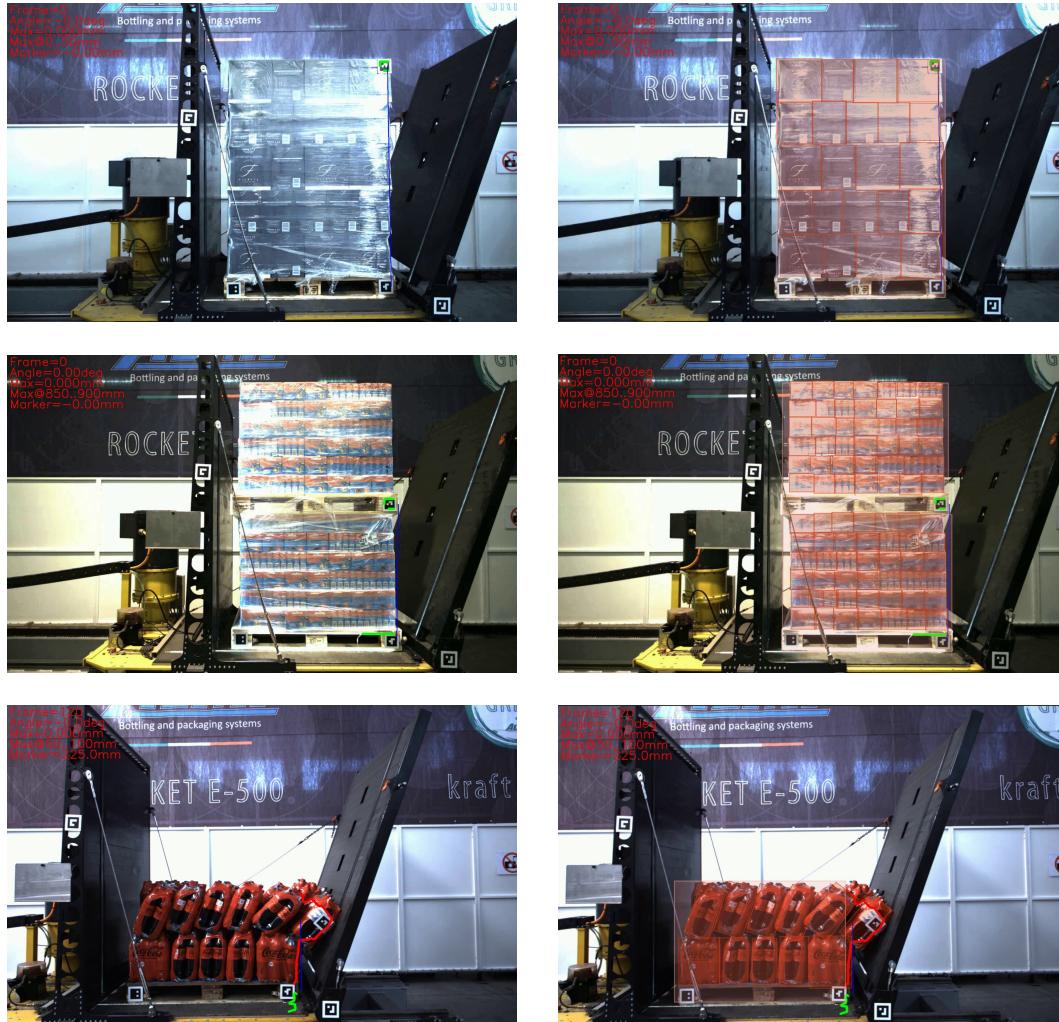


Figura 2.4: Esempi di frame etichettati tramite Label Studio, prima e dopo

2.3 Addestramento dei modelli YOLO

Come detto in precedenza, partendo dal processo di annotazione del dataset, sono state etichettate complessivamente 735 immagini. Da queste annotazioni sono stati generati 17.214 bounding box, di cui 16.407 associati alla classe *Objects* e 807 relativi alla classe *Pallets*.

Una volta esportate le label nel formato originario, si è reso necessario sviluppare uno script di preprocessing finalizzato alla costruzione di un dataset strutturato e compatibile con il formato richiesto da YOLO. Tale procedura consente di organizzare in maniera coerente immagini e annotazioni, suddividendole in insiemi di training e validation e garantendo una corrispondenza univoca tra ciascuna immagine e il relativo file di label.

Inoltre, il processo include la generazione automatica del file di configurazione `data.yaml`, fondamentale per l'utilizzo del dataset all'interno del framework YOLO. Questo file definisce la struttura del dataset, i percorsi delle directory e le classi del problema di rilevamento, permettendo al modello di interpretare correttamente i dati durante le fasi di training e validazione.

Il contenuto del file `data.yaml` generato è il seguente:

```
---
path: # percorso del dataset
train: train/images
val: val/images
test: # opzionale

# Task
task: obb | hbb

names:
  0: Objects
  1: Pallets
---
```

Viene di seguito riportato l'algoritmo 2 che descrive in dettaglio il processo di costruzione del dataset a partire dalle immagini e dalle annotazioni, con particolare attenzione alla gestione dei file, alla suddivisione in training e validation e alla generazione del file di configurazione.

Inoltre, l'algoritmo 3 riporta l'essenza di un tipico processo di training per un modello di object detection basato su YOLO, evidenziando le fasi chiave dell'addestramento, dalla preparazione del dataset alla gestione delle epoche e al salvataggio del modello finale.

Algoritmo 2 Costruzione del dataset di addestramento da immagini e label

Require: I : insieme delle immagini, L : insieme delle label, O : directory di output del dataset, $r = 0.8$ (rapporto train/validation)

Ensure: n_{train}, n_{val}

```

1: procedure BUILDDATASET( $I, L, O, r$ )
2:   if  $L = \emptyset$  then
3:     return
4:    $C \leftarrow$  classi (se disponibili)
5:   ricreazione delle directory  $O/train$  e  $O/val$ 
6:    $M \leftarrow \emptyset$  ▷ mappa di indicizzazione:
baseline  $\rightarrow$  path immagine
7:   for  $i \in I$  do
8:      $b_i \leftarrow$  baseline( $i$ )
9:      $M[b_i] \leftarrow i$ 
10:   $S \leftarrow$  shuffle( $L$ ) ▷ randomizzazione del dataset
11:   $k \leftarrow \lfloor r \cdot |S| \rfloor$  ▷ punto di split train/val
12:   $S_{train} \leftarrow S[1..k]$ 
13:   $S_{val} \leftarrow S[k + 1..|S|]$ 
14:  function COPYSPPLIT( $S_{split}, type$ )
15:     $n \leftarrow 0$ 
16:    for  $s \in S_{split}$  do
17:       $b_s \leftarrow$  clean( $s$ ) ▷ normalizzazione del nome file
18:      if  $b_s \in M$  then
19:        copy  $M[b_s] \rightarrow O/type/images$ 
20:        copy  $s \rightarrow O/type/labels$ 
21:         $n \leftarrow n + 1$ 
22:      else
23:        skip ▷ assenza di corrispondenza
immagine-label
24:    return  $n$ 
25:   $n_{train} \leftarrow$  COPYSPPLIT( $S_{train}, train$ )
26:   $n_{val} \leftarrow$  COPYSPPLIT( $S_{val}, val$ )
27:  generazione del file di configurazione data.yaml contenente:
    path del dataset, suddivisione train/val e lista delle classi
28:  return  $n_{train}, n_{val}$ 

```

Algoritmo 3 Procedura di training per object detection basata su YOLO

Require: W : pesi iniziali, D : configurazione del dataset, E : numero di epoche, S : dimensione delle immagini, T : task di detection (HBB o OBB)

Ensure: Modello addestrato M^*

```
1: procedure TRAINMODEL( $W, D, E, S, T$ )
2:   Carica modello YOLO con pesi preaddestrati  $W$ 
3:   Determinare la directory di esecuzione corrente
4:   Costruire il percorso della configurazione del dataset  $D$ 
5:   if configurazione dataset non valida then
6:     return null
7:   Inizializzare i parametri di training:
8:     task  $\leftarrow T$ 
9:     epochs  $\leftarrow E$ 
10:    image size  $\leftarrow S$ 
11:    batch size  $\leftarrow$  auto
12:    device  $\leftarrow$  GPU disponibile
13:   Inizializzare il processo di training
14:   for  $e = 1$  to  $E$  do
15:     Load batch di immagini e annotazioni
16:     Eseguire forward propagation
17:     Calcolare la funzione di loss
18:     Aggiornare i pesi del modello tramite backpropagation
19:     Registrare metriche e checkpoint intermedi
20:   Salvare il modello finale addestrato e gli artefatti del training
21:   return  $M^*$ 
```

2.3.1 Addestramento con bounding box orizzontali

Il primo approccio adottato per l'addestramento dei modelli ha previsto l'utilizzo di bounding box orizzontali (HBB), rappresentate tramite il formato standard YOLO. Questa scelta è stata dettata principalmente dalla semplicità di gestione e di annotazione delle bounding box rispetto alle varianti orientate, un fattore che ha permesso di accelerare la configurazione iniziale del flusso di lavoro. Per ottimizzare le tempistiche e verificare l'effettiva validità del modello sul dominio, si è scelto di adottare una strategia incrementale: una prima fase di validazione è stata condotta su una porzione di dataset di dimensioni molto ridotte. L'obiettivo di questo test preliminare era accertare le potenzialità di YOLO prima di investire ulteriori risorse in un'annotazione massiva.

A fronte dei notevoli risultati ottenuti in questa prima fase, che hanno confermato l'idoneità dell'architettura, si è proceduto all'etichettatura dell'intero dataset descritto nel capitolo 2.3 e a un nuovo addestramento completo. L'espansione dei dati di training si è resa necessaria per garantire una maggiore eterogeneità dei campioni, introducendo una più ampia varietà di scenari, condizioni di illuminazione, occlusioni e conformazioni del carico. Questo ha consentito di mitigare il rischio di overfitting, intrinseco nell'utilizzo di un set di dati ridotto, e di valutare la reale capacità di generalizzazione della rete.

Inoltre, al fine di individuare il miglior compromesso tra l'accuratezza del rilevamento e l'onere computazionale richiesto in fase di inferenza, l'addestramento definitivo non si è limitato a una singola rete, ma ha coinvolto diverse varianti dimensionali dei modelli YOLO. Per valutare in modo oggettivo e rigoroso le performance di ciascuna architettura, sono state analizzate le metriche standard di object detection, ponendo particolare attenzione alla Mean Average Precision (mAP), alla Precision e alla Recall.

Nelle tre sottosezioni successive verranno analizzati nel dettaglio i risultati sperimentali ottenuti dai singoli modelli. Per chiarezza, verranno dapprima riportate le prestazioni definitive con questa configurazione HBB, andando a evidenziare i punti di forza ma anche i limiti geometrici riscontrati; limiti che, come anticipato, motiveranno la successiva transizione verso l'utilizzo delle bounding box orientate.

2.3.1.1 Modello Nano

Il primo modello addestrato si basa su YOLOv8n, ovvero il modello base della famiglia e quello dotato del numero di parametri inferiore, contandone circa 3,2 milioni.

Consultando i risultati raccolti durante la fase di training e dai relativi grafici, il modello ha mostrato una rapida convergenza e prestazioni finali eccellenti al termine delle 100 epoche di addestramento previste. Le metriche di valutazione hanno raggiunto valori molto elevati: la precision si è attestata intorno al 99,3%, la recall al 99,5% e il mAP@50 (mean Average Precision con IoU al 50%) ha toccato il 99,4%.

Anche la metrica più rigorosa, il mAP@50-95, ha raggiunto ottimi livelli, superando l'81,8%. Le curve di loss relative alle bounding box e alle classi mostrano un andamento decrescente e stabile sia sul set di addestramento che su quello di validazione. Questo conferma che la rete ha appreso efficacemente le feature del dataset senza manifestare evidenti segni di overfitting.

I tempi di addestramento sono stati contenuti, ammontando a 15 minuti e 50 secondi.

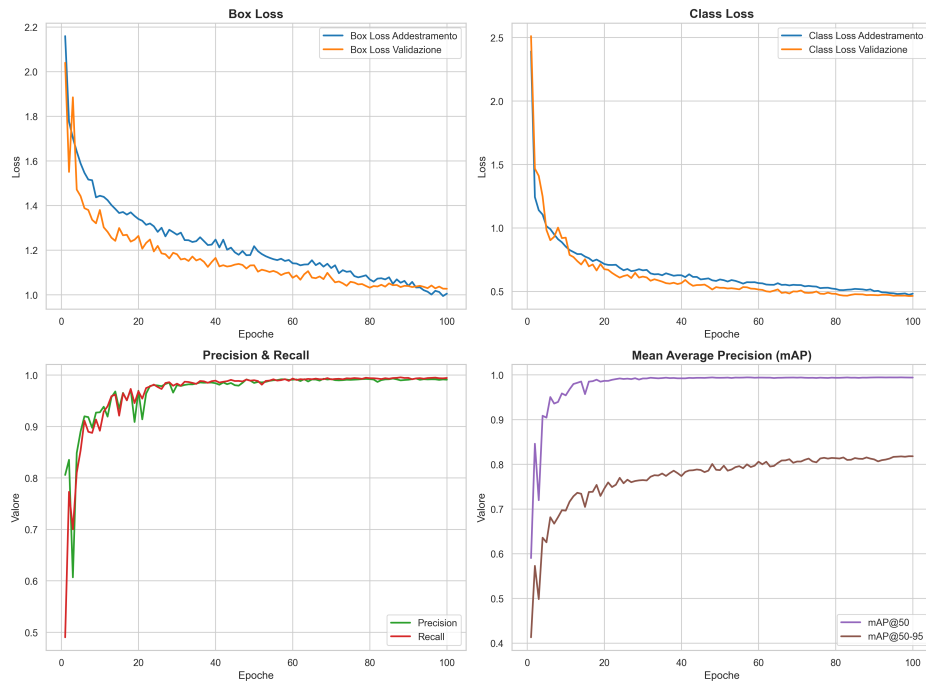


Figura 2.5: Grafici di training e validation del modello YOLOv8n addestrato con bounding box orizzontali

2.3.1.2 Modello Small

Il secondo modello addestrato si basa su YOLOv8s, la versione *small* della famiglia YOLOv8. Pur mantenendo un'architettura leggera, questa variante è dotata di un numero maggiore di parametri rispetto alla versione *nano*, contandone circa 11,2 milioni.

Come si evince dai risultati raccolti durante la fase di training e dai relativi grafici, il modello ha mostrato prestazioni ancora superiori rispetto alla controparte più piccola, continuando a migliorare durante le 100 epoche di addestramento previste. Le metriche di valutazione hanno registrato valori eccellenti e prossimi alla perfezione: la precision ha toccato punte del 99,68%, la recall è arrivata al 99,77% e il mAP@50 ha raggiunto il 99,50%. Risulta particolarmente significativo il miglioramento sulla metrica più stringente, il mAP@50-95, che si è attestato intorno all'83,90%. Le curve di loss di addestramento e validazione confermano una discesa regolare e una convergenza estremamente stabile. In sintesi, la versione Small garantisce una maggiore accuratezza nel rilevamento e nel posizionamento delle bounding box (evidente nell'incremento del mAP@50-95), a fronte di un fisiologico aumento dei tempi di calcolo.

I tempi di addestramento salgono leggermente rispetto al modello nano, attestandosi intorno a 21 minuti e 32 secondi.

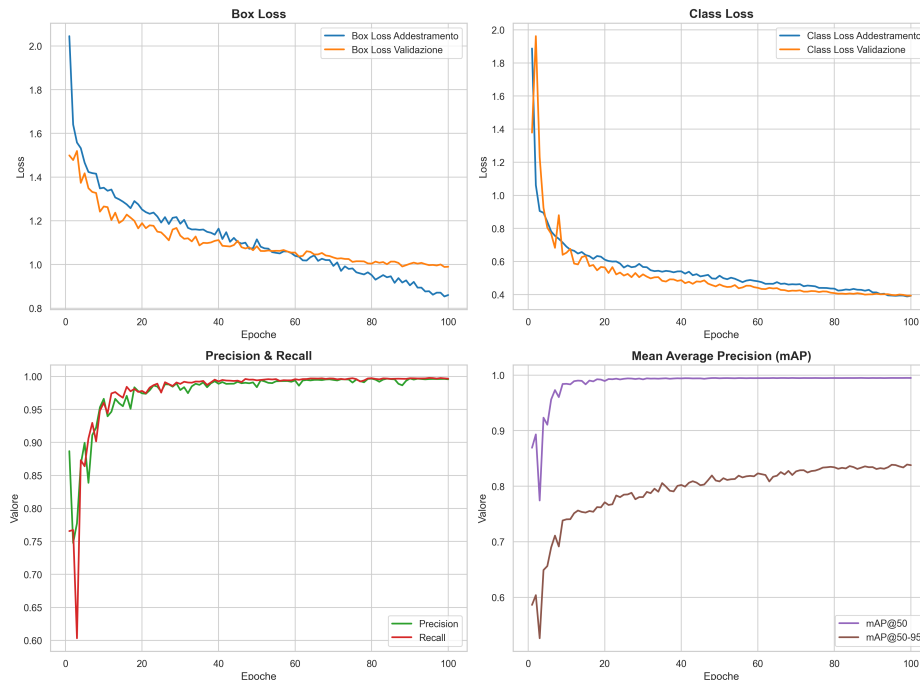


Figura 2.6: Grafici di training e validation del modello YOLOv8s addestrato con bounding box orizzontali

2.3.1.3 Modello Medium

Il terzo ed ultimo modello addestrato si basa su YOLOv8m, la versione *medium* della famiglia YOLOv8. Questa configurazione rappresenta un punto di equilibrio ideale tra complessità computazionale e capacità di estrazione delle feature, contando circa 25,9 milioni di parametri.

Stando ai dati raccolti e dai relativi grafici di addestramento, il modello ha confermato l'ottimo andamento già osservato nelle versioni più leggere. Le metriche di valutazione hanno registrato valori eccellenti: la precision si è attestata al 99,70%, la recall al 99,73% e il mAP@50 ha raggiunto il 99,47%. Risulta particolarmente significativo il risultato della metrica più rigorosa, il mAP@50-95, che è arrivata all'84,20%, segnando un ulteriore miglioramento nella precisione di localizzazione dei bounding box rispetto al modello Small. Le curve di loss relative alle bounding box e alla classificazione confermano una solida fase di convergenza, suggerendo che l'aumento dei parametri ha permesso alla rete di generalizzare in modo efficace le caratteristiche del dataset, garantendo rilevamenti altamente accurati senza incorrere in overfitting.

I tempi di addestramento raddoppiano rispetto al modello small, arrivando a 41 minuti e 13 secondi.

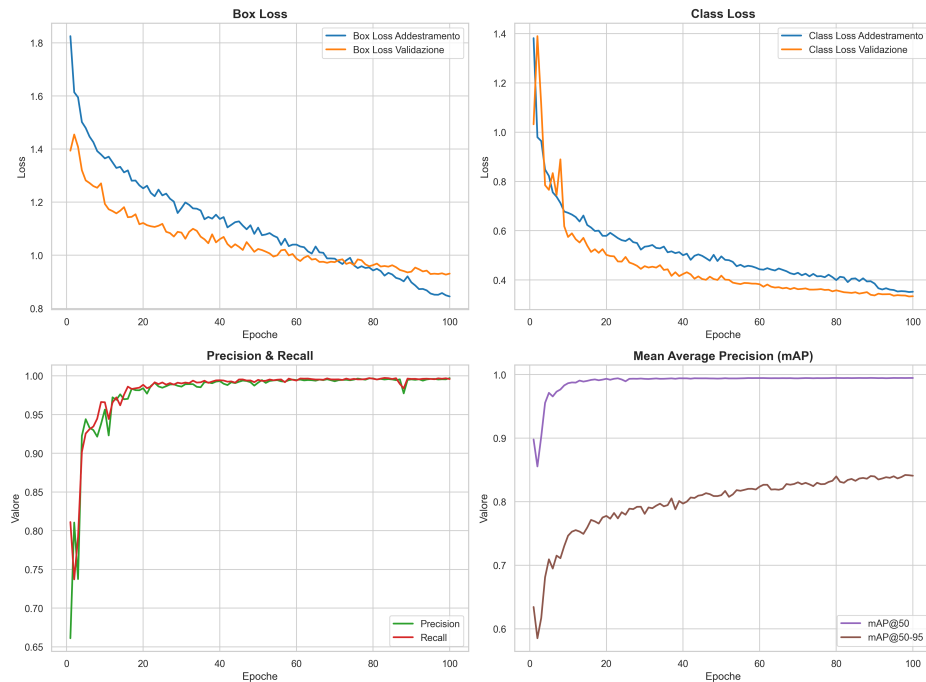


Figura 2.7: Grafici di training e validation del modello YOLOv8m addestrato con bounding box orizzontali

2.3.1.4 Confronto tra i diversi modelli

Si è deciso di non proseguire con l'addestramento di modelli di taglia superiore, come le varianti *Large* (YOLOv8l) o *Extra-Large* (YOLOv8x), in quanto l'analisi dei risultati suggerisce il netto raggiungimento di un plateau prestazionale. Aumentare ulteriormente il numero di parametri e la complessità computazionale della rete non giustificherebbe l'incremento dei tempi di calcolo e di inferenza, a fronte di guadagni in accuratezza che si rivelerebbero del tutto trascurabili.

Analizzando i dati aggregati, si nota come le differenze prestazionali tra i modelli *Nano*, *Small* e *Medium* siano diventate via via più sottili. Le metriche di base mostrano una convergenza eccellente per tutte le architetture testate: il mAP@50 si è assestato stabilmente intorno al 99,5% (con fluttuazioni irrilevanti tra il 99,45% del modello *Nano* e il 99,50% del modello *Small*). Precision e Recall hanno toccato tetti prossimi al 99,7% nei modelli più capaci.

Il vero indicatore del plateau prestazionale emerge osservando la metrica più rigorosa, ovvero il mAP@50-95, che valuta la precisione geometrica dei bounding box al variare delle soglie di Intersection over Union (IoU). Se il passaggio dal modello *Nano* al modello *Small* ha garantito un salto qualitativo evidente (dall'81,82% all'83,90%), la transizione dal modello *Small* al *Medium* ha fruttato un miglioramento quasi impercettibile, attestandosi all'84,20% (un incremento di appena 0,3 punti percentuali).

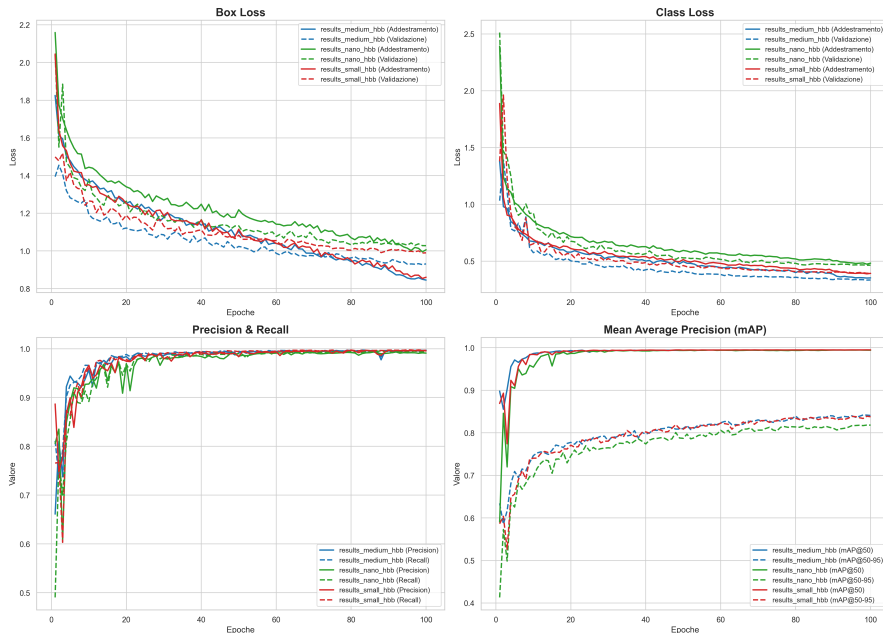


Figura 2.8: Grafici comparativi delle metriche di validazione tra i modelli YOLOv8 Nano, Small e Medium (addestramento HBB)

2.3.2 Evoluzione verso bounding box orientate

Nonostante i risultati eccellenti in termini di prestazioni ottenute con l'approccio basato su bounding box orizzontali, è emerso un limite intrinseco a questa rappresentazione: la difficoltà nel gestire oggetti inclinati o non allineati agli assi dell'immagine. In particolare, nei casi in cui i pallet o gli oggetti presenti nella scena fossero disposti in modo obliquo, per esempio a causa di una caduta di essi, le bounding box orizzontali tendevano a includere porzioni di sfondo non rilevanti, compromettendo la precisione della localizzazione e, di conseguenza, la qualità del rilevamento, come mostrato in figura 2.9.

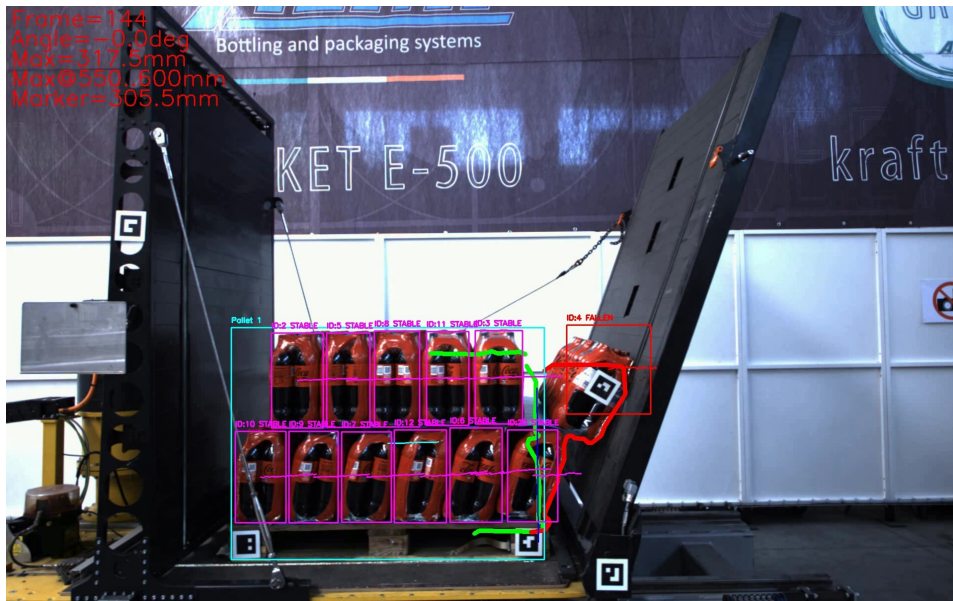


Figura 2.9: Esempio di frame elaborato con bounding box orizzontali, evidenziando i limiti nella localizzazione di oggetti inclinati

A fronte di questa problematica, si è deciso di evolvere verso l'utilizzo di bounding box orientate (OBB), rappresentate tramite il formato specifico per YOLOv8 OBB. L'introduzione di un parametro angolare aggiuntivo permette infatti di far aderire in modo molto più fedele i vertici del rettangolo ai contorni reali del carico, riducendo drasticamente l'interferenza visiva dello sfondo circostante e le sovrapposizioni fittizie tra i box.

Questa scelta ha portato all'esportazione di un nuovo dataset di annotazioni, in cui ogni oggetto è stato etichettato con una bounding box orientata. Si è proceduto quindi ad una nuova sessione di allenamento, addestrando ulteriori tre modelli ottimizzati specificamente per l'estrazione e la gestione di queste feature geometriche complesse.

2.3.2.1 Modello Small OBB

Il modello addestrato in questa nuova fase è la variante YOLOv8s-obb, la versione *small* progettata nativamente per la regressione delle bounding box orientate. Rispetto alla sua controparte orizzontale, l'architettura integra i rami di predizione necessari per calcolare l'angolo di inclinazione degli oggetti, introducendo di conseguenza la *angle loss* per l'ottimizzazione in fase di training.

Analizzando i grafici, durante le 100 epoche di addestramento, emergono risultati che confermano la bontà della scelta strutturale. Le metriche generali si mantengono su livelli ottimi, con una precision del 99,40%, una recall del 92,95% e un mAP@50 pari al 95,90%. Tuttavia, il dato più rilevante è fornito dalla metrica mAP@50-95, che ha raggiunto il considerevole valore dell'87,16%. Confrontando questo risultato con l'83,90% ottenuto dal modello Small addestrato con le bounding box orizzontali, risulta evidente il salto qualitativo: l'introduzione dell'angolo di rotazione ha permesso di far aderire in modo molto più rigoroso i vertici dei box ai contorni reali del carico. Questa precisione geometrica superiore si riflette proprio nell'incremento della metrica ai valori di IoU più stringenti. La progressiva riduzione delle funzioni di loss testimonia l'assenza di overfitting e un'ottima stabilità del processo di apprendimento.

I tempi di addestramento aumentano rispetto al modello HBB, arrivando a 23 minuti e 54 secondi.

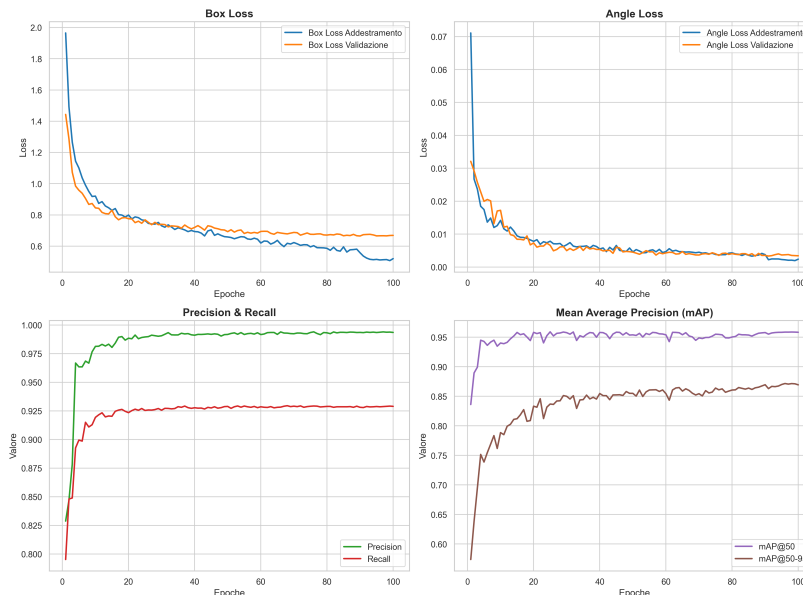


Figura 2.10: Grafici di training e validation del modello YOLOv8s-obb addestrato con bounding box orientate

2.3.2.2 Modello Medium OBB

Il secondo modello addestrato in questa fase è basato su YOLOv8m-obbb, la configurazione *medium* concepita per la rilevazione con bounding box orientate. Grazie al suo maggior numero di parametri rispetto alla versione *Small*, l'aspettativa era quella di verificare se una rete con una più alta capacità di estrazione delle feature potesse raffinare ulteriormente il calcolo dell'angolo e la precisione geometrica dei box in scenari di sovrapposizione complessa.

Tuttavia, analizzando i dati registrati durante le 100 epoche di training, emerge un andamento che ricalca fedelmente quanto già osservato con le bounding box orizzontali: il raggiungimento di un netto plateau prestazionale. Le metriche di valutazione si sono infatti stabilizzate su valori eccellenti ma quasi identici a quelli della variante più leggera. Nello specifico, il modello ha registrato una precision del 99,43%, una recall del 92,96% e un mAP@50 del 95,90%. Particolarmente indicativa è la metrica mAP@50-95, che si è attestata all'87,12%, mostrando scostamenti del tutto trascurabili (nell'ordine dei centesimi di punto percentuale) rispetto al modello Small OBB. Le loss angolari e di bounding box mostrano un'ottima convergenza sia in training che in validazione, confermando che la rete ha appreso correttamente il task senza overfittare.

Anche in questo caso i tempi di addestramento sono maggiori rispetto alla controparte HBB, attestandosi a 47 minuti e 47 secondi.

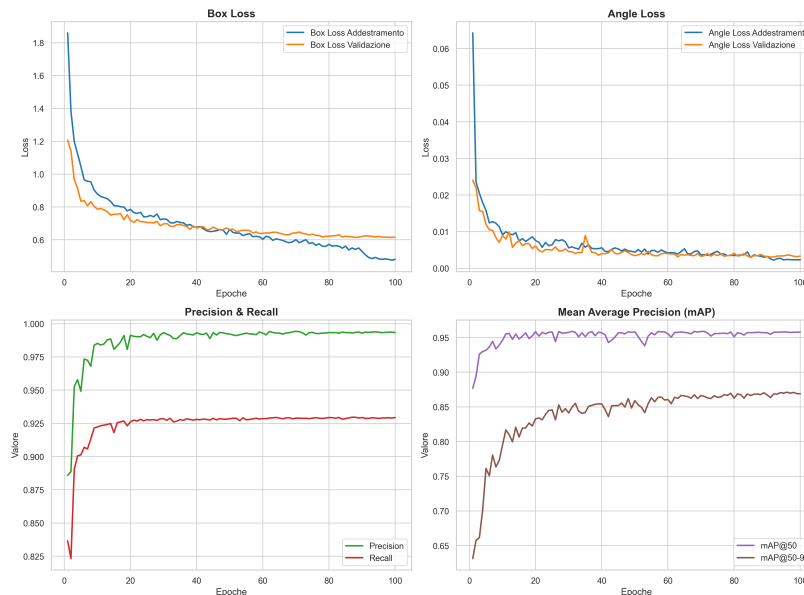


Figura 2.11: Grafici di training e validation del modello YOLOv8m-obbb addestrato con bounding box orientate

2.3.2.3 Modello Large OBB

Per concludere la sperimentazione sulle bounding box orientate e verificare definitivamente i limiti di apprendimento sul dataset a disposizione, si è proceduto all'addestramento di un terzo modello basato su YOLOv8l-obbb, la variante *large*. Questa architettura rappresenta un salto notevole in termini di complessità e capacità rappresentativa, contando circa 43,7 milioni di parametri, ed è stata testata per valutare se una rete più profonda potesse catturare pattern geometrici ancora più sottili legati all'inclinazione dei colli.

Tuttavia, l'analisi dei risultati estratti dopo le 100 epoche di training fornisce una risposta inequivocabile. Le metriche si sono assestate su livelli altissimi, ma non hanno mostrato alcun miglioramento rispetto alle versioni più leggere: la precision ha registrato un valore di picco del 99,45%, la recall si è fermata al 92,92% e il mAP@50 ha toccato il 95,80%. Il dato più esplicativo è fornito dal mAP@50-95, che ha registrato una lieve flessione, scendendo all'86,64% (contro l'87,12% ottenuto dal modello Medium OBB).

I tempi di addestramento, come successe con le varianti HBB, raddoppiano rispetto al modello inferiore, toccando le 2 ore 31 minuti e 23 secondi.

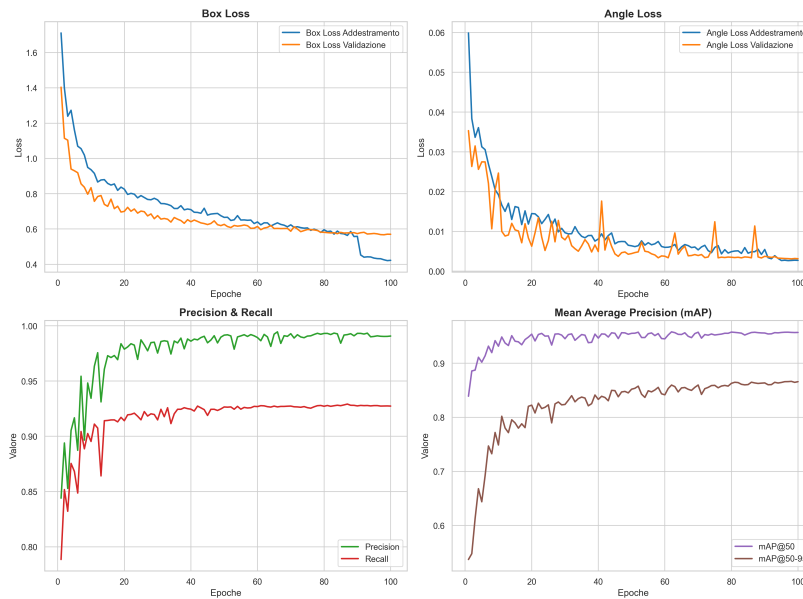


Figura 2.12: Grafici di training e validation del modello YOLOv8l-obbb addestrato con bounding box orientate

2.3.2.4 Confronto tra i diversi modelli OBB

L'ultima fase sperimentale ha messo a confronto le tre architetture addestrate con bounding box orientate (YOLOv8s-obb, YOLOv8m-obb e YOLOv8l-obb). Come già evidenziato nell'analisi singola dei modelli, l'obiettivo era verificare se un incremento della capacità computazionale e del numero di parametri potesse tradursi in una maggiore accuratezza nel calcolo dell'angolo di rotazione e, conseguentemente, in una migliore aderenza spaziale ai colli.

Analizzando i dati aggregati nel file di confronto, emerge un quadro estremamente chiaro che conferma in modo definitivo il raggiungimento di un plateau prestazionale. A differenza della transizione iniziale osservata nei modelli orizzontali, l'evoluzione verso le architetture Medium e Large in ambito OBB non ha prodotto alcun beneficio significativo.

Le metriche di base si sono infatti allineate su valori quasi identici per tutte e tre le reti: la precision ha oscillato in un range infinitesimale (dal 99,40% del modello Small al 99,45% del Large), la recall si è stabilizzata intorno al 92,9% e il mAP@50 ha mantenuto un picco pressoché costante del 95,90%.

L'indicatore più emblematico di questa saturazione è tuttavia il mAP@50-95, la metrica più sensibile alla precisione geometrica dei box. I risultati mostrano addirittura una lieve regressione all'aumentare della complessità del modello: il valore di picco è stato registrato dalla versione Small (87,16%), seguita a ruota dalla Medium (87,12%) e, infine, dalla Large (86,64%). Questo andamento suggerisce che il modello Small possiede già la capacità estrattiva ideale per decodificare le feature geometriche del dataset. Le reti più profonde, al contrario, tendono a *sovra-parametrizzare* il problema, introducendo lievi inefficienze senza riuscire a generalizzare meglio la loss angolare o spaziale.

A valle dell'analisi comparativa per il task specifico di rilevamento orientato, l'architettura YOLOv8m-obb è stata selezionata come candidato definitivo per il prosieguo del lavoro. Sebbene la variante Small si distingua per il minor carico computazionale, il modello Medium rappresenta il miglior trade-off complessivo tra prestazioni di generalizzazione e costo operativo. Questa configurazione garantisce infatti una solidità strutturale superiore e una maggiore capacità di estrazione delle feature in scenari di sovrapposizione complessa, mantenendo al contempo tempi di inferenza ampiamente compatibili con la necessità di un'analisi rapida e stabile nelle successive fasi di elaborazione dinamica del carico.

CAPITOLO 2. COSTRUZIONE DEL DATASET E SVILUPPO DEI MODELLI YOLO

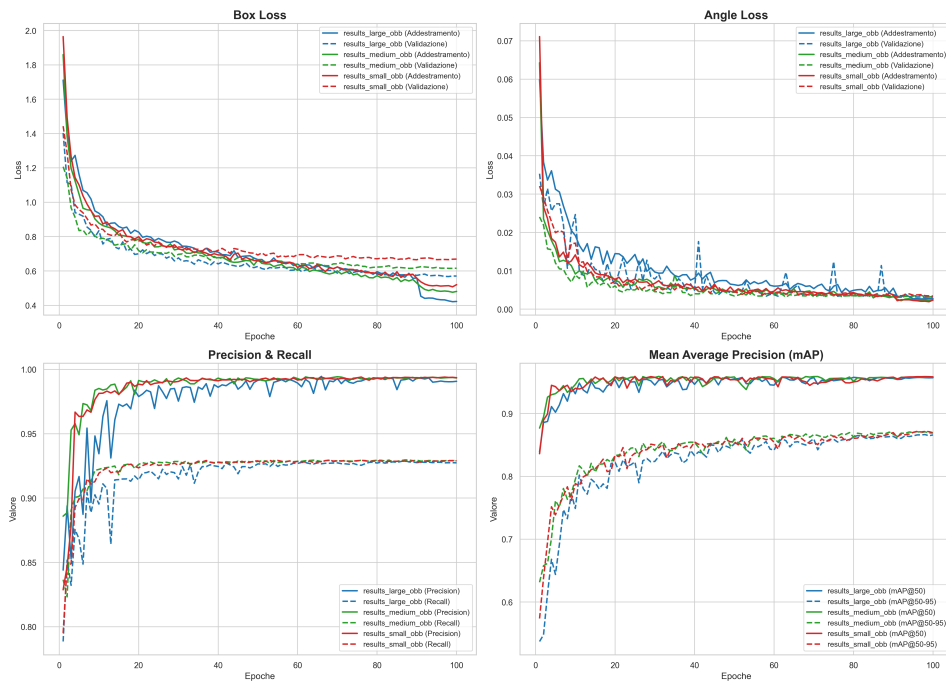


Figura 2.13: Grafici comparativi delle metriche di validazione tra i modelli YOLOv8 Small, Medium e Large (addestramento OBB)

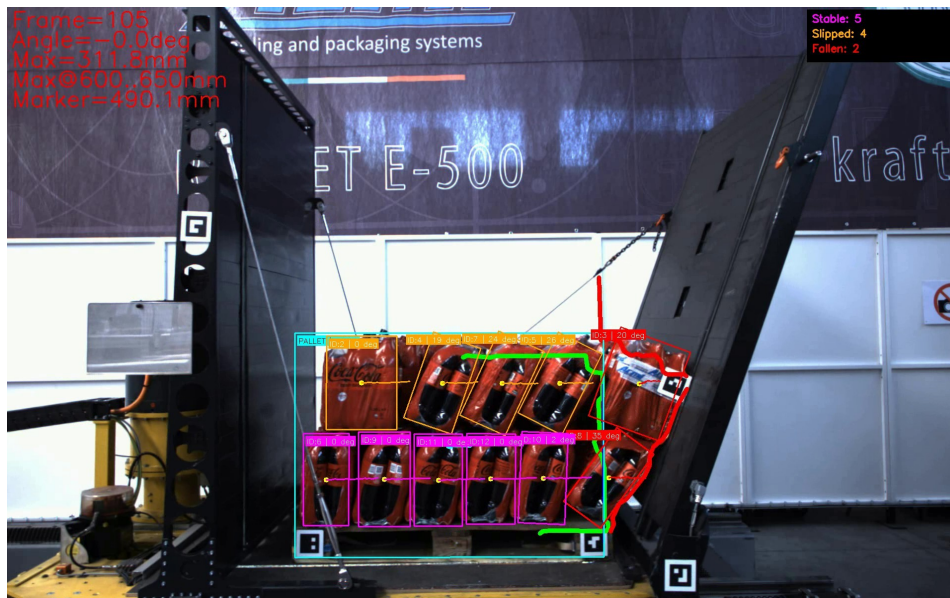


Figura 2.14: Esempio di frame elaborato con bounding box orientate

Capitolo 3

Dall'analisi video alla simulazione fisica

3.1 Pipeline generale del sistema

Il sistema sviluppato si configura come un software completo, dotato di un'interfaccia grafica intuitiva, concepito per fungere da ponte logico e funzionale tra l'analisi visiva dei test fisici e la simulazione in ambiente virtuale. L'obiettivo ultimo del progetto, rispondendo direttamente alle problematiche relative agli standard europei di trasporto, è fornire uno strumento predittivo per valutare la stabilità dei carichi pallettizzati sottoposti a crash test su slitta. In questo modo si riduce drasticamente la necessità di ricorrere a costose e dispendiose iterazioni empiriche per ogni singola variazione di imballaggio.

Il flusso di lavoro si articola in una pipeline sequenziale che trasforma il dato visivo grezzo in un ambiente di simulazione interattivo. Il processo prende avvio dall'input di una registrazione video di un test reale, la quale funge da *ground truth* per l'analisi. I frame di tale sequenza vengono elaborati dai modelli della famiglia YOLO sviluppati in precedenza; in particolare, sfruttando i modelli ottimizzati per le *Oriented Bounding Box* analizzati nel capitolo precedente, l'algoritmo individua e traccia i singoli colli e il pallet nel tempo. Questa fase di *object tracking* permette di estrarre con precisione le traiettorie e le rotazioni, ricavando l'esatta cinematica reale degli oggetti durante il test. I dati geometrici e posizionali così estratti vengono successivamente tradotti e iniettati all'interno di PyBullet, il motore di simulazione fisica le cui caratteristiche e potenzialità sono state introdotte nel precedentemente. Questo passaggio è fondamentale in quanto consente di generare un vero e proprio gemello digitale della scena originaria. Una volta ricreato l'ambiente virtuale, l'utente può interagire attivamente tramite la GUI, alterando i parametri fisici del sistema che nel mondo reale richiederebbero la preparazione

di un nuovo campione fisico. È possibile, ad esempio, modificare le masse dei singoli elementi oppure variare le caratteristiche e il numero di giri della pellicola estensibile applicata al pallet, simulando in tempo reale scenari operativi inediti.

Avviata la simulazione con la nuova configurazione, il motore calcola la dinamica degli impatti e la reazione dei vincoli, fornendo un esito predittivo immediato. Il sistema permette quindi di verificare se la variante proposta mantenga la stabilità del carico o se conduca al fallimento del test, evidenziando ribaltamenti o scivolamenti eccessivi. Grazie a questa profonda integrazione tra visione artificiale e simulazione fisica, il software consente all'utente di testare innumerevoli configurazioni di imballaggio in pochi minuti. Nelle sezioni successive di questo capitolo verrà esplorato il dettaglio tecnico di ogni singolo blocco di questa pipeline, partendo dall'estrazione delle traiettorie fino ad arrivare alla complessa modellazione a parametri concentrati della pellicola protettiva.

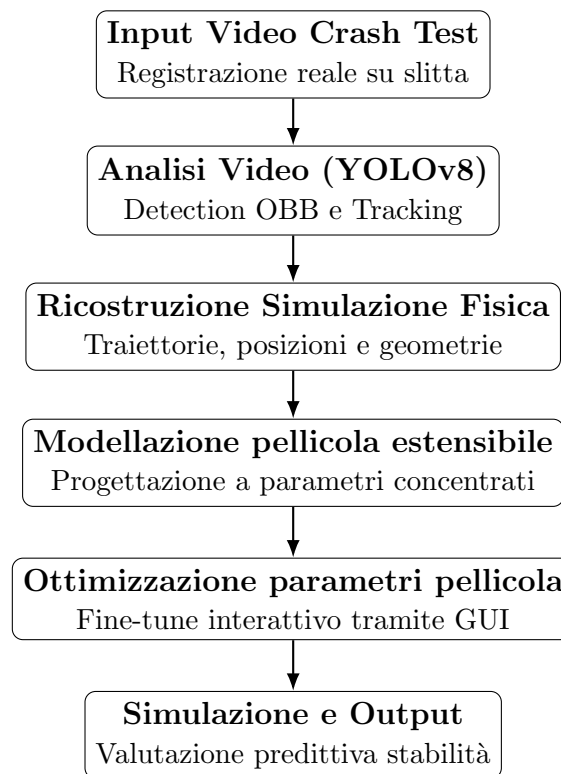


Figura 3.1: Pipeline generale del software sviluppato.

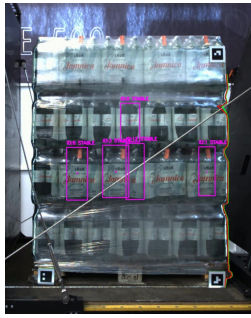
3.2 Analisi video tramite YOLO

Come descritto nel capitolo 2.3.2.4, il modello scelto per l'analisi video è stato YOLOv8m-OBb, il migliore in termini di trade-off tra precisione e velocità di inferenza.

Come mostrato in figura 3.2, i risultati ottenuti con il modello pre-addestrato su COCO sono stati confrontati con quelli del modello custom addestrato sui dati raccolti.

Il nuovo modello ha dimostrato un'estrema efficacia nel rilevamento degli oggetti di interesse, con una precisione significativamente superiore rispetto al modello pre-addestrato.

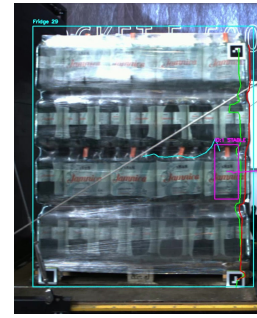
Seppure con una precisione estremamente alta, questo nuovo modello necessita comunque di post-processing per migliorare ulteriormente le prestazioni, soprattutto in scenari complessi o con condizioni di illuminazione variabili per poter ottenere un tracking più stabile e accurato degli oggetti durante l'intero test.



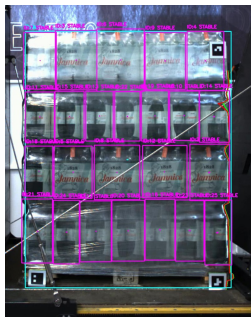
(a) Fase iniziale (COCO)



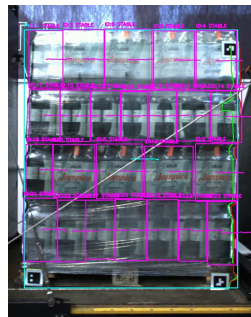
(b) Fase di movimento (COCO)



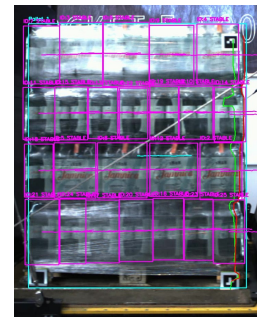
(c) Fase finale (COCO)



(d) Fase iniziale (custom)



(e) Fase di movimento (custom)



(f) Fase finale (custom)

Figura 3.2: Confronto tra i rilevamenti del modello YOLO pre-addestrato su COCO e il modello custom

3.2.1 Tracking e stabilizzazione delle detections

Il sistema di post-processing sviluppato implementa un modulo di tracciamento avanzato volto a garantire la consistenza temporale e spaziale delle detection fornite dal modello. L'algoritmo non si limita ad accettare passivamente le predizioni *frame-by-frame*, ma costruisce una memoria storica per ogni oggetto tracciato, affrontando tre sfide principali: l'associazione temporale, la perdita improvvisa dei target (despawn) e le instabilità geometriche (oscillazioni e rotazioni spurie).

3.2.1.1 Associazione temporale e persistenza degli ID

Il mantenimento degli identificativi nel tempo è principalmente delegato al tracker integrato nel modello. Tuttavia, per ridurre l'influenza di rilevamenti anomali, il sistema applica un filtro basato sull'area delle bounding box all'interno di una finestra temporale mobile.

Sia $\mathcal{A} = \{A_1, A_2, \dots, A_k\}$ l'insieme delle aree delle bounding box rilevate negli ultimi k frame, con $k \leq 200$. Gli elementi di $\mathcal{A}$ sono ordinati in modo non decrescente: $A_1 \leq A_2 \leq \dots \leq A_k$.

L'area mediana della finestra è definita come:

$$\tilde{A} = \begin{cases} A_{\frac{k+1}{2}}, & \text{se } k \text{ è dispari} \\ \frac{A_{\frac{k}{2}} + A_{\frac{k}{2}+1}}{2}, & \text{se } k \text{ è pari} \end{cases} \quad (3.1)$$

Per ogni bounding box B_i con area A_i , viene definita una funzione di validazione:

$$f(B_i) = \begin{cases} 1, & \text{se } A_i \geq 0.4 \cdot \tilde{A} \\ 0, & \text{altrimenti} \end{cases} \quad (3.2)$$

Le detection per cui $f(B_i) = 0$, ovvero con area inferiore al 40% della mediana della finestra, vengono considerate rumore e scartate prima della fase di associazione degli ID.

Gli ID validi, per i quali $f(B_i) = 1$, vengono infine inseriti in un registro storico H , che mantiene traccia di posizione, orientamento e persistenza temporale per ciascun oggetto nel tempo.

3.2.1.2 Riduzione dei falsi negativi e interruzioni di tracking

Le occlusioni parziali, i rapidi mutamenti di contrasto o i cali di confidenza del modello possono causare la temporanea scomparsa di un target, generando falsi negativi e interruzioni della continuità di tracking.

Il sistema mitiga tali interruzioni mediante una logica di persistenza basata su memoria storica dello stato degli oggetti.

Sia D_t l'insieme degli identificativi associati alle detection valide al frame corrente t , e sia H_{t-1} il registro storico contenente lo stato dei target fino al frame $t - 1$.

Lo stato dell' i -esimo oggetto al tempo t è definito come:

$$\mathbf{S}_{i,t} = \langle \mathbf{B}_{i,t}, \mathbf{P}_{i,t} \rangle \quad (3.3)$$

dove $\mathbf{B}_{i,t} = [x, y, w, h, \theta]^T \in \mathbb{R}^5$ rappresenta il vettore dei parametri della bounding box orientata, mentre $\mathbf{P}_{i,t} \in \mathbb{R}^{4 \times 2}$ rappresenta le coordinate dei vertici del poligono associato.

L'evoluzione dello stato nel tempo è definita tramite una regola a casi:

$$\mathbf{S}_{i,t} = \begin{cases} \mathbf{S}_{i,t}^{\text{new}}, & \text{se } i \in D_t \\ \mathbf{S}_{i,t-1}, & \text{se } i \notin D_t \text{ e } i \in H_{t-1} \end{cases} \quad (3.4)$$

dove $\mathbf{S}_{i,t}^{\text{new}}$ rappresenta lo stato aggiornato a partire dalla nuova osservazione, sottoposta alle procedure di filtraggio e smoothing del sistema, mentre $\mathbf{S}_{i,t-1}$ rappresenta la propagazione dello stato precedente in assenza di nuova osservazione.

3.2.1.3 Stabilizzazione delle bounding box

Le predizioni dei modelli OBB sono soggette a micro-oscillazioni e variazioni angolari improvvise (*glitch*), particolarmente evidenti in presenza di oggetti simmetrici o condizioni di illuminazione non ottimali. Per garantire la stabilità dei descrittori geometrici e la robustezza delle metriche di scivolamento e ribaltamento, il sistema adotta un approccio di filtraggio basato su variazione angolare e smoothing temporale.

Sia $\mathbf{S}_t^{\text{raw}} = \langle \mathbf{B}_t^{\text{raw}}, \mathbf{P}_t^{\text{raw}} \rangle$ lo stato osservato al tempo t , dove θ_t^{raw} rappresenta l'angolo di rotazione stimato.

La variazione angolare istantanea è definita come:

$$\Delta\theta_t = |\theta_t^{\text{raw}} - \theta_{t-1}| \quad (3.5)$$

Il sistema mantiene un contatore di persistenza delle anomalie g_t , utilizzato per limitare la durata massima di soppressione delle osservazioni instabili:

$$g_t = \begin{cases} g_{t-1} + 1, & \text{se } \Delta\theta_t > 20^\circ \text{ e } g_{t-1} < 10 \\ 0, & \text{altrimenti} \end{cases} \quad (3.6)$$

L'aggiornamento dello stato filtrato è definito in forma a casi come:

$$\mathbf{S}_t = \begin{cases} \mathbf{S}_{t-1}, & \text{se } \Delta\theta_t > 20^\circ \text{ e } g_{t-1} < 10 \\ \mathbf{S}_t^{\text{raw}}, & \text{se } \Delta\theta_t > 20^\circ \text{ e } g_{t-1} \geq 10 \\ (1 - \alpha)\mathbf{S}_{t-1} + \alpha\mathbf{S}_t^{\text{raw}}, & \text{se } \Delta\theta_t \leq 20^\circ \end{cases} \quad (3.7)$$

dove il filtro esponenziale è applicato componente per componente ai parametri della bounding box e del poligono associato.

Il parametro $\alpha = 0.4$ controlla il livello di smoothing, bilanciando la reattività del sistema e la riduzione del *jitter*.

Come mostrato in Figura 3.3, il filtro riduce significativamente le oscillazioni spurie mantenendo una buona reattività anche in scenari complessi.

Il modulo di detection fornisce quindi una base stabile per l'estrazione delle rotazioni e la persistenza temporale dei target, garantendo continuità geometrica nel tempo.

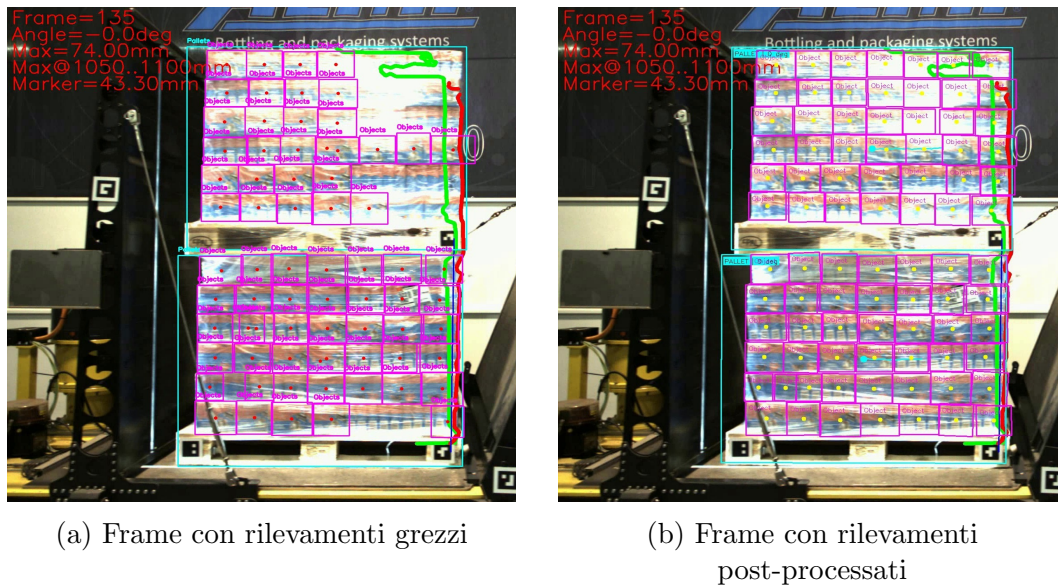


Figura 3.3: Confronto tra i rilevamenti grezzi (a) e quelli post-processati (b) con il sistema di tracking e stabilizzazione implementato.

3.2.2 Analisi delle traiettorie e generazione della reportistica grafica

Il modulo di telemetria e analisi cinematica riceve i vettori di stato geometrici precedentemente stabilizzati al fine di estrarre le traiettorie temporali dei colli, quantificarne le anomalie di posizionamento e mapparne l'evoluzione dello stato di stabilità. I dati così accumulati vengono infine formalizzati in un report grafico sintetico ad alta risoluzione.

3.2.2.1 Estrazione delle traiettorie e calcolo del drift relativo

Al fine di determinare l'effettivo scivolamento dei singoli colli, isolandoli dai movimenti cinematici rigidi della base di appoggio, il sistema opera una traslazione del sistema di riferimento, esprimendo le traiettorie degli oggetti in un sistema solidale al baricentro del pallet.

Sia $C_p(t) = (x_p(t), y_p(t))$ il baricentro bidimensionale del pallet al tempo (frame) t , ottenuto mediante la stima lasciata della bounding box orientata associata alla classe rilevata dal modello. Analogamente, per ogni oggetto tracciato con identificativo i , si definisce la sua posizione nel piano immagine come $C_i(t) = (x_i(t), y_i(t))$.

La posizione orizzontale relativa dell'oggetto rispetto al pallet è quindi definita come:

$$x_{\text{rel}}^{(i)}(t) = x_i(t) - x_p(t) \quad (3.8)$$

Il sistema prevede una fase iniziale di assestamento geometrico pari a un intervallo di frame prefissato $t_{\text{init}} = 15$. Al raggiungimento di tale soglia, la posizione relativa viene considerata come configurazione di riferimento iniziale e viene registrata come:

$$x_{\text{rel}}^{(i)}(0) = x_{\text{rel}}^{(i)}(t_{\text{init}}) \quad (3.9)$$

Per i frame successivi ($t > t_{\text{init}}$), lo scivolamento (drift) dell'oggetto i è quantificato in pixel come la deviazione assoluta rispetto alla configurazione iniziale:

$$D^{(i)}(t) = \left| x_{\text{rel}}^{(i)}(t) - x_{\text{rel}}^{(i)}(0) \right| \quad (3.10)$$

Per quanto concerne il pallet, il suo spostamento non viene relativizzato, ma calcolato in modo assoluto rispetto alla posizione al frame di inizializzazione:

$$D_p(t) = |x_p(t) - x_p(t_{\text{init}})| \quad (3.11)$$

3.2.2.2 Quantificazione del tilt e classificazione degli stati di stabilità

Oltre all'analisi del *drift*, il sistema monitora la deviazione angolare (*tilt*) per rilevare precocemente fenomeni di sbilanciamento o ribaltamento.

L'angolo filtrato $\theta_{\text{deg},t}^{(i)}$ viene confrontato con l'orientamento iniziale $\theta_0^{(i)}$ al frame t_{init} .

Per gestire le ambiguità dovute alla simmetria delle bounding box rettangolari, la variazione angolare effettiva è definita come:

$$\Delta\theta_{\text{raw},t}^{(i)} = \left| \theta_{\text{deg},t}^{(i)} - \theta_0^{(i)} \right| \pmod{90^\circ} \quad (3.12)$$

$$\Delta\theta_{eff,t}^{(i)} = \begin{cases} \Delta\theta_{raw,t}^{(i)}, & \text{se } \Delta\theta_{raw,t}^{(i)} \leq 45^\circ \\ 90^\circ - \Delta\theta_{raw,t}^{(i)}, & \text{altrimenti} \end{cases} \quad (3.13)$$

Le metriche di drift $D_t^{(i)}$, inclinazione $\Delta\theta_{eff,t}^{(i)}$ e contenimento topologico rispetto al pallet contribuiscono congiuntamente alla classificazione dello stato.

Sia $\mathcal{P}_{p,t}$ il poligono del pallet al frame t . La distanza con segno del centro dell'oggetto è definita come:

$$d_{poly}(C_{i,t}, \mathbf{P}_{p,t}) \quad (3.14)$$

Il sistema utilizza una soglia di tolleranza $M_{pallet} = 30$ px per rilevare condizioni di fuoriuscita.

Lo stato dell'oggetto è definito come una macchina a stati finiti:

$$\sigma_t^{(i)} = \begin{cases} \text{CADUTO}, & \text{se condizione persistente o stato precedente CADUTO} \\ \text{SCIVOLATO}, & \text{se } D_t^{(i)} > 30 \text{ px e non caduto} \\ \text{STABILE}, & \text{altrimenti} \end{cases} \quad (3.15)$$

La persistenza degli eventi critici è controllata da un contatore discreto $\omega_t^{(i)}$, che incrementa quando si verificano violazioni geometriche:

$$\omega_t^{(i)} = \begin{cases} \omega_{t-1}^{(i)} + 1, & \text{se violazione geometrica} \\ 0, & \text{altrimenti} \end{cases} \quad (3.16)$$

La transizione allo stato **CADUTO** avviene solo in presenza di persistenza minima di 5 frame consecutivi, riducendo falsi positivi dovuti a occlusioni o rumore di segmentazione.

3.2.2.3 Generazione automatica della reportistica analitica

Al termine della sequenza video, il modulo dedicato alla reportistica analizza l'intero registro storico H memorizzato in memoria per estrarre un quadro diagnostico dell'evoluzione temporale del carico.

Per prevenire l'inclusione di tracce spurie o rilevamenti fantasma dovuti a transienti di rumore ad alta frequenza, viene applicato un filtro di potatura a livello di persistenza temporale. Vengono considerati validi ai fini della reportistica grafica esclusivamente i target $\mathcal{T}^{(i)}$ che soddisfano la seguente condizione di sopravvivenza minima:

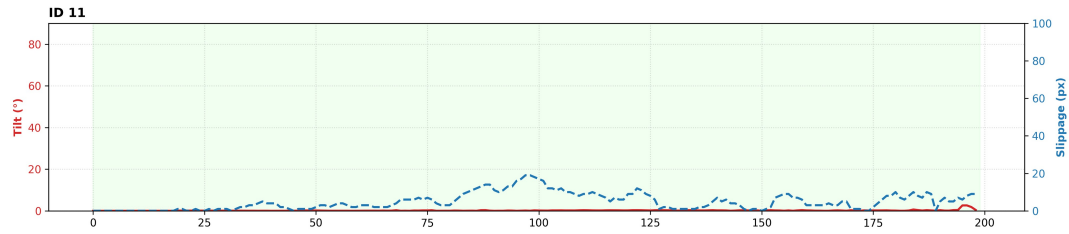
$$\text{len}(\mathcal{T}^{(i)}) > N_{min}, \quad \text{con } N_{min} = 15 \text{ frame} \quad (3.17)$$

Dato un numero di oggetti validi pari a N_{obj} , il framework genera una figura strutturata tramite la libreria `matplotlib` [14], organizzata in una griglia verticale formata da N_{obj} subplots indipendenti di dimensioni parametrizzate. Ciascun grafico è deputato all'analisi del singolo target e sfrutta una configurazione a doppio asse ordinato indipendente per visualizzare simultaneamente le grandezze fisiche eterogenee stimate:

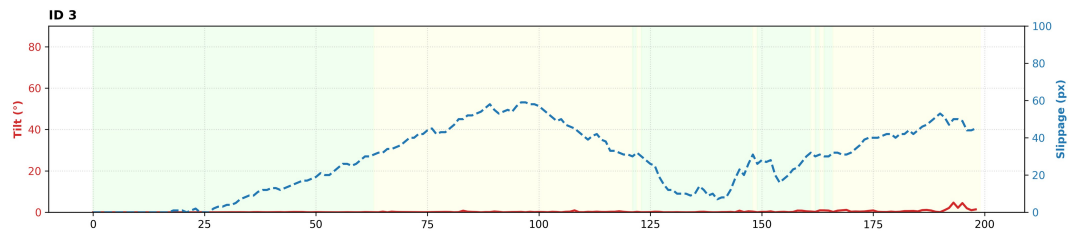
1. **Asse Ordinato Primario (Sinistro):** Dedicato alla visualizzazione del profilo di inclinazione (*Tilt*). Viene tracciata una curva continua di colore rosso, mentre limiti di visualizzazione verticali sono impostati dinamicamente nell'intervallo $[0, \max(90, \Delta\theta_{max} + 10)]$, normalizzando l'ispezione visiva indipendentemente dall'entità del ribaltamento.
2. **Asse Ordinato Secondario (Destro):** Destinato al monitoraggio del fenomeno di sbandamento (*Drift*). Viene renderizzata una curva tratteggiata di colore blu, mentre per la semantica della dicitura dell'asse varia dinamicamente in base alla natura del target tracciato: assume la dicitura *Absolute Displacement (px)* se l'istanza è riconosciuta come pallet e *Slippage (px)* nel caso di colli standard. I limiti verticali sono bloccati nell'intervallo $[0, \max(100, D_{max} + 20)]$.

La caratteristica fondamentale della visualizzazione risiede nella segmentazione cronologica dello stato di stabilità. Il modulo esegue una scansione sequenziale del vettore degli stati; ogni qualvolta viene rilevata una transizione di stato o la terminazione della traccia, viene calcolato l'intervallo di frame $[t_{start}, t_{end}]$ associato a quel comportamento specifico. Il grafico applica una sovrapposizione di sfondo poligonale traslucida utilizzando colori preimpostati per mappare i vari stati ottenuti dagli oggetti. Nel dettaglio: verde per lo stato STABILE, giallo per SCIVOLATO, rosso per lo stato critico CADUTO e azzurro per il profilo del pallet.

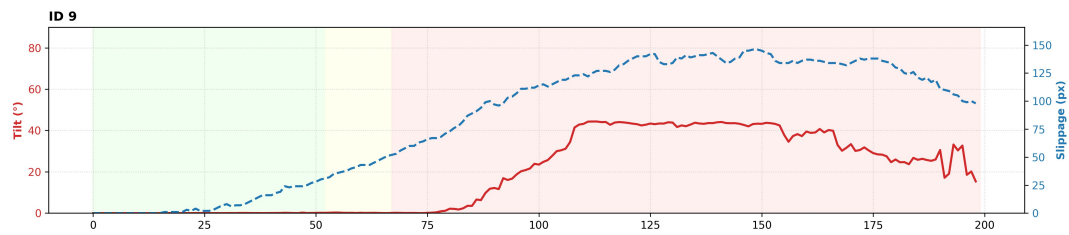
La reportistica grafica così strutturata, visibile in figura 3.4 consente un'immediata correlazione visiva tra le variazioni microscopiche di natura geometrica (picchi di sbandamento o pendenza) e le macro-decisioni logiche assunte dagli algoritmi di classificazione a stati, fornendo uno strumento diagnostico essenziale per la validazione della stabilità dei carichi in contesti industriali.



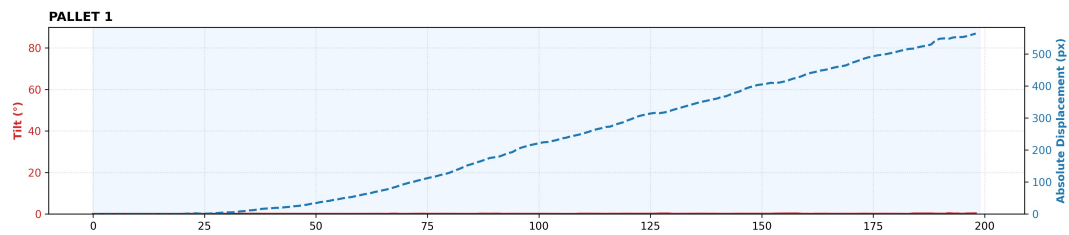
(a) Grafico di un oggetto stabile



(b) Grafico di un oggetto scivolato



(c) Grafico di un oggetto caduto



(d) Grafico relativo ad un pallet

Figura 3.4: Esempio di grafico generato dal modulo di reportistica, con segmentazione cromatica degli stati di stabilità.

3.3 Ricostruzione della simulazione fisica

Una volta completata la fase di tracking e consolidata la stabilità del sistema di rilevamento, il passo successivo consiste nella ricostruzione di un ambiente fisico tridimensionale capace di riprodurre in maniera coerente il comportamento dinamico reale del carico durante il crash test. L'obiettivo non è limitato alla semplice visualizzazione delle traiettorie estratte dal video, ma comprende la creazione di uno scenario simulativo interattivo nel quale poter analizzare configurazioni alternative di imballaggio, valutare differenti condizioni di attrito e studiare l'evoluzione dinamica del pallet in presenza di sollecitazioni controllate.

La conversione delle informazioni provenienti da un sistema di tracking bidimensionale in una simulazione fisica tridimensionale introduce diverse problematiche tecniche. Tra le più rilevanti rientrano la ricostruzione volumetrica degli oggetti a partire da una singola vista monoculare, la gestione numerica delle collisioni tra corpi rigidi e la definizione di un modello di attrito sufficientemente realistico da riprodurre l'effetto stabilizzante della pellicola estensibile applicata al carico.

3.3.1 Creazione dello scenario simulato

L'ambiente virtuale viene implementato utilizzando il motore fisico *open-source* PyBullet, configurato per operare mediante una modellazione a corpi rigidi nello spazio tridimensionale euclideo $\mathbb{R}^3$. La scena simulata viene ricostruita direttamente a partire dalle informazioni geometriche prodotte dal modulo di tracking basato su YOLOv8 con bounding box orientate.

Ogni elemento rilevato all'interno della sequenza video viene classificato secondo due categorie principali: `CLASS_OBJECTS`, associata ai colli dinamici presenti sul pallet e `CLASS_PALLET`, corrispondente alla slitta di supporto.

Per ciascun frame temporale t , il sistema di tracking restituisce, per ogni oggetto i , il seguente vettore di stato bidimensionale:

$$\mathbf{q}_i^t = [x_i^t \ z_i^t \ w_i^t \ l_i^t \ \theta_i^t]^T \quad (3.18)$$

dove:

- (x_i^t, z_i^t) rappresentano le coordinate del centroide dell'oggetto nel piano immagine;
- w_i^t e l_i^t descrivono rispettivamente larghezza e lunghezza della proiezione osservata;
- θ_i^t identifica l'orientamento planare della bounding box.

La conversione dalle coordinate espresse in pixel allo spazio metrico reale viene effettuata mediante un fattore di scala costante:

$$\rho = 0.002 \text{ m/px} \quad (3.19)$$

Il valore di tale coefficiente è stato determinato in modo empirico attraverso una procedura di calibrazione manuale direttamente sui frame della sequenza video. Nello specifico, è stato selezionato un frame rappresentativo della scena in cui l'intera estensione della slitta risulta visibile e non soggetta a occlusioni significative.

A partire da tale immagine, la lunghezza della slitta è stata misurata in pixel mediante conteggio diretto delle coordinate estreme lungo l'asse principale di traslazione. Tale misura è stata poi messa in relazione con la lunghezza reale nota del pallet di tipo Europallet utilizzato nella prova sperimentale, pari a 1.20 m (dimensione standard di riferimento).

Indicando con L_{px} la lunghezza della slitta misurata nel dominio dell'immagine e L_{real} la lunghezza fisica nota del pallet in metri, il fattore di scala viene quindi ricavato come rapporto diretto tra grandezze omogenee:

$$\rho = \frac{L_{\text{real}}}{L_{\text{px}}} \quad (3.20)$$

Il valore numerico risultante, mediato su più misurazioni per ridurre l'errore di quantizzazione dovuto alla discretizzazione pixel-wise, conduce al coefficiente finale utilizzato nel sistema.

L'origine del sistema di riferimento cartesiano globale viene quindi fissata in corrispondenza dell'impronta geometrica del pallet, scelta che consente di mantenere consistente la trasformazione tra dominio immagine e spazio fisico e di semplificare la ricostruzione delle traiettorie cinematiche nel modello tridimensionale.

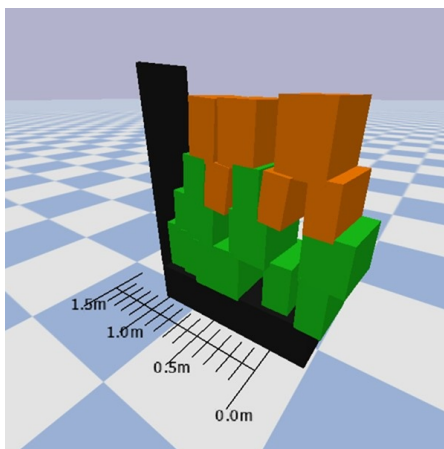
Poiché l'acquisizione video avviene mediante una singola camera laterale monoculare, la coordinata relativa alla profondità reale non è direttamente disponibile. Per compensare questa limitazione, la ricostruzione tridimensionale viene ottenuta discretizzando il volume del carico in N_l strati paralleli, ciascuno caratterizzato da uno spessore costante:

$$\delta_l = 0.4 \text{ m} \quad (3.21)$$

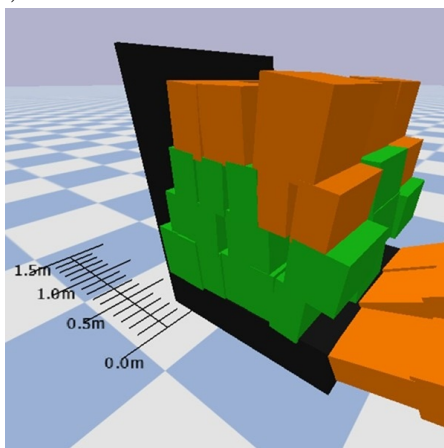
La coordinata lungo l'asse y associata allo strato s viene definita come:

$$y_s = -\frac{N_l \cdot \delta_l}{2} + s \cdot \delta_l + \frac{\delta_l}{2} \quad (3.22)$$

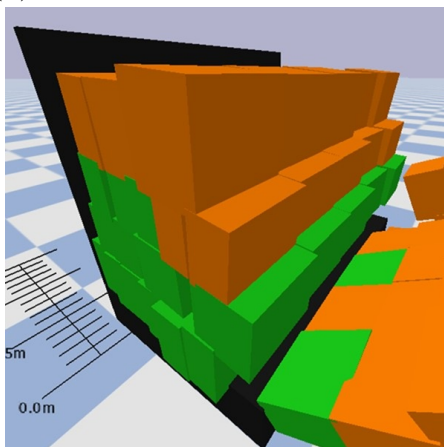
In questo modo gli oggetti vengono distribuiti simmetricamente rispetto alla mezzeria dello scenario simulato come osservabile in figura 3.5.



(a) Simulazione con uno strato di colli



(b) Simulazione con tre strati di colli



(c) Simulazione con cinque strati di colli

Figura 3.5: Esempio di tre simulazioni dello stesso video con diverse profondità di colli

Durante la fase di inizializzazione in PyBullet, ciascun collo viene modellato mediante una primitiva geometrica di tipo box. I semiassi della box tridimensionale vengono derivati direttamente dalle dimensioni osservate nel piano immagine:

$$\mathbf{h}_i = \left[\frac{\rho \cdot w_i}{2} \quad \frac{\delta_i}{2} \quad \frac{\rho \cdot l_i}{2} \right]^T \quad (3.23)$$

dove il vettore $\mathbf{h}_i \in \mathbb{R}^3$ definisce geometricamente i semiassi del corpo rigido associato al collo i -esimo lungo gli assi cartesiani x , y e z . Ciascuna componente del vettore rappresenta quindi la distanza compresa tra il centro di massa dell'oggetto e la rispettiva faccia esterna della box tridimensionale utilizzata dal motore fisico.

Le dimensioni laterali del corpo vengono ricostruite a partire dalle misure bidimensionali estratte dal modulo OBB della rete YOLOv8. In particolare, la larghezza w_i e la lunghezza verticale proiettata l_i , entrambe espresse nel dominio discreto dei pixel, vengono convertite nello spazio metrico reale attraverso il coefficiente di riscalamento ρ (*pixel-to-meters*, espresso in m/px), ottenuto dalla procedura di calibrazione geometrica descritta precedentemente.

Poiché la configurazione sperimentale utilizza una singola telecamera laterale, la profondità reale dell'oggetto lungo l'asse y non risulta direttamente osservabile. Per compensare questa limitazione intrinseca della ricostruzione monoculare, la terza dimensione spaziale viene introdotta artificialmente mediante uno spessore nominale costante δ_i , associato allo strato logistico di appartenenza del collo all'interno del pallet.

Questa formulazione consente quindi di trasformare le informazioni geometriche bidimensionali provenienti dal tracking in volumi tridimensionali coerenti con lo spazio fisico della simulazione, mantenendo una struttura sufficientemente semplice da garantire stabilità numerica ed efficienza computazionale durante la fase di integrazione dinamica.

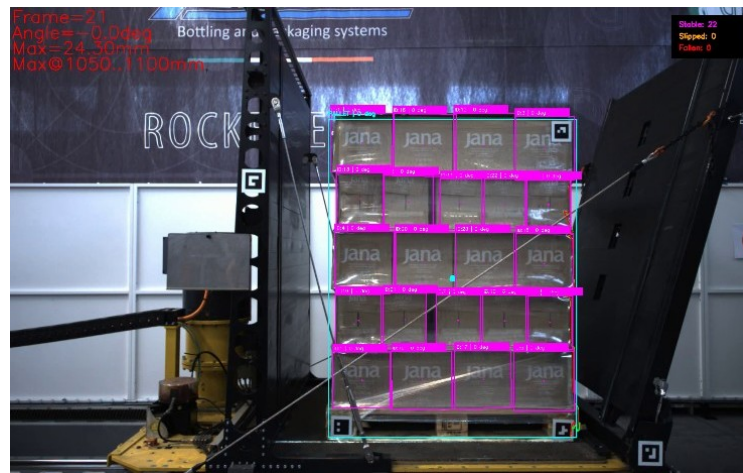
Tale operazione consente di estrarre volumetricamente le bounding box bidimensionali ottenendo una rappresentazione tridimensionale semplificata ma coerente con la geometria osservata.

Per incrementare la coerenza tra lo scenario simulato e la configurazione reale osservata nei video sperimentali, viene inoltre introdotta una barriera laterale rigida a spessore infinitesimo, vincolata all'estremità del pallet. La presenza di questo elemento non deriva esclusivamente da esigenze di stabilità numerica, ma riproduce direttamente una componente fisica realmente presente nel banco prova utilizzato durante il crash test.

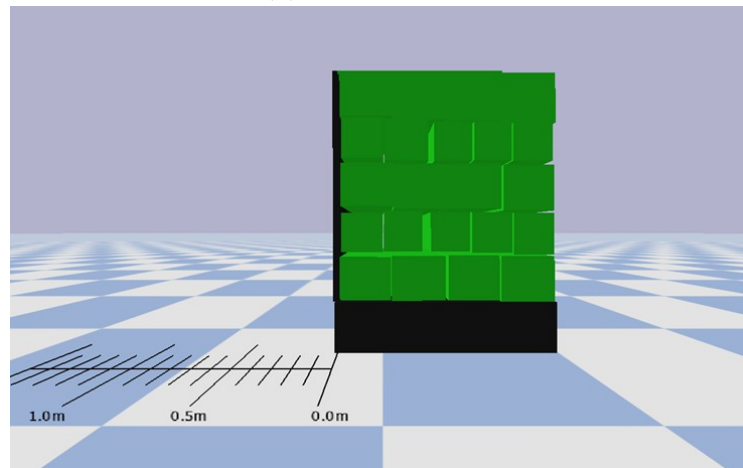
Nel sistema reale, infatti, il pallet risulta vincolato lateralmente da una lastra metallica posizionata sul lato sinistro della struttura, avente la funzione di impedire traslazioni indesiderate e mantenere il corretto allineamento della slitta durante la fase di accelerazione. Per garantire la massima corrispondenza tra

il comportamento osservato nel video e quello ricostruito all'interno del motore fisico, tale vincolo viene replicato anche nella simulazione tridimensionale.

Dal punto di vista computazionale, questa superficie agisce inoltre come elemento stabilizzante durante le prime iterazioni del solver fisico, riducendo il rischio di espulsioni incontrollate dei corpi rigidi causate da compenetrazioni geometriche iniziali o da temporanee instabilità numeriche nella risoluzione dei contatti. Il framework implementato prevede due differenti modalità operative.



(a) Visuale YOLO



(b) Scenario ricreato in PyBullet

Figura 3.6: Creazione dello scenario simulato a partire dalle informazioni geometriche estratte dal sistema di tracking.

3.3.1.1 Simulazione YOLO

Questa modalità è dedicata alla validazione visuale del tracking e alla ricostruzione fotogrammetrica delle traiettorie osservate. In tale configurazione la dinamica fisica viene completamente disattivata imponendo:

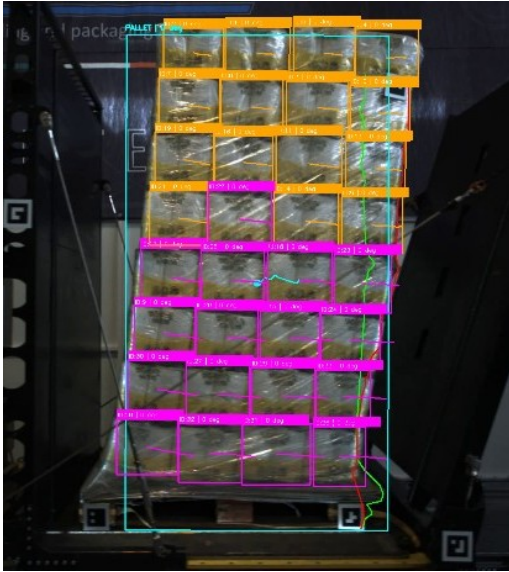
$$\mathbf{g} = \mathbf{0} \quad (3.24)$$

Lo stato di ciascun corpo rigido viene aggiornato direttamente ad ogni frame e le coordinate tridimensionali del corpo vengono quindi forzate nella forma:

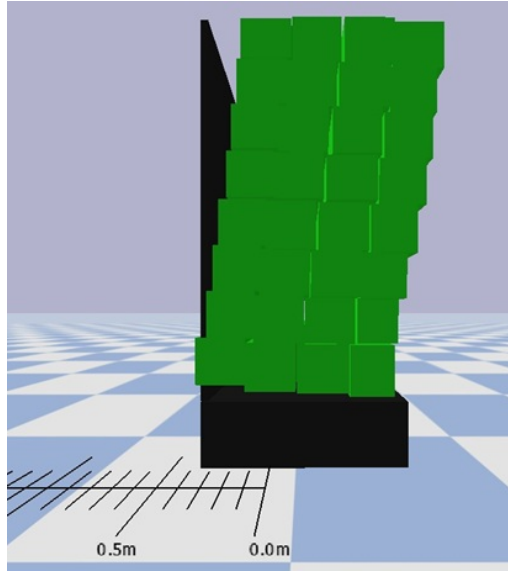
$$\mathbf{x}_i^t = [x_i^t \ y_s \ z_i^t]^T \quad (3.25)$$

mentre l'orientamento viene ricostruito a partire dall'angolo θ_i^t tramite conversione in quaternioni.

In questa modalità collisioni, attriti e interazioni dinamiche risultano completamente ignorati. L'obiettivo è ottenere una rappresentazione tridimensionale fedele delle traiettorie misurate dal sistema di tracking, utile soprattutto per la validazione qualitativa del processo di ricostruzione.



(a) Visuale YOLO



(b) Replay in PyBullet

Figura 3.7: Comparazione tra rilevamento YOLO e Replay in PyBullet

3.3.1.2 Simulazione Fisica

In questa configurazione il motore fisico viene utilizzato per simulare il comportamento dinamico reale del carico. La gravità viene ripristinata imponendo:

$$g_z = 9.81 \text{ m/s}^2 \quad (3.26)$$

L'avanzamento temporale della simulazione adotta uno schema di integrazione numerica a passo fisso:

$$\Delta t = \frac{1}{\text{FPS}} \quad (3.27)$$

Per aumentare la stabilità numerica durante collisioni caratterizzate da elevate variazioni impulsive, ogni passo temporale viene ulteriormente suddiviso in:

$$N_{\text{sub}} = 4 \quad (3.28)$$

sotto-passi interni di integrazione, ottenendo una risoluzione temporale:

$$\delta t = \frac{\Delta t}{4} \quad (3.29)$$

Questa tecnica di *sub-stepping* consente di migliorare la convergenza del solver LCP (*Linear Complementarity Problem*) e ridurre la comparsa di instabilità numeriche durante le interazioni tra i corpi rigidi.

3.3.2 Modellazione cinematica della slitta

Nel mondo reale, il pallet poggia sulla slitta del macchinario di crash test, che ne imprime il profilo di accelerazione. In PyBullet i due elementi vengono fusi in un unico corpo rigido, un parallelepipedo nero con una parete verticale laterale, che rappresenta al contempo la base di appoggio e la sponda che impedisce ai colli di scivolare nella direzione del moto. Questa semplificazione non altera la fedeltà dinamica della simulazione, poiché ciò che governa il comportamento dei colli è esclusivamente il profilo cinematico imposto alla base e il vincolo geometrico laterale.

La slitta viene quindi modellata come un corpo cinematico statico, con massa virtualmente infinita rispetto ai colli sovrastanti, il cui moto lungo l'asse orizzontale x è determinato direttamente dal profilo temporale estratto dal tracking video.

A causa del rumore di quantizzazione introdotto dalle predizioni geometriche della rete neurale, la sequenza delle posizioni grezze:

$$x_{\text{raw}}^t \quad (3.30)$$

viene preliminarmente sottoposta ad un processo di regolarizzazione mediante filtro a media mobile centrata, definito come:

$$\bar{x}^t = \frac{1}{2W+1} \sum_{\tau=-W}^W x_{\text{raw}}^{t+\tau} \quad (3.31)$$

dove la finestra temporale utilizzata possiede ampiezza:

$$2W+1 = 15 \quad (3.32)$$

frame complessivi.

Questa operazione consente di attenuare oscillazioni impulsive e micro-variazioni indesiderate prodotte dal detector, migliorando la stabilità del profilo cinematico ricostruito.

La traiettoria filtrata viene successivamente approssimata mediante un modello cinematico analitico continuo a tratti, progettato per replicare il comportamento tipico delle slitte utilizzate nei crash test industriali conformi alle direttive EUMOS 40509. [11]

Il modello assume una fase iniziale ad accelerazione costante:

$$a = \alpha \cdot g \quad (3.33)$$

dove α rappresenta un coefficiente adimensionale generalmente compreso nell'intervallo:

$$\alpha \in [0.3, 0.5] \quad (3.34)$$

seguita da una fase a velocità uniforme v_c .

Definiti:

$$\Delta X = |x_{\text{end}} - x_{\text{initial}}| \quad (3.35)$$

come spostamento totale della slitta e:

$$T_{\text{motion}} \quad (3.36)$$

come durata complessiva del moto, i principali parametri cinematici vengono determinati come:

$$v_c = \frac{\Delta X}{T_{\text{motion}}}, \quad t_{\text{acc}} = \frac{v_c}{a}, \quad d_{\text{acc}} = \frac{1}{2} a t_{\text{acc}}^2 \quad (3.37)$$

dove:

- t_{acc} rappresenta la durata della fase accelerativa;
- d_{acc} indica lo spazio percorso durante tale intervallo.

La posizione teorica della slitta viene quindi descritta dalla seguente funzione a tratti:

$$x_{\text{syn}}(t) = x_0 + \text{sgn}(x_{\text{end}} - x_0) \cdot \begin{cases} \frac{1}{2} a t^2 & \text{per } t \leq t_{\text{acc}} \\ d_{\text{acc}} + v_c(t - t_{\text{acc}}) & \text{per } t > t_{\text{acc}} \end{cases} \quad (3.38)$$

3.4 Modellazione della pellicola estensibile mediante modello a parametri concentrati

La pellicola estensibile è il principale elemento di contenimento impiegato nell'avvolgimento dei pallet industriali. Dal punto di vista meccanico, essa agisce come un vincolo distribuito che si oppone sia allo scivolamento laterale dei colli rispetto alla base del pallet, sia alla loro rotazione nel piano. Modellare tale vincolo in modo fedele ma computazionalmente praticabile è uno dei nodi centrali dell'intera pipeline proposta.

3.4.1 Scelta del paradigma di modellazione: parametri concentrati o analisi agli elementi finiti

Una descrizione fisicamente completa della pellicola estensibile richiederebbe un'analisi agli elementi finiti tridimensionale (FEA, [25]) della membrana in polietilene lineare a bassa densità (LLDPE), in grado di considerare il comportamento costitutivo iperelastico del materiale, tipicamente descritto tramite modelli di tipo Mooney-Rivlin [23] o Neo-Hookean [27]. Il modello dovrebbe inoltre tenere conto del contatto distribuito tra la membrana e ciascuna faccia dei colli, includendo eventuali fenomeni di slittamento tangenziale, nonché della cinematica a grandi deformazioni e delle instabilità che possono manifestarsi quando la pellicola viene portata oltre il limite di trazione.

Tale approccio è tuttavia del tutto impraticabile nel contesto applicativo qui considerato, per due ragioni concorrenti e quantificabili.

3.4.1.1 Vincolo computazionale del motore fisico.

PyBullet esegue interamente su CPU e non espone alcun percorso di calcolo accelerato su GPU; ogni chiamata al passo di simulazione viene quindi eseguita su un singolo processo, senza hardware acceleration per computazioni di tipo mesh-based. La discretizzazione FEM di una membrana LLDPE di dimensioni realistiche (larghezza 500 mm, spessore 23 μm) richiederebbe almeno $\mathcal{O}(10^3)$ elementi shell, ciascuno dei quali partecipa a equazioni di contatto con le superfici dei colli: un singolo passo di integrazione temporale ammonterebbe a decine di secondi su CPU.

3.4.1.2 Vincolo dell'ottimizzazione a griglia.

Come descritto nella Sezione 3.5.1, la stima dei parametri meccanici della pellicola avviene tramite una ricerca esaustiva a griglia (*exhaustive grid search*) su uno spazio a cinque dimensioni. Per ciascun video del dataset vengono eseguite

fino a 1 000 simulazioni indipendenti di lunghezza completa, ognuna valutata rispetto alla traiettoria osservata. Con una simulazione FEM che impiegherebbe nell'ordine dei minuti per singolo replay, l'intera procedura di ottimizzazione richiederebbe centinaia di ore per video; un approccio chiaramente infattibile in produzione.

3.4.1.3 Soluzione tramite parametri concentrati.

In risposta a queste limitazioni si adotta una strategia a *parametri concentrati* (*lumped-parameter model*), in cui l'intera influenza meccanica della pellicola su ciascun collo è rappresentata da un singolo elemento viscoelastico. Questa scelta non è arbitraria: i modelli a parametri concentrati sono prassi consolidata nell'analisi delle dinamiche di imballaggio e nel dimensionamento dei sistemi di ritenuta su veicoli, ogni qual volta sia richiesto un risultato in tempo reale o quasi-reale. Come verrà mostrato nei risultati sperimentali (Capitolo 4), il modello proposto riproduce fedelmente il comportamento cinematico osservato su un dataset di 103 video reali, senza evidenziare degradazioni sistematiche al variare della classe di massa o del livello di accelerazione.

3.4.2 Modello viscoelastico di Kelvin–Voigt

Il modello a parametri concentrati adottato è la formulazione di Kelvin–Voigt [19], classica nell'analisi di materiali viscoelastici a comportamento prevalentemente elastico con smorzamento viscoso moderato. Il modello è composto da una molla elastica lineare e da uno smorzatore viscoso collegati in parallelo tra il punto di ancoraggio sul pallet e il centroide del collo.

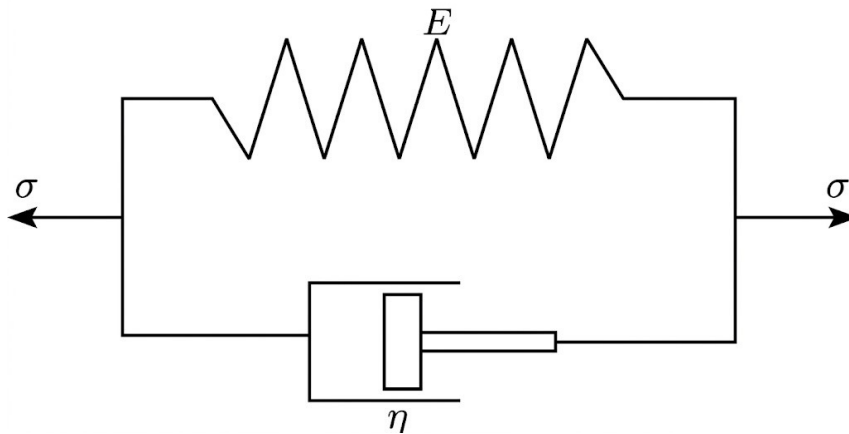


Figura 3.8: Rappresentazione del modello a parametri concentrati di Kelvin–Voigt adottato per la pellicola estensibile.

Per il generico collo i , la forza restitutiva esercitata dalla pellicola al passo di simulazione corrente è:

$$\mathbf{F}_i = -k_i \boldsymbol{\delta}_i - c_i \dot{\boldsymbol{\delta}}_i, \quad (3.39)$$

dove k_i [N/m] è la rigidità effettiva della molla, c_i [N · s/m] è il coefficiente di smorzamento viscoso, $\boldsymbol{\delta}_i$ è il vettore di spostamento relativo tra il centroide del collo e il punto di ancoraggio espresso nel sistema di riferimento assoluto, e $\dot{\boldsymbol{\delta}}_i$ è la velocità relativa corrispondente.

Le quantità cinematiche coinvolte sono definite come:

$$\boldsymbol{\delta}_i = \mathbf{p}_i - \mathbf{a}_i^{\text{world}}, \quad (3.40)$$

$$\dot{\boldsymbol{\delta}}_i = \dot{\mathbf{p}}_i - \dot{\mathbf{p}}_{\text{pallet}}, \quad (3.41)$$

dove $\mathbf{p}_i \in \mathbb{R}^3$ è la posizione del centroide del collo i nel riferimento assoluto, $\mathbf{a}_i^{\text{world}} \in \mathbb{R}^3$ è il punto di ancoraggio della molla espresso nel riferimento assoluto (ottenuto trasformando l'ancora locale $\mathbf{a}_i^{\text{local}}$, fissa nel frame del pallet, attraverso la posa corrente del pallet), e $\dot{\mathbf{p}}_{\text{pallet}}$ è la velocità del corpo pallet interrogata direttamente dal simulatore PyBullet.

L'equazione (3.39) descrive pertanto la forza come somma di un termine elastico proporzionale all'allungamento della pellicola rispetto alla configurazione di riposo, e di un termine viscoso proporzionale alla velocità di deformazione relativa tra il collo e il pallet. Il secondo termine ha un ruolo fisicamente ben motivato: durante l'impulso di accelerazione della slitta, la velocità relativa istantanea tra il collo (per inerzia, inizialmente fermo) e il pallet (in rapido movimento) è elevata; la componente viscosa genera quindi un'oppositività immediata alla separazione, riproducendo il comportamento smorzante della pellicola LLDPE di tipo semi-cristallino.

Restrizione assiale. La forza (3.39) viene azzerata lungo l'asse verticale (z) in ogni sotto-passo di simulazione: la pellicola non oppone resistenza alla quota gravitazionale dei colli, poiché l'assestamento verticale è governato dai contatti rigidi tra le superfici. Opzionalmente, la forza viene ristretta al solo asse laterale x , dato che il carico inerziale dominante durante l'accelerazione della slitta è orizzontale. Tale configurazione è controllata da una guardia booleana e corrisponde all'ipotesi che la pellicola lavori principalmente in trazione laterale, trascurando le componenti di taglio nel piano yz . Formalmente:

$$\mathbf{F}_i^{\text{eff}} = \begin{cases} F_{i,x} \hat{\mathbf{e}}_x & \text{se } \text{trazione laterale} = \text{True}, \\ F_{i,x} \hat{\mathbf{e}}_x + F_{i,y} \hat{\mathbf{e}}_y & \text{altrimenti,} \end{cases} \quad (3.42)$$

con la componente z sempre nulla: $F_{i,z} \equiv 0$.

3.4.3 Implementazione della rete di molle

3.4.3.1 Struttura dati FilmSpring

Nella implementazione software, ogni elemento viscoelastico è rappresentato da un'istanza della classe `FilmSpring`, che raccoglie:

- l'identificatore `PyBullet` del corpo rigido associato al collo;
- il punto di ancoraggio nel frame locale del pallet, calcolato come la posizione iniziale del centroide del collo trasformata nel sistema di riferimento del pallet al momento della registrazione;
- la rigidezza k_i , lo smorzamento c_i e la soglia di rottura $f_{\text{break},i}$;
- un flag booleano `broken` che indica se la molla è già in stato di rottura.

La registrazione di un collo avviene tramite la funzione `registerPackage`, che riceve la posizione iniziale del collo nel riferimento assoluto, calcola la trasformazione inversa rispetto alla posa corrente del pallet e memorizza il punto di ancoraggio locale. Formalmente, se $(\mathbf{p}_{\text{pal}}, \mathbf{q}_{\text{pal}})$ è la posa del pallet (posizione e quaternion), allora:

$$\mathbf{a}_i^{\text{local}} = R(\mathbf{q}_{\text{pal}})^{-1}(\mathbf{p}_i^{(0)} - \mathbf{p}_{\text{pal}}), \quad (3.43)$$

dove $\mathbf{p}_i^{(0)}$ è la posizione iniziale del collo nel riferimento assoluto. In questo modo, l'ancora si muove solidalmente con il pallet durante tutta la simulazione, e la deformazione δ_i misura correttamente lo spostamento del collo rispetto alla configurazione ideale di piena aderenza al pallet. L'implementazione è consultabile nell'Algoritmo 4.

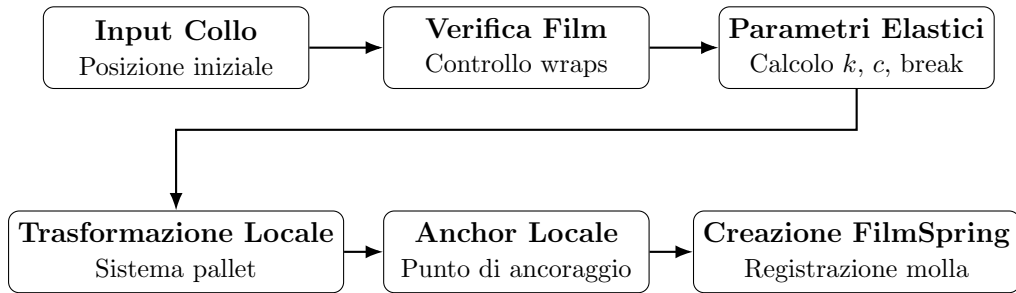


Figura 3.9: Schema della registrazione di un package e generazione della molla virtuale del film estensibile.

Algoritmo 4 Registrazione di un package e creazione della molla di film

```

1: procedure REGISTERPACKAGE
Require: package_id, world_pos e world_orn, sistema pallet inizializzato
Ensure: lista delle molle di film aggiornata con una nuova FilmSpring se
        wraps > 0
2:   wraps ← numero di avvolgimenti del film
3:   if wraps ≤ 0 then
4:     return                                ▷ nessun film attivo, nessuna
                                                molla registrata
5:   k ← se definito custom_k altrimenti kbase · wraps
6:   c ← se definito custom_c altrimenti cbase · wraps
7:   break ← se definito custom_break altrimenti breakbase · wraps
8:   (pal_pos, pal_orn) ← posizione e orientazione globale del pallet
9:   (rel_pos, _) ← trasformazione inversa pallet-mondo applicata alla
        posizione del package
10:  anchor_local ← conversione di rel_pos in vettore numerico
11:  creazione di una nuova FilmSpring con:
12:    package_id, anchor_local, k, c, break, pkg_w
13:  aggiunta della molla alla lista delle molle attive

```

3.4.3.2 Calcolo e applicazione della forza al sotto-passo

La funzione `ApplySpringForce`, mostrata tramite l'Algoritmo 5, viene invocata a ogni sotto-passo fisico (la simulazione opera con $N_{\text{sub}} = 4$ sotto-passi per frame, ciascuno di durata $\Delta t_{\text{sub}} = \Delta t/4$).

La sequenza operativa prevede: la trasformazione dell'ancora locale $\mathbf{a}_i^{\text{local}}$ nel riferimento globale del pallet per ottenere $\mathbf{a}_i^{\text{world}}$:

$$\mathbf{a}_i^{\text{world}} = R(\mathbf{q}_{\text{pal}}) \mathbf{a}_i^{\text{local}} + \mathbf{p}_{\text{pal}}. \quad (3.44)$$

Il calcolo della deformazione δ_i e della relativa velocità $\dot{\delta}_i$ tramite le equazioni (3.40)–(3.41) interrogando le API PyBullet;

La verifica della deformazione massima, per cui se $\|\delta_i\| > \delta_{\text{max}} = 0.25$ m la molla viene marcata come **broken**; il calcolo della forza secondo il modello di Kelvin–Voigt (3.39) con vincoli assiali (3.42);

La verifica della soglia di rottura meccanica, per cui se $\|\mathbf{F}_i\| > f_{\text{break},i}$ la molla cede;

Infine, se integra, la forza risultante $\mathbf{F}_i^{\text{eff}}$ viene applicata al centroide del collo nel riferimento assoluto.

Algoritmo 5 Applicazione della forza elastica-viscosa della film spring

```
1: procedure APPLYSPRINGFORCE
Require: molla elastica inizializzata, stato fisico del pallet e del package
Ensure: applicazione della forza elastica-viscosa e aggiornamento dello stato
        della molla
2:   if spring è broken then
3:     return (0, 0)
4:   pallet_pos  $\leftarrow$  posizione globale del pallet
5:   pallet_orn  $\leftarrow$  orientazione globale del pallet
6:   anchor_world  $\leftarrow$  trasformazione da coordinate locali del pallet a mondo
        applicata all'ancora della molla
7:   pkg_pos  $\leftarrow$  posizione globale del package
8:   pkg_vel  $\leftarrow$  velocità lineare del package
9:   pal_vel  $\leftarrow$  velocità lineare del pallet
10:  if sled_moving = false or allow_break = false then
11:    aggiorna spring.anchor_local proiettando la posizione corrente del
        package nel sistema locale del pallet
12:    return (0, 0)                                ▷ evita accumulo di tensione
                                                    durante stato statico
13:  displacement  $\leftarrow$  pkg_pos – anchor_world
14:  relative_vel  $\leftarrow$  pkg_vel – pal_vel
15:  annulla componente verticale della dinamica (asse z)
16:  if lateral_only = true then
17:    annulla componente laterale selezionata della dinamica
18:  disp_mag  $\leftarrow$   $||displacement||$ 
19:  if disp_mag > max_deform then
20:    marca la molla come broken
21:    applica condizioni fisiche di rilascio al package
22:    return (0, 1)                                ▷ rottura per deformazione
                                                    eccessiva
23:   $F \leftarrow -k \cdot displacement - c \cdot relative\_vel$ 
24:  force_mag  $\leftarrow ||F||$ 
25:  aggiorna memoria interna della forza massima
26:  if force_mag > spring.break_force then
27:    marca la molla come broken
28:    applica condizioni fisiche di rilascio al package
29:    return (force_mag, 1)                        ▷ rottura per superamento soglia
                                                    di forza
30:  tension_ratio  $\leftarrow$  min(1, force_mag/spring.break_force)
31:  annulla componente verticale della forza
32:  applica forza al package nel sistema globale
33:  return (force_mag, tension_ratio)
```

3.4.3.3 Gestione della pre-tensione e del regime stazionario

Un aspetto delicato nella modellazione è la gestione dello stato di riposo *prima* che la slitta inizi a muoversi. Le posizioni dei colli stimate da YOLO contengono rumore metrico residuo; se l'ancora fosse congelata sin dallo spawn, la piccola discrepanza tra posizione stimata e posizione reale genererebbe una forza elastica spuria che, non appena il pallet inizia ad accelerare, produrrebbe un transitorio artificiale nelle traiettorie simulate.

Per evitare questo artefatto, mentre la slitta è ferma, l'ancora locale viene aggiornata a ogni sotto-passo per inseguire la posizione corrente del collo nel frame del pallet:

$$\mathbf{a}_i^{\text{local}} \leftarrow R(\mathbf{q}_{\text{pal}})^{-1}(\mathbf{p}_i - \mathbf{p}_{\text{pal}}), \quad \forall t < t_{\text{start}}. \quad (3.45)$$

In questo regime, la forza applicata è identicamente nulla e i colli vengono mantenuti in posizione da un vincolo fisso. Nel momento in cui viene rilevato l'inizio del moto della slitta (frame t_{start} , determinato dal modello cinematico descritto nella Sezione 3.3.2), i vincoli fissi vengono rimossi, l'ancora viene congelata nel frame del pallet, e la forza restitutiva diventa attiva. Da quel momento in poi, qualsiasi spostamento del collo rispetto alla sua posizione iniziale sul pallet si traduce in una forza di richiamo misurata dall'ancora fissa.

3.4.3.4 Rottura a cascata

Il modello di rottura della pellicola include due criteri di cedimento indipendenti, già descritti sopra: (a) *rottura meccanica*, quando $\|\mathbf{F}_i\| > f_{\text{break},i}$, e (b) *rottura geometrica*, quando $\|\boldsymbol{\delta}_i\| > \delta_{\text{max}}$. Entrambi contrassegnano la singola molla come **broken**.

Il meccanismo più significativo dal punto di vista fisico è però la *rottura a cascata*: non appena una qualsiasi molla del sistema cede, tutte le molle rimanenti vengono istantaneamente marcate come rotte. Questo comportamento riflette quanto osservato sperimentalmente con la pellicola di avvolgimento reale: quando il film cede in un punto, la ridistribuzione degli sforzi è non uniforme e il vincolo residuo collassa quasi istantaneamente. Formalmente, definendo $\mathcal{B}_t$ l'insieme delle molle rotte al passo t :

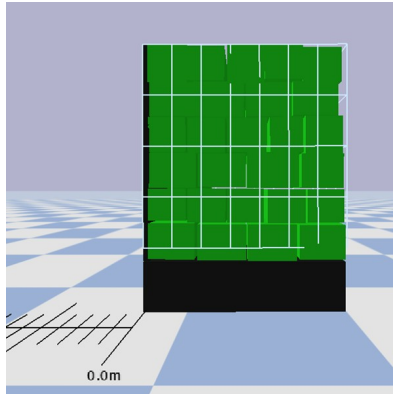
$$|\mathcal{B}_t| > 0 \implies \mathcal{B}_{t+1} = \{1, 2, \dots, N\}, \quad (3.46)$$

dove N è il numero totale di molle attive.

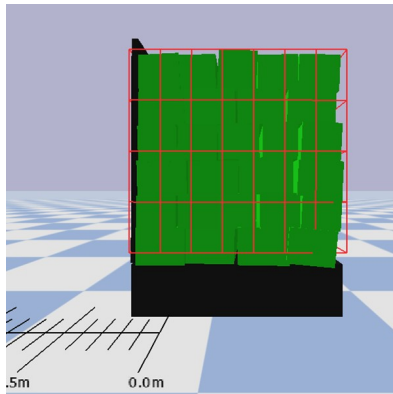
Dopo la cascata, ciascun collo ancora integro acquisisce la componente laterale della velocità del pallet al momento del cedimento:

$$\dot{\mathbf{p}}_i \leftarrow (|\dot{p}_{\text{pal},x}|, \dot{p}_{\text{pal},y}, \dot{p}_{\text{pal},z}), \quad (3.47)$$

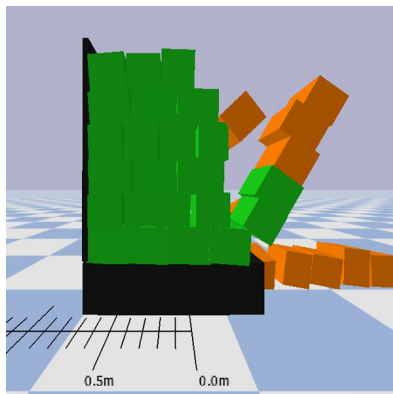
e successivamente evolve liberamente sotto l'azione della gravità e della frizione con le superfici circostanti. Il flag `wrap_active` viene posto a **False** e la rete di molle è disattivata per i frame successivi. L'intero processo di evoluzione degli stati della pellicola è illustrato nella Figura 3.10.



(a) Simulazione con pellicola rilassata



(b) Simulazione con pellicola in tensione



(c) Simulazione durante il cedimento della pellicola

Figura 3.10: Esempi di lavoro della pellicola in diverse fasi della simulazione

3.4.4 Parametri del modello

I valori base per singolo strato di avvolgimento (*per wrap*) sono:

$$k_0 = 500 \text{ N/m}, \quad (3.48)$$

$$c_0 = 3 \text{ N} \cdot \text{s/m}, \quad (3.49)$$

$$f_0 = 25 \text{ N}. \quad (3.50)$$

La scelta di questi valori è ancorata a considerazioni fisiche sulla pellicola LLDPE di riferimento (spessore $23 \mu\text{m}$, larghezza 500 mm).

3.4.4.1 Rigidezza k_0 .

La stima da primi principi della rigidezza assiale di un segmento di pellicola di lunghezza $L \approx 0.3\text{--}0.6 \text{ m}$ con modulo di Young $E \approx 200\text{--}400 \text{ MPa}$ e area della sezione trasversale $A = 500 \text{ mm} \times 23 \mu\text{m} = 1.15 \times 10^{-5} \text{ m}^2$ dà:

$$k_{\text{teorico}} = \frac{EA}{L} \approx 4\,000\text{--}15\,000 \text{ N/m}. \quad (3.51)$$

Il valore base $k_0 = 500 \text{ N/m}$ è deliberatamente conservativo rispetto a questa stima. La riduzione è giustificata dalla pre-stiratura applicata durante l'avvolgimento (tipicamente $150\text{--}300\%$), che riduce la sezione trasversale effettiva e abbassa il modulo secante del materiale in modo significativo. Consentendo all'ottimizzatore di scalare k_0 tramite il moltiplicatore $\kappa \in [0.2, 2.5]$, il range di esplorazione arriva fino a $1\,250 \text{ N/m}$ per strato, fisicamente realistico per una pellicola fortemente pre-stirata.

3.4.4.2 Smorzamento c_0 .

Il valore $c_0 = 3 \text{ N} \cdot \text{s/m}$ riflette la bassa dissipazione viscosa dei film poliolefinici semi-cristallini. Il range di ottimizzazione del moltiplicatore $\gamma \in [0.5, 1.5]$ è volutamente ristretto, in accordo con il risultato sperimentale che lo smorzamento abbia un effetto secondario sulla traiettoria grezza rispetto a rigidezza e soglia di rottura.

3.4.4.3 Soglia di rottura f_0 .

La resistenza ultima teorica del materiale LLDPE non deformato è stimabile come:

$$F_{\text{UTS}} = \sigma_{\text{UTS}} \cdot A \approx 30 \text{ MPa} \times 1.15 \times 10^{-5} \text{ m}^2 \approx 345 \text{ N}, \quad (3.52)$$

che si riduce a circa 170 N dopo una pre-stiratura del 200% . Il valore $f_0 = 25 \text{ N}$ non rappresenta quindi la resistenza ultima del materiale, ma una *soglia*

comportamentale calibrata per eguagliare il carico inerziale tipico su un collo durante l'accelerazione della slitta. Per un collo di massa $m = 20 \text{ kg}$ soggetto a un'accelerazione di $a = 0.3g$:

$$F_{\text{inerziale}} = m \cdot a \approx 20 \text{ kg} \times 0.3 \times 9.81 \text{ m/s}^2 \approx 59 \text{ N}, \quad (3.53)$$

cosicché con due strati di avvolgimento ($f_{\text{break,eff}} = 2 \times 25 \text{ N} = 50 \text{ N}$) un carico scarsamente avvolto si trova al limite tra scivolamento e tenuta, coerentemente con le osservazioni empiriche riportate in letteratura sulle dinamiche di imballaggio.

3.4.4.4 Scalatura con il numero di strati

I parametri effettivi per il collo i vengono scalati linearmente con il numero di strati di avvolgimento n_w :

$$k_{\text{eff}} = n_w \cdot k_0 \cdot \kappa, \quad (3.54)$$

$$c_{\text{eff}} = n_w \cdot c_0 \cdot \gamma, \quad (3.55)$$

$$f_{\text{break,eff}} = n_w \cdot f_0 \cdot \beta, \quad (3.56)$$

dove (κ, γ, β) sono i moltiplicatori di ottimizzazione. Questa scalatura lineare è motivata dall'assunzione che ogni strato di pellicola contribuisca in parallelo agli altri, raddoppiando rigidità, smorzamento e forza di rottura per strato aggiunto. L'ipotesi di linearità è una semplificazione: in pratica, gli strati interagiscono per attrito inter-strato, e la pre-tensione di ciascuno dipende dall'ordine di avvolgimento. Tuttavia, la scalatura lineare è un'approssimazione standard nei modelli a parametri concentrati per sistemi di ritenuta, e la griglia di ottimizzazione su $n_w \in \{1, 3, 6, 8\}$ permette al modello di adattarsi a configurazioni reali di avvolgimento.

3.4.4.5 Frizione inter-pacchi come parametro del modello

Il moltiplicatore ϕ non agisce direttamente sulla forza della molla, bensì sulla frizione laterale assegnata a ciascun collo tramite la primitiva `PyBullet changeDynamics`:

$$\mu_{\text{lat}} = \mu_{\text{lat},0} \cdot \phi, \quad (3.57)$$

dove $\mu_{\text{lat},0} = 0.6$ è il coefficiente di attrito laterale di riferimento (ridotto a 0.35 nel caso senza pellicola, $n_w = 0$). La frizione agisce come secondo meccanismo di contenimento: in assenza di pellicola, è l'unica forza che si oppone allo scivolamento reciproco dei colli. Durante la discussione dei risultati verrà mostrato come essa giocherà un ruolo fondamentale e fisicamente coerente: la frizione governa direttamente la soglia di avvio dello scivolamento, che determina se un collo rimane stabile o inizia a scivolare, cioè l'evento binario che domina la funzione di costo.

3.5 Ottimizzazione dei parametri

Un passaggio fondamentale nello sviluppo del simulatore fisico è la calibrazione dei parametri meccanici del modello, in particolare di quelli che governano il comportamento della pellicola estensibile che avvolge il carico sul pallet. Poiché tali parametri dipendono dalla tipologia di pellicola, dal numero di giri applicati e dalle condizioni operative del test, non è possibile determinarli analiticamente con precisione sufficiente: si rende quindi necessaria una procedura automatica di ottimizzazione basata sul confronto tra la simulazione e i dati reali acquisiti dai video dei test di slittamento.

Il processo è strutturato in tre fasi principali: estrazione delle traiettorie reali tramite YOLO, esecuzione di una grid search sui parametri meccanici, salvataggio e analisi dei risultati.

Il modulo `PhysicsOptimizer` e il runner di batch orchestrano l'intero flusso in modo completamente automatico su un insieme di video di calibrazione.

3.5.1 Struttura della grid search

L'approccio adottato per la ricerca dei parametri ottimali è una *grid search* esaustiva: lo spazio dei parametri viene discretizzato in una griglia multidimensionale e ogni combinazione viene valutata eseguendo una simulazione fisica completa, dopodiché viene selezionata la combinazione che minimizza la funzione di costo descritta nella Sezione 3.5.3.

3.5.1.1 Parametri ottimizzati

La griglia di ricerca è definita in configurazione e coinvolge cinque dimensioni, ovvero i parametri descritti in precedenza nella sezione 3.4.4:

- **Moltiplicatore di rigidità** (`stiffness_mult`): scala la rigidità elastica delle molle di pellicola rispetto al valore base $k_0 = 500 \text{ N/m}$ per giro. I valori testati sono $\{0.2, 0.6, 1.0, 1.8, 2.5\}$.
- **Moltiplicatore di smorzamento** (`damping_mult`): scala il coefficiente di smorzamento viscoso $c_0 = 3 \text{ N} \cdot \text{s/m}$ per giro. I valori testati sono $\{0.5, 1.5\}$.
- **Moltiplicatore di rottura** (`break_mult`): scala la forza massima sopportabile dalla pellicola prima della rottura $F_{\text{break},0} = 25 \text{ N}$ per giro. I valori testati sono $\{0.4, 0.8, 1.2, 1.6, 2.0\}$.
- **Moltiplicatore di attrito** (`friction_mult`): scala il coefficiente di attrito laterale tra i colli e il pallet. I valori testati sono $\{0.5, 0.8, 1.0, 1.3, 1.6\}$.

- **Numero di giri di pellicola** (`film.wraps`): numero intero di avvolgimenti applicati, che moltiplica linearmente rigidità, smorzamento e forza di rottura. I valori testati sono $\{1, 3, 6, 8\}$.

I valori base per ciascun parametro fisico vengono quindi modificati secondo le relazioni (3.54), (3.55), (3.56) descritte nella sezione 3.4.4.4 dove n_w è il numero di giri di pellicola.

Il numero totale di combinazioni nell'ipotesi di griglia completa è pertanto:

$$N_{\text{tasks}} = |\mathcal{W}| \times |\mathcal{K}| \times |\mathcal{C}| \times |\mathcal{B}| \times |\mathcal{F}| = 4 \times 5 \times 2 \times 5 \times 5 = 1000 \quad (3.58)$$

3.5.1.2 Caso speciale: video senza pellicola

Quando il parametro `target.wraps` è impostato a 0, il sistema riconosce che nel video il carico non è avvolto da pellicola estensibile. In questa configurazione la sola sorgente di coesione del carico è l'attrito tra i colli e tra i colli e il piano del pallet; le molle di film non vengono istanziate e i parametri k_{mult} , c_{mult} e b_{mult} perdono significato fisico.

La grid search viene di conseguenza ridotta alla sola dimensione del moltiplicatore di attrito:

$$N_{\text{tasks}}^{(w=0)} = |\mathcal{F}| = 5 \quad (3.59)$$

Contestualmente, la dinamica dei corpi rigidi viene regolata su valori più conservativi per i coefficienti di attrito radente ($\mu = 0.35 \cdot f_{\text{mult}}$), di attrito volvente e di smorzamento lineare/angolare, appropriati a un carico non vincolato. Questa separazione garantisce che i parametri calibrati nel caso con pellicola non vengano contaminati dai dati relativi a configurazioni senza pellicola, e viceversa.

3.5.2 Adattamento dinamico della griglia: rilevamento della stabilità

Prima di costruire la lista dei task, l'ottimizzatore analizza le traiettorie YOLO per classificare il video come *stabile* o *dinamico*. Per ogni pacco e per ogni frame viene calcolata la variazione angolare rispetto alla posizione iniziale

$$\Delta r = |r - r_0| \quad (3.60)$$

se tale variazione supera la soglia

$$\Delta r_{\text{stab}} = 30^\circ \quad (3.61)$$

il frame viene conteggiato come evento di caduta.

Viene quindi calcolato il rapporto:

$$\rho_{\text{fall}} = \frac{N_{\text{fall events}}}{N_{\text{total pkg-frames}}} \quad (3.62)$$

Se $\rho_{\text{fall}} < 0.05$ il video viene classificato come **stabile**: la griglia viene ristretta ai soli valori di $k_{\text{mult}} \geq 1.0$ e $b_{\text{mult}} \geq 1.0$, escludendo configurazioni con film debole che genererebbero falsi minimi, ovvero combinazioni che minimizzano l'errore posizionale solo perché simulano una caduta che nella realtà non avviene. Viceversa, se $\rho_{\text{fall}} \geq 0.05$ il video è classificato come **dinamico** e viene utilizzata la griglia completa per consentire anche la ricerca di configurazioni con film più cedevole.

3.5.3 Funzione di costo

La funzione di costo adottata è un errore quadratico medio (MSE) sul posizionamento tridimensionale dei colli, arricchito da un termine di penalità sugli eventi di caduta.

3.5.3.1 Termine di errore posizionale

Per ogni frame f e per ogni pacco i ancora in piedi al frame f nella sequenza YOLO, ovvero $f < f_{\text{fall},i}^{\text{YOLO}}$, dove $f_{\text{fall},i}^{\text{YOLO}} = -1$ indica che il pacco non cade mai, viene calcolato l'errore quadratico tra la posizione simulata $\hat{\mathbf{p}}_i^f$ e quella osservata $\mathbf{p}_i^f$:

$$E_{\text{pos}}^{f,i} = (\hat{x}_i^f - x_i^f)^2 + (\hat{y}_i^f - y_{0,i})^2 + (\hat{z}_i^f - z_i^f)^2 + 0.1 \cdot (\hat{r}_i^f - r_i^f)^2 \quad (3.63)$$

dove $y_{0,i}$ è la coordinata di profondità attesa per il pacco (determinata dal numero di layer e dalla sua posizione nella catasta), e il fattore 0.1 pondera il contributo dell'orientamento rispetto agli spostamenti lineari, riflettendo la minore affidabilità della stima angolare da YOLO. L'errore posizionale medio per frame è:

$$\bar{E}_{\text{pos}} = \frac{1}{N_{\text{eval}}} \sum_{f,i} E_{\text{pos}}^{f,i} \quad (3.64)$$

con N_{eval} pari al numero totale di coppie (f, i) valutate.

3.5.3.2 Termine di penalità sugli eventi di caduta

Un elemento essenziale della funzione di costo è la coerenza qualitativa tra la simulazione e la realtà riguardo alle cadute dei colli. Un pacco viene classificato come *caduto* nella simulazione se la sua quota scende di più di $\Delta z_{\text{fall}} = 0.15$ m

rispetto alla posizione iniziale, oppure se la sua variazione angolare supera $\Delta r_{\text{fall}} = 30^\circ$.

Per il rilevamento delle cadute sulle traiettorie YOLO viene invece richiesta una continuità minima di 5 frame consecutivi al di sopra delle soglie, per filtrare falsi positivi dovuti a rumore di rilevamento.

Per ciascun pacco i viene quindi confrontato l'esito binario:

- **Falso negativo:** il pacco cade nel video reale ma non nella simulazione.
- **Falso positivo:** il pacco cade nella simulazione ma non nel video reale.

In entrambi i casi viene aggiunta una penalità pari a:

$$P_{\text{fall}} = \lambda \cdot \bar{E}_{\text{pos}} \quad \text{con} \quad \lambda = 20 \quad (3.65)$$

dove il moltiplicatore $\lambda = 20$ garantisce che una discrepanza nell'esito di caduta domini sull'errore posizionale puro, costringendo l'ottimizzatore a privilegiare la correttezza topologica della dinamica rispetto alla minimizzazione metrica della traiettoria.

3.5.3.3 Funzione di costo complessiva

L'errore totale per una combinazione di parametri è:

$$\mathcal{L}(k_m, c_m, b_m, f_m, n_w) = \bar{E}_{\text{pos}} + \sum_{i \in \mathcal{P}} P_{\text{fall},i} \quad (3.66)$$

dove $\mathcal{P}$ è l'insieme dei pacchi per cui si verifica una discrepanza nell'evento di caduta tra simulazione e osservazione reale. La combinazione con $\mathcal{L}$ minima costituisce il risultato dell'ottimizzazione.

3.5.4 Esecuzione parallela e integrazione cinematica

3.5.4.1 Parallelismo multi-processo

Ciascun task della grid search costituisce una simulazione PyBullet indipendente e completamente isolata: ogni processo istanzia il proprio motore fisico in modalità headless, ovvero senza rendering grafico, esegue la simulazione frame per frame e restituisce la tupla $(k_m, c_m, b_m, f_m, n_w, \mathcal{L})$. Il modulo `PhysicsOptimizer` sfrutta la tecnologia multithread per lanciare tutti i task in parallelo, saturando i core disponibili della macchina. I risultati vengono raccolti tramite il pattern `as_completed`, aggiornando progressivamente il miglior risultato non appena un task viene completato.

3.5.4.2 Traiettoria sintetica della slitta

Un aspetto critico delle simulazioni di ottimizzazione è la gestione della traiettoria del pallet slitta. Poiché il rilevamento YOLO può produrre traiettorie rumorose o con frame mancanti, prima di ogni simulazione la traiettoria del pallet viene prima levigata con una media mobile di finestra $w = 3$ frame, e successivamente integrata cinematicamente dalla funzione apposita.

Quest'ultima modella il moto della slitta come un'accelerazione costante pari al valore g nominale del test estratto dal nome del file video, per esempio:

$$03g \rightarrow a = 0.3g = 2.94 \text{ m/s}^2 \quad (3.67)$$

seguita da una fase a velocità costante di crociera v_c :

$$x(t) = \begin{cases} x_0 & t < t_{\text{start}} \\ x_0 + \frac{1}{2}a t'^2 & t' \leq t_{\text{acc}} \\ x_0 + d_{\text{acc}} + v_c(t' - t_{\text{acc}}) & t' > t_{\text{acc}} \end{cases} \quad (3.68)$$

dove $t' = t - t_{\text{start}}$, $t_{\text{acc}} = v_c/a$ e $d_{\text{acc}} = \frac{1}{2}a t_{\text{acc}}^2$. La velocità di crociera v_c è stimata come rapporto tra la distanza totale percorsa dalla slitta e il tempo totale del moto. Questa traiettoria sintetica sostituisce i valori YOLO nei frame della slitta durante l'ottimizzazione, garantendo una cinematica regolare e fisicamente plausibile indipendente dal rumore di rilevamento.

Nei frame in cui la slitta risulta ancora ferma i pacchi vengono mantenuti bloccati tramite un *fixed constraint* di PyBullet, e vengono rilasciati istantaneamente al frame di inizio movimento. Questo meccanismo previene l'accumulo di forze di molla durante la fase stazionaria, che avrebbe introdotto un bias sistematico nella funzione di costo.

Per garantire la stabilità numerica della simulazione, ogni frame della traiettoria viene suddiviso in 4 sotto-passi di integrazione con interpolazione lineare della posizione e dell'orientamento del pallet, e il motore PyBullet esegue anch'esso 4 sotto-step interni per frame.

3.5.5 Processo di fine-tuning in batch

Il modulo di `batch-fine-tuning` automatizza l'applicazione della procedura di ottimizzazione su un intero dataset di video di calibrazione. I video sono organizzati in sotto-cartelle nominate con la massa del carico (es. 0.5/, 10.5/), e il nome di ciascun file video codifica il valore di accelerazione nominale del test (es. `prova_03g.avi` $\rightarrow a = 0.3g$). Il valore viene estratto automaticamente tramite espressione regolare sul nome del file.

Per ogni video il processo si articola nelle seguenti fasi:

1. **Analisi YOLO:** il video viene processato dal modello di rilevamento, che restituisce le traiettorie frame-per-frame di pallet e colli, con il relativo frame rate.
2. **Costruzione dei parametri:** vengono aggregati massa, numero di layer e accelerazione nominale in un dizionario di configurazione.
3. **Ottimizzazione:** viene istanziato un `PhysicsOptimizer` e viene lanciata la grid search parallela.
4. **Salvataggio risultati:** l'intera tabella dei risultati $(k_m, c_m, b_m, f_m, n_w, \mathcal{L})$, ordinata per errore crescente, viene esportata in un file di output con nome derivato dal video sorgente.

Al termine del batch viene prodotto un riepilogo con il numero di video elaborati con successo e quelli falliti, con i rispettivi percorsi dei file di output generati.

3.5.6 Risultati e selezione dei parametri ottimali

Al termine dell'ottimizzazione per ciascun video, il sistema seleziona la combinazione di parametri $(k_m^*, c_m^*, b_m^*, f_m^*, n_w^*)$ che minimizza $\mathcal{L}$. I file di output prodotti permettono inoltre un'analisi a posteriori della superficie di errore: è possibile, ad esempio, verificare la sensibilità del modello rispetto a ciascun parametro individuale, individuare degenerazioni (più combinazioni con errore simile) o rilevare comportamenti non monotoni che possano suggerire la necessità di una griglia più fine in un intorno del minimo trovato.

I parametri ottimali vengono quindi applicati alla simulazione interattiva principale per le sessioni di validazione e di analisi.

Capitolo 4

Ottimizzazione dei parametri e risultati

Quest'ultimo capitolo riporta i risultati della campagna sperimentale di fine-tuning dei parametri fisici del modello di simulazione del film di avvolgimento. L'obiettivo è identificare le configurazioni parametriche che minimizzano la funzione di perdita composita $\mathcal{L}$ (MSE) tra la traiettoria simulata e quella misurata sperimentalmente, e di valutare in modo rigoroso il contributo statistico di ciascun parametro in modo da ottenere un ambiente il più veritiero rispetto alla realtà per ogni video in modo da poter modificarne i parametri e capire come si comporterebbe in diversi scenari nel mondo reale.

4.1 Descrizione del protocollo sperimentale

Il dataset su cui è stato condotto il fine-tuning, descritto approfonditamente nel capitolo 2, comprende **103 video** acquisiti sperimentalmente, ciascuno relativo a una specifica combinazione di prodotto, massa e livello di accelerazione. Le masse dei singoli oggetti caricati sul pallet testati appartengono all'insieme $\{0.5, 5.0, 6.0, 10.0, 10.5\}$ kg, mentre le accelerazioni di riferimento sono $\{0.1, 0.3, 0.4, 0.5\}g$.

Per ogni video si è effettuata una ricerca esaustiva su una griglia discreta di cinque parametri fisici, descritti in Tabella 4.1. Il numero complessivo di combinazioni valutate è stato **34 328**, con una media di 333.3 combinazioni per video.

La funzione di perdita, descritta esaustivamente nella sezione 3.5.3 del capitolo 3, adottata è il *Mean Squared Error*

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{T} \sum_{t=1}^T \|\mathbf{x}_{\text{sim}}(t; \boldsymbol{\theta}) - \mathbf{x}_{\text{exp}}(t)\|^2, \quad (4.1)$$

Parametro	Simbolo	Livelli	Valori testati
Numero di avvolgimenti	N_w	7	$\{0, 1, 2, 3, 4, 6, 8\}$
Moltiplicatore di rigidezza	k_s	3	$\{1.0, 1.8, 2.5\}$
Moltiplicatore di smorzamento	k_d	3	$\{0.5, 1.0, 1.5\}$
Moltiplicatore di rottura	k_b	4	$\{1.0, 1.2, 1.6, 2.0\}$
Moltiplicatore di attrito	k_f	16	$\{0.1, 0.2, \dots, 1.6\}$

Tabella 4.1: Parametri fisici oggetto di fine-tuning e relativi spazi di ricerca.

dove $\mathbf{x}_{\text{sim}}$ e $\mathbf{x}_{\text{exp}}$ sono, rispettivamente, la traiettoria simulata e quella sperimentale, e $\boldsymbol{\theta} = (N_w, k_s, k_d, k_b, k_f)$ è il vettore dei parametri.

Lo spazio di ricerca relativo al moltiplicatore di attrito è significativamente più ampio rispetto a quello degli altri parametri, poiché viene riportato l'intero dominio esplorato. Tuttavia, tale intervallo viene utilizzato nella sua completezza esclusivamente nei casi dei video senza pellicola, analizzati in dettaglio nella Sezione 3.5.1.2.

4.2 Distribuzione globale della funzione di perdita

4.2.1 Statistica descrittiva sull'intero spazio di ricerca

La Tabella 4.2 riassume la statistica descrittiva di $\mathcal{L}$ calcolata su tutte le 34 328 valutazioni.

Statistica	Tutti i run ($n = 34\,328$)	Best-per-video ($n = 103$)
Minimo	1.223×10^{-3}	1.223×10^{-3}
Massimo	1.387×10^4	2.316×10^3
Media (μ)	142.16	34.28
Dev. std. (σ)	474.16	232.19
Mediana	6.210	0.158
Q1 (25° percentile)	5.10×10^{-2}	9.57×10^{-3}
Q3 (75° percentile)	89.06	2.572
IQR	89.01	2.563
Asimmetria	8.364	9.542
Curtosi (eccesso)	112.55	94.01
IC 95% inf.	137.15	-10.56
IC 95% sup.	147.18	79.12

Tabella 4.2: Statistica descrittiva di $\mathcal{L}$ su tutti i run e sul sottoinsieme delle configurazioni migliori per video.

La distribuzione è fortemente asimmetrica verso destra (asimmetria = 8.36) e leptocurtica (curtosi in eccesso = 112.55), indicando la presenza di una coda pesante dovuta a configurazioni parametriche lontane dall'ottimo. Fisicamente, questo implica che un piccolo numero di configurazioni profondamente errate degrada fortemente la simulazione, mentre la maggior parte produce errori moderati. Il rapporto media/media (142.16/6.21 $\approx$ 22.9) conferma che la media è dominata da queste combinazioni con errore molto elevato. La mediana ($\mathcal{L}_{50} = 6.21$) e il Q1 ($\mathcal{L}_{25} = 5.10 \times 10^{-2}$) sono perciò diagnostici più robusti della tendenza centrale.

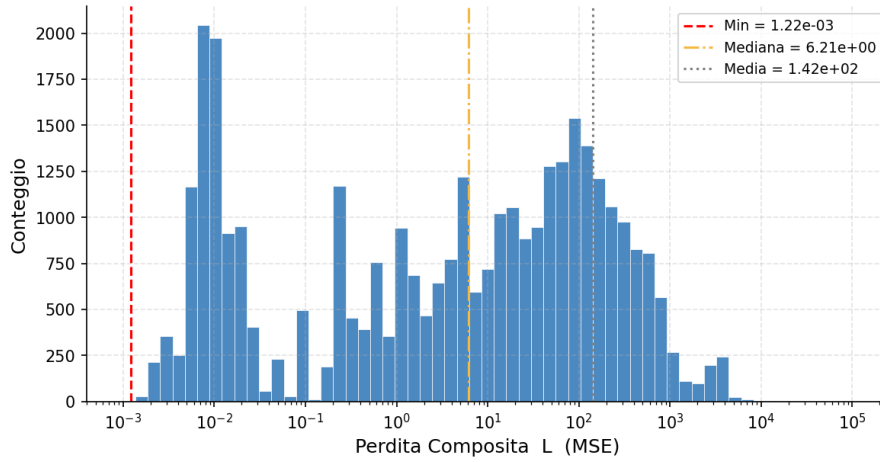


Figura 4.1: Istogramma della funzione di perdita $\mathcal{L}$ su tutti i 34 328 run (scala logaritmica sull'asse x). Le linee verticali tratteggiate indicano il minimo (rosso), la mediana (arancio) e la media (grigio).

4.2.2 Distribuzione delle configurazioni ottimali

Considerando esclusivamente la configurazione a minimo MSE per ciascuno dei 103 video (*best-per-video*), la distribuzione di $\mathcal{L}$ rimane asimmetrica (asimmetria = 9.54, curtosi = 94.01) ma la mediana scende a 0.158, un ordine di grandezza inferiore rispetto ai tutti i run.

Il Q1 si attesta a 9.57×10^{-3} : per oltre il 25% dei video si ottiene dunque una perdita inferiore al 1% in unità normalizzate. La distribuzione dell'errore suddivisa per fasce mutuamente esclusive è dettagliata nella Tabella 4.3.

Nel 64.1% dei casi, quindi, il modello riesce a riprodurre il comportamento reale con errore inferiore all'unità normalizzata, indicando una buona capacità di adattamento del framework.

La configurazione assoluta migliore è stata ottenuta con $\mathcal{L}^* = 1.223 \times 10^{-3}$ e parametri $(N_w, k_s, k_d, k_b, k_f) = (8, 2.5, 1.5, 1.6, 1.0)$ mentre la configurazione

peggiore raggiunge una loss di $\mathcal{L}_{\max} = 2316.24$, suggerendo che in tale scenario il modello fisico non riesce a riprodurre fedelmente il comportamento dinamico osservato probabilmente per la condizione critica di luminosità e contrasto del video che non permette di estrarre correttamente la traiettoria da analizzare.

Intervallo Errore MSE ($\mathcal{L}$)	Numero di Video	Percentuale
$\mathcal{L} < 10^{-2}$	27	26.2%
$10^{-2} \leq \mathcal{L} < 10^{-1}$	22	21.4%
$10^{-1} \leq \mathcal{L} < 1.0$	17	16.5%
$1.0 \leq \mathcal{L} \leq 10$	22	21.4%
$10 < \mathcal{L} \leq 100$	11	10.7%
$\mathcal{L} > 100$	4	3.8%
Totale	103	100%

Tabella 4.3: Distribuzione dell'errore MSE ($\mathcal{L}$) per le configurazioni *best-per-video*.

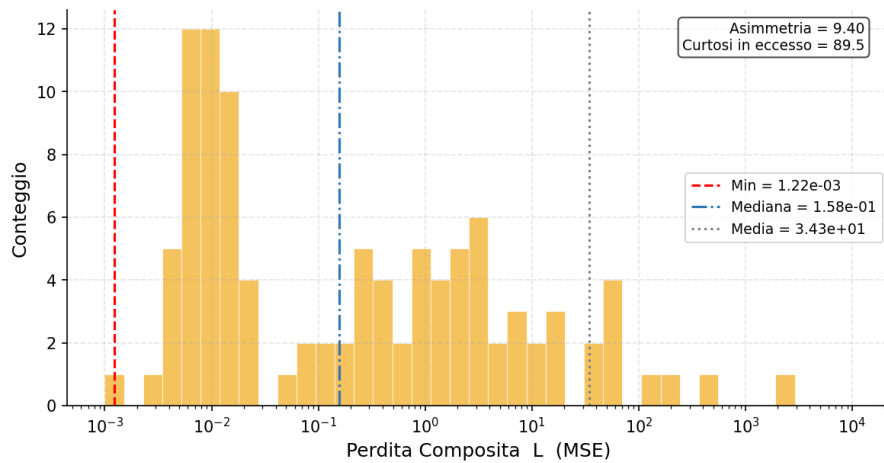


Figura 4.2: Istogramma di $\mathcal{L}$ per le configurazioni best-per-video ($n = 103$, scala logaritmica). In alto a destra sono riportati i valori di asimmetria e curtosi in eccesso.

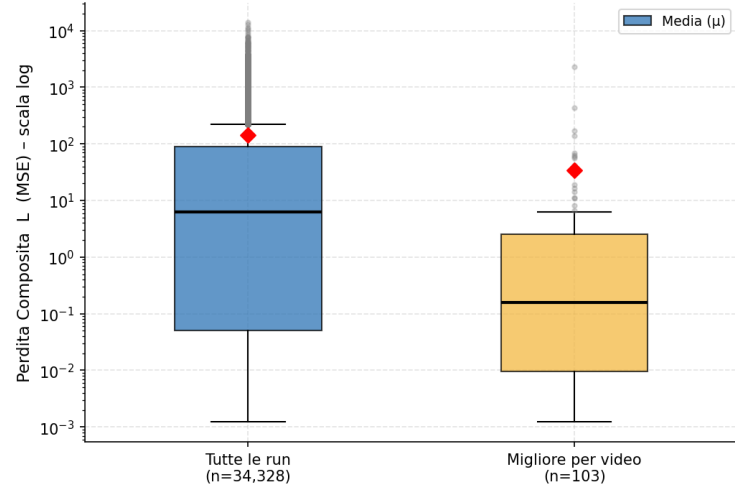


Figura 4.3: Confronto tra la distribuzione di $\mathcal{L}$ su tutti i run e sulle configurazioni best-per-video (scala logaritmica). Il diamante rosso indica la media aritmetica.

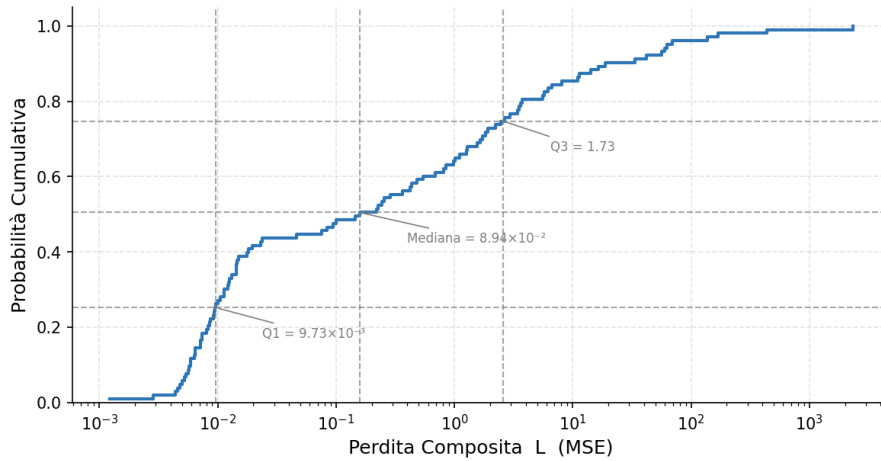


Figura 4.4: Funzione di ripartizione empirica (ECDF) di $\mathcal{L}$ per le configurazioni best-per-video. Le linee tratteggiate evidenziano i quantili Q1, mediana e Q3 derivati dalla Tabella 4.2.

4.3 Analisi per gruppi: massa e accelerazione

4.3.1 Effetto della massa del pallet

La Tabella 4.4 riporta la statistica descrittiva di $\mathcal{L}^*$ (best-per-video) stratificata per classe di massa.

Massa (kg)	n	μ	σ	Min	Mediana	Max
0.5	18	1.184	2.979	1.22×10^{-3}	0.015	11.37
5.0	2	0.00677	0.00047	6.44×10^{-3}	0.0068	0.0071
6.0	2	84.18	119.03	1.21×10^{-2}	84.18	168.35
10.0	16	3.506	6.686	6.48×10^{-3}	0.013	18.84
10.5	65	50.53	291.14	2.83×10^{-3}	0.688	2316.24

Tabella 4.4: Statistica della funzione di perdita best-per-video per classe di massa del singolo oggetto caricato sul pallet.

Il test ANOVA a una via applicato alle cinque classi di massa fornisce $F(4, 98) = 0.267$, $p = 0.898$: non è possibile rigettare l'ipotesi nulla di uguaglianza delle medie al livello $\alpha = 0.05$. Tuttavia, bisogna chiarire che questo risultato *non significa* un'assenza di effetto fisico della massa. La mancata significatività è infatti pesantemente influenzata dalla forte dispersione intra-gruppo (la classe 10.5 kg presenta una deviazione standard di 291.14 contro 0.00047 della classe 5.0 kg) e dallo sbilanciamento numerico del dataset. Ciò segnala che l'assunzione di omoschedasticità dell'ANOVA è violata, per cui il risultato va interpretato con estrema cautela; tuttavia anche test non parametrici (si veda la Sezione 4.7) non mostrano un effetto sistematico. La dispersione elevata della classe 10.5 kg è imputabile a scenari con configurazioni di avvolgimento particolarmente difficili da calibrare.

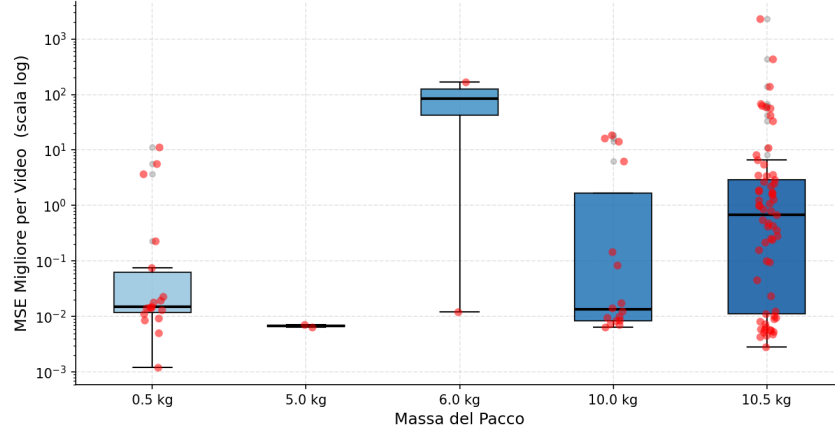


Figura 4.5: Distribuzione di $\mathcal{L}^*$ per classe di massa del pallet. I box-plot mostrano Q1, mediana e Q3; i punti rossi indicano le singole osservazioni (scala logaritmica).

4.3.2 Effetto del livello di accelerazione

La Tabella 4.5 riporta la statistica per livello di accelerazione.

Accel. (g)	n	μ	σ	Min	Mediana	Max
0.1	2	251.16	257.73	68.92	251.16	433.40
0.3	59	11.41	30.67	1.22×10^{-3}	0.422	168.35
0.4	19	122.69	531.20	5.03×10^{-3}	0.046	2316.24
0.5	23	1.09	2.07	4.55×10^{-3}	0.018	6.65

Tabella 4.5: Statistica della funzione di perdita best-per-video per livello di accelerazione.

Il test ANOVA fornisce $F(3, 98) = 1.870$, $p = 0.140$: l'effetto dell'accelerazione non risulta statisticamente significativo a livello $\alpha = 0.05$. È comunque degno di nota che la classe $0.4 g$ presenta la varianza più elevata ($\sigma = 531.20$), spinta dal caso estremo con $\mathcal{L} = 2316.24$; la corrispondente mediana (0.046) è invece tra le più basse. Questo significa che il modello è generalmente molto stabile e robusto anche ad accelerazioni elevate, riuscendo a calibrare la maggioranza dei video con buona precisione, ma esistono rari casi critici che falliscono drasticamente generando outlier. La classe $0.1 g$ è rappresentata da soli 2 video, limitando la potenza statistica del confronto.

4.4 Distribuzione dei parametri ottimali

La Tabella 4.6 descrive la distribuzione di ciascun parametro nell'insieme delle 103 configurazioni best-per-video.

Parametro	Moda (freq.)	μ	σ	Mediana
N_w (Film Wraps)	8 (34.95%)	4.75	2.99	6
k_s (Stiffness Mult)	1.0 (46.6%)	1.68	0.68	1.8
k_d (Damping Mult)	1.5 (51.5%)	1.05	0.48	1.5
k_b (Break Mult)	2.0 (46.6%)	1.67	0.35	1.6
k_f (Friction Mult)	1.6 (26.2%)	1.07	0.42	1.0

Tabella 4.6: Distribuzione dei parametri nelle configurazioni ottimali (best-per-video, $n = 103$). La moda è evidenziata in grassetto.

4.4.1 Numero di avvolgimenti (N_w)

Il valore più ricorrente è stato riscontrato con $N_w = 8$ (34.95%), coerentemente con l'ipotesi che un maggior numero di strati incrementa la rigidità del sistema e migliora la fedeltà della simulazione. Il 7.77% delle configurazioni ottimali corrisponde a $N_w = 0$ (assenza di film), relativo a video eseguiti senza avvolgimento.

4.4.2 Moltiplicatore di rigidità (k_s)

Il valore unitario $k_s = 1.0$ è quello più frequente (46.6%), seguito da $k_s = 2.5$ (35.9%): la distribuzione bimodale suggerisce la presenza di due regimi fisici distinti nella risposta del sistema.

4.4.3 Moltiplicatore di smorzamento (k_d)

Il valore $k_d = 1.5$ è la moda con il 51.5% dei casi, mentre $k_d = 0.5$ rappresenta il 40.8%: la distribuzione è quasi bimodale, e i due estremi dell'intervallo sono complessivamente preferiti rispetto al valore intermedio ($k_d = 1.0$, solo 7.77%). Fisicamente, il sistema tende a preferire configurazioni estreme: o con una forte dissipazione dell'energia o con una marcata libertà dinamica, mentre un comportamento viscoso intermedio risulta raramente ottimale per replicare il test reale.

4.4.4 Moltiplicatore di rottura (k_b)

Il valore $k_b = 2.0$ è la moda (46.6%), con $k_b = 1.6$ come secondo valore più frequente (29.1%): la distribuzione mostra una preferenza per valori elevati di resistenza alla rottura del film.

4.4.5 Moltiplicatore di attrito (k_f)

Il parametro di attrito presenta la maggiore dispersione (16 livelli testati), poiché per i video senza pellicola lo spazio di ricerca dell'attrito aumenta notevolmente in quanto non è più presente il contributo dell'attrito generato dalla pellicola, mentre rimangono dominanti gli effetti di attrito statico tra pallet, prodotti e superficie di appoggio, oltre all'attrito tra i prodotti stessi. I valori modali sono $k_f = 1.6$ (26.2%) e $k_f = 1.0$ (20.4%), con una distribuzione che copre l'intero intervallo $[0.1, 1.6]$, riflettendo la forte dipendenza contestuale di questo parametro dal tipo di superficie del prodotto e dal numero di avvolgimenti.

4.5 Analisi di sensibilità

Metodologicamente, è essenziale specificare che l'analisi di sensibilità qui presentata misura l'effetto medio *marginale* sull'intero spazio di ricerca, non l'effetto locale attorno al punto di ottimo. Di conseguenza, quantifica l'escursione relativa della perdita media al variare di un singolo parametro, pur mantenendo gli altri fissi alla loro media globale:

$$\Delta_j = \frac{\mu_{\max}^{(j)} - \mu_{\min}^{(j)}}{\mu_{\min}^{(j)}} \times 100\%, \quad (4.2)$$

dove $\mu_{\max}^{(j)}$ e $\mu_{\min}^{(j)}$ sono, rispettivamente, la perdita media massima e minima rispetto al j -esimo parametro. Questa distinzione evita interpretazioni errate, poiché parametri meno sensibili globalmente potrebbero comunque avere una forte incidenza locale per raggiungere la convergenza finale.

La Tabella 4.7 riporta il ranking per influenza decrescente.

Il parametro di attrito k_f risulta di gran lunga il più influente ($\Delta = 1650.95\%$), seguito da N_w ($\Delta = 1199.30\%$) e da k_d ($\Delta = 973.61\%$). Il moltiplicatore di rigidità k_s è il parametro meno determinante, con un'escursione relativa di soli 59.83%, quasi due ordini di grandezza inferiore rispetto a k_f . È importante osservare che il valore ottimo marginale di N_w risulta 0 (nessun avvolgimento): ciò riflette l'effetto statistico dei video senza pellicola, che presentano una perdita media inferiore grazie alla ridotta complessità dinamica

Rank	Parametro	Δ_j [%]	Valore ottimo marginale
1	Friction Mult (k_f)	1650.95	1.5
2	Film Wraps (N_w)	1199.30	0
3	Damping Mult (k_d)	973.61	1.0
4	Break Mult (k_b)	904.55	1.0
5	Stiffness Mult (k_s)	59.83	1.0

Tabella 4.7: Ranking di sensibilità dei parametri fisici. La colonna *Valore ottimo marginale* indica il livello con la perdita media minima nella griglia di ricerca globale.

del sistema non vincolato. In una valutazione condizionata ai soli video con film ($N_w > 0$), l'ottimo si sposta verso valori elevati ($N_w = 8$), in accordo con la distribuzione modale delle configurazioni best-per-video (Tabella 4.6).

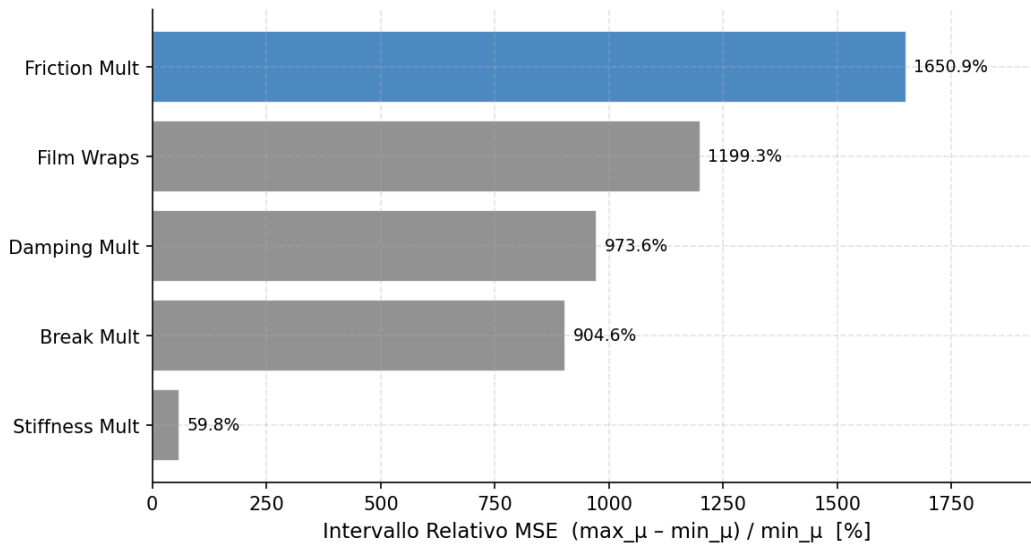


Figura 4.6: Ranking di sensibilità dei parametri fisici. L'asse orizzontale riporta Δ_j (equazione (4.2)); la barra evidenziata in blu corrisponde al parametro più influente.

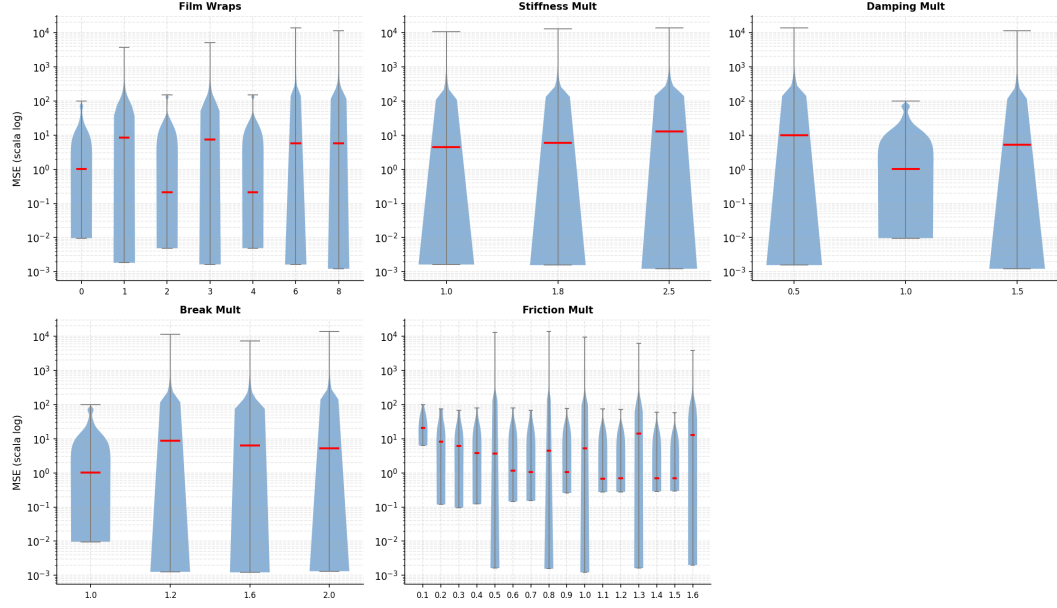


Figura 4.7: Distribuzione di $\mathcal{L}$ per ciascun valore discreto dei cinque parametri (violin plot, scala logaritmica). La linea rossa indica la mediana di ciascun gruppo.

4.6 Analisi delle correlazioni

La Tabella 4.8 riporta i coefficienti di correlazione di Pearson (r) e di Spearman (ρ) tra ciascun parametro e la funzione di perdita, con i relativi valori- p .

Parametro	Pearson r	Sig.	Spearman ρ	Sig.
N_w (Film Wraps)	+0.073	***	+0.011	*
k_s (Stiffness Mult)	+0.057	***	+0.071	***
k_d (Damping Mult)	-0.046	***	-0.053	***
k_b (Break Mult)	+0.018	***	-0.022	***
k_f (Friction Mult)	+0.011	*	+0.064	***

Tabella 4.8: Correlazioni parametro $\rightarrow \mathcal{L}$ ($n = 34\,328$). Significatività:

*** $p < 0.001$, * $p < 0.05$.

Benché tutti i coefficienti siano statisticamente significativi (ampia dimensione campionaria $n = 34\,328$), le loro magnitudini sono molto ridotte ($|r|, |\rho| < 0.10$ in tutti i casi), e tutti rientrano nella categoria *debole* secondo i criteri di Cohen. Tuttavia, è cruciale esplicitare che avere correlazioni così deboli non implica che i parametri siano fisicamente irrilevanti; indica, piuttosto, che le relazioni dominanti sono largamente non lineari e strettamente dipendenti

dalle interazioni (rendendo inefficace l'analisi univariata). Questo risultato è perfettamente coerente con un paesaggio di perdita pesantemente non lineare e non monotono: le correlazioni lineari catturano solo una frazione trascurabile della varianza di $\mathcal{L}$.

Il segno negativo del coefficiente di Pearson per k_d ($r = -0.046$) indica che un incremento dello smorzamento tende, in media, a ridurre la perdita, confermando l'osservazione sulla moda ottimale $k_d = 1.5$ riportata in Sezione 4.4. La discordanza di segno tra Pearson e Spearman per k_b ($r = +0.018$ vs $\rho = -0.022$) evidenzia la presenza di una non-monotonicità nella relazione tra il moltiplicatore di rottura e la perdita.

4.7 Test non parametrico di Kruskal-Wallis

Per verificare se le distribuzioni di $\mathcal{L}$ differiscono significativamente tra i diversi livelli di ciascun parametro, è stato applicato il test non parametrico di Kruskal-Wallis (alternativa all'ANOVA indipendente dall'assunzione di normalità e omoschedasticità):

$$H_0 : \text{la distribuzione di } \mathcal{L} \text{ è identica a tutti i livelli del parametro.} \quad (4.3)$$

I risultati sono riportati in Tabella 4.9.

Parametro	H	p -valore	Sig.	Livelli
N_w (Film Wraps)	299.00	< 0.001	***	7
k_s (Stiffness Mult)	176.18	< 0.001	***	3
k_d (Damping Mult)	100.23	< 0.001	***	3
k_b (Break Mult)	23.28	3.5×10^{-5}	***	4
k_f (Friction Mult)	217.55	< 0.001	***	16

Tabella 4.9: Test di Kruskal-Wallis: effetto di ciascun parametro sulla distribuzione di $\mathcal{L}$ ($n = 34\,328$).

L'ipotesi nulla è rigettata per tutti e cinque i parametri al livello $\alpha = 0.001$: ciascuno influenza in modo statisticamente significativo la distribuzione di $\mathcal{L}$. Va sottolineato, tuttavia, che il test rileva l'esistenza di differenze distribuzionali ma non identifica quali livelli siano migliori. Per questo motivo i risultati vanno necessariamente interpretati alla luce delle heatmap e dell'analisi di sensibilità. Il valore H più elevato è associato a N_w ($H = 299.00$), a conferma del ruolo predominante degli avvolgimenti nel determinare la risposta del sistema. k_f segue con $H = 217.55$, coerentemente con il ranking di sensibilità (Sezione 4.5). Il moltiplicatore di rottura k_b presenta il valore H più basso ($H = 23.28$), pur restando fortemente significativo.

4.8 Analisi delle interazioni tra parametri

L'analisi univariata non cattura gli effetti di interazione tra parametri. La Tabella 4.10 e la Figura 4.8 riportano le griglie di $\mathcal{L}$ media per le cinque coppie parametriche selezionate.

$k_s \backslash k_b$	1.0	1.2	1.6	2.0
1.0	15.31	110.72	113.58	112.75
1.8	–	134.32	128.36	150.46
2.5	–	158.79	176.62	198.13

Tabella 4.10: Interazione Stiffness Mult $\times$ Break Mult: $\mathcal{L}$ media per cella della griglia. Il minimo globale è evidenziato in grassetto.

La Tabella 4.10 e la Figura 4.8 riportano le griglie di $\mathcal{L}$ media per le cinque coppie parametriche selezionate. Si nota l'assenza di dati per le combinazioni caratterizzate da alta rigidezza ($k_s \in \{1.8, 2.5\}$) e bassa soglia di rottura ($k_b = 1.0$); queste configurazioni generano picchi di forza istantanei che portano a rotture premature e instabilità numerica nella simulazione, impedendo il calcolo di una metrica di errore valida.

Il minimo globale della griglia $k_s \times k_b$ si trova nell'angolo $(k_s, k_b) = (1.0, 1.0)$, con $\mathcal{L} = 15.31$. L'effetto di interazione è evidente: aumentando k_s da 1.0 a 2.5, la perdita media a $k_b = 2.0$ passa da 112.75 a 198.13 (+75.7%), mentre a $k_b = 1.2$ l'incremento è del 43.4%. L'interazione si qualifica pertanto come *antagonista*: valori elevati di entrambi i parametri si combinano in modo super-additivo nel peggiorare la calibrazione. Da un punto di vista fisico, aumentare simultaneamente la rigidezza (k_s) e la soglia di rottura (k_b) costringe il sistema in una condizione eccessivamente vincolata. L'impossibilità del film di cedere o allungarsi plasticamente impedisce di dissipare l'energia e restituisce traiettorie molto meno realistiche, allontanando fortemente la simulazione dal dato empirico.

Per la coppia $k_d \times k_s$, il valore minimo è $\mathcal{L} = 15.31$ a $(k_d, k_s) = (1.0, 1.0)$: il valore intermedio di smorzamento si combina virtuosamente con la rigidezza unitaria, mentre configurazioni con $k_d = 0.5$ e qualsiasi k_s presentano perdite medie superiori a 120.

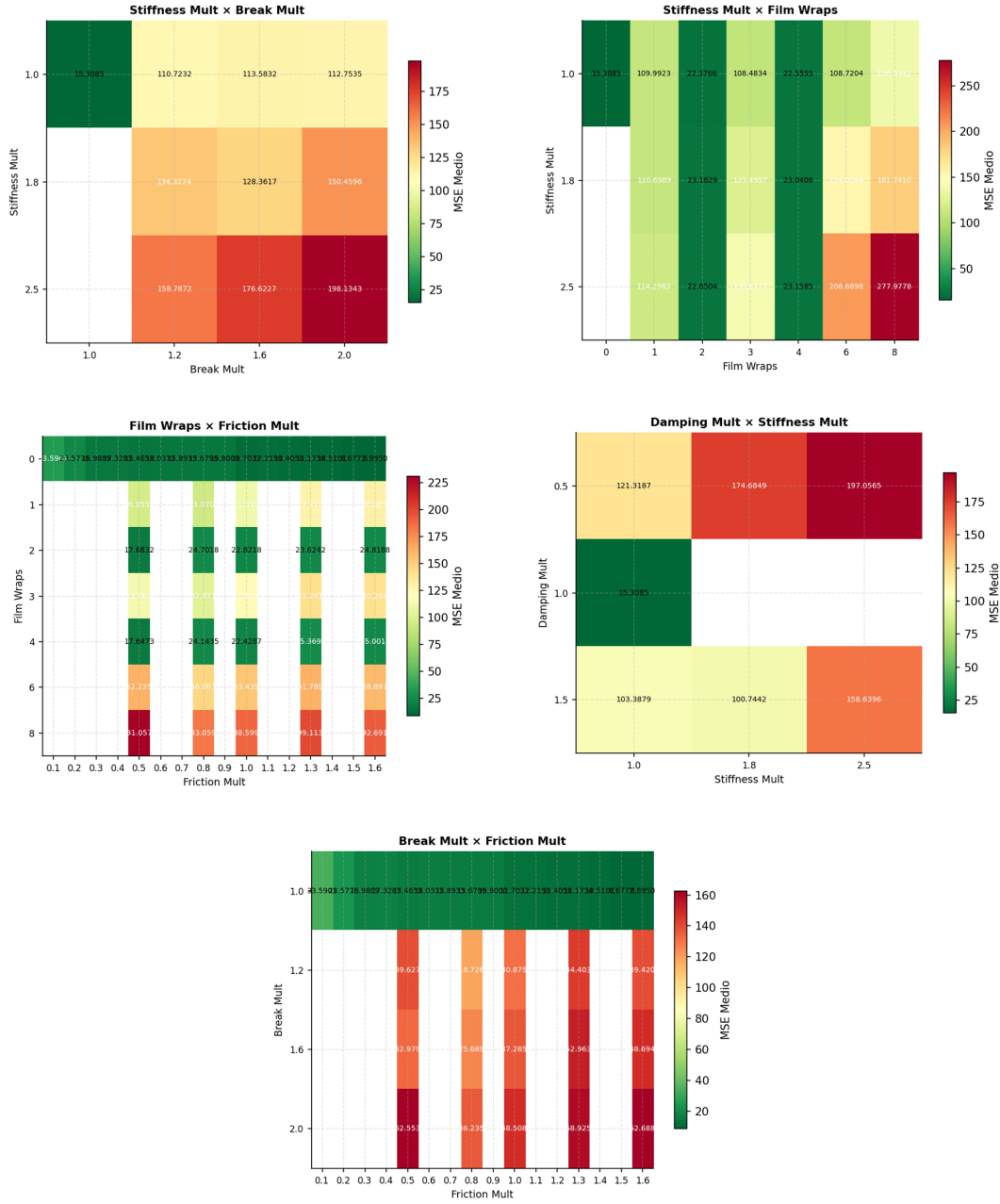


Figura 4.8: Mappe di calore della $\mathcal{L}$ media per le cinque coppie parametriche selezionate. La scala cromatica va dal verde (minimo MSE) al rosso (massimo MSE).

4.9 Analisi di convergenza

L'analisi di convergenza quantifica la *piattezza* del paesaggio di perdita attorno all'ottimo: per ciascun video, si misura la percentuale di configurazioni la cui perdita non supera la perdita ottimale di una frazione ϵ :

$$\Pi(\epsilon) = \frac{|\{\boldsymbol{\theta} : \mathcal{L}(\boldsymbol{\theta}) \leq \mathcal{L}^*(1 + \epsilon)\}|}{|\Theta|} \times 100\%, \quad (4.4)$$

dove $|\Theta|$ è il numero totale di configurazioni valutate per quel video. La Tabella 4.11 riporta i valori medi e la deviazione standard di $\Pi(\epsilon)$ per $\epsilon \in \{1\%, 5\%, 10\%, 20\%\}$.

Soglia ϵ	$\bar{\Pi}$ [%]	σ_{Π} [%]
Top 1%	1.90	3.80
Top 5%	6.25	10.7
Top 10%	12.0	17.6
Top 20%	19.7	25.4

Tabella 4.11: Convergenza media per soglia ϵ (media e dev. std. su 103 video).

4.9.0.1 Video con paesaggio piatto

Cinque video presentano $\Pi(20\%) > 80\%$, indicando che oltre quattro quinti delle configurazioni si trovano entro il 20% del valore ottimale (paesaggio quasi piatto):

- video1_04g: $\Pi(1\%) = 3.9\%$, $\Pi(20\%) = 99.7\%$;
- video2_03g: $\Pi(1\%) = 0.6\%$, $\Pi(20\%) = 95.8\%$;
- video3_03g: $\Pi(1\%) = 1.4\%$, $\Pi(20\%) = 87.5\%$;
- video4_04g: $\Pi(1\%) = 32.2\%$ (massimo assoluto), $\Pi(20\%) = 80.6\%$.
- video5_05g: $\Pi(1\%) = 1.1\%$, $\Pi(20\%) = 81.9\%$;

Un paesaggio piatto indica che il modello è poco sensibile alla scelta dei parametri in quel particolare scenario cinematografico, e l'ottimo trovato ha robustezza elevata rispetto a piccole perturbazioni parametriche.

4.9.0.2 Video con ottimo isolato

All'opposto, 29 video presentano $\Pi(20\%) < 2\%$, ovvero meno del 2% delle configurazioni rientra entro il 20% dell'ottimo. Esempi rappresentativi sono: **video6_03g** ($\Pi(20\%) = 0.3\%$), **video7_03g** ($\Pi(20\%) = 0.3\%$) e **video8_03g** ($\Pi(20\%) = 0.8\%$, $\mathcal{L}^* = 1.223 \times 10^{-3}$). In questi casi il paesaggio è pronunciato e l'ottimo ben isolato: il fine-tuning ha individuato una configurazione parametrica precisa e riproducibile.

4.10 Variabilità cross-video e generalizzazione

La Figura 4.9 mostra i valori $\mathcal{L}^*$ ordinati in modo crescente su tutti i 103 video. La distribuzione è marcatamente eterogenea: circa la metà dei video (mediana = 0.158) è calibrabile con perdita inferiore all'unità, mentre una minoranza ($\sim 15\%$) presenta perdite superiori a 10. Tale eterogeneità riflette la variabilità intrinseca delle configurazioni sperimentali: geometria del pallet, rigidità strutturale, interfaccia film-prodotto e dinamica del trasporto differiscono tra scenari, rendendo alcuni casi strutturalmente più difficili da replicare in simulazione.

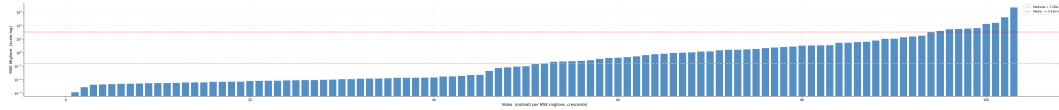


Figura 4.9: Valori di $\mathcal{L}^*$ (best-per-video) ordinati in modo crescente. La linea arancione tratteggiata indica la mediana (0.158), quella rossa la media aritmetica (34.28). Scala logaritmica sull'asse y .

L'elevato rapporto $\mu/\text{mediana} \approx 217$ indica che la media è tirata verso l'alto da un ristretto numero di video difficili. L'intervallo interquartile $[Q1, Q3] = [9.57 \times 10^{-3}, 2.572]$ mostra che il 50% centrale dei video è compreso in un intervallo di circa tre decenni logaritmiche, a conferma dell'ampiezza della variabilità.

4.11 Sintesi dei risultati

I principali risultati della campagna di fine-tuning su 103 video e 34 328 valutazioni si possono riassumere come segue:

1. **Distribuzione asimmetrica della perdita.** Sia sull'intero spazio di ricerca che nel sottoinsieme best-per-video, $\mathcal{L}$ segue una distribuzione pesantemente asimmetrica (skewness > 8) e leptocurtica. Ciò dimostra che gran parte dei risultati si attesta su errori moderati, ma poche combinazioni profondamente errate degradano severamente la simulazione.
2. **Alta calibrabilità per la maggioranza dei video.** Il modello offre una robusta capacità adattiva: il 64.1% dei video raggiunge $\mathcal{L}^* < 1$, e il 26.2% scende sotto 10^{-2} ; solo il 3.9% supera la soglia critica di 100.
3. **Attrito e avvolgimenti come parametri dominanti.** Il ranking di sensibilità indica k_f ($\Delta = 1651\%$) e N_w ($\Delta = 1199\%$) come i parametri che mediano l'escursione globale dell'errore, confermato dal test di Kruskal-Wallis.
4. **Rigidezza parametricamente irrilevante.** Il moltiplicatore di rigidità k_s presenta $\Delta = 59.83\%$, circa 28 volte inferiore a k_f : il suo ruolo marginale medio è trascurabile nel determinare la bontà della calibrazione globale rispetto agli altri parametri.
5. **Correlazioni lineari deboli.** Tutte le correlazioni sono statisticamente significative ma di magnitudine inferiore a 0.10. Piuttosto che irrilevanza, questo denuncia la presenza di forti dipendenze non lineari e l'impossibilità di usare metriche univariate.
6. **Interazioni antagoniste rilevanti.** L'effetto combinato di due parametri in controtendenza peggiora drasticamente il risultato. Ad esempio, aumentare rigidità e soglia di rottura simultaneamente rende il sistema troppo vincolato per dissipare energia.
7. **Paesaggio eterogeneo tra video.** L'analisi di convergenza mostra una forte bipartizione: $\sim 28\%$ dei video presenta un ottimo ben isolato ($\Pi(20\%) < 2\%$), mentre $\sim 5\%$ presenta un paesaggio quasi piatto ($\Pi(20\%) > 80\%$).
8. **Massa e accelerazione non discriminanti in ANOVA.** I p-value non risultano significativi, tuttavia la fortissima dispersione intra-gruppo suggerisce che il modello gestisca bene la maggioranza delle condizioni, generando outlier solo in casi estremi (specialmente a $0.4 g$).

Conclusione

Questo lavoro ha affrontato il problema della valutazione predittiva della stabilità dei carichi pallettizzati, proponendo un framework che integra computer vision e simulazione fisica in una pipeline coerente e automatizzata.

Il punto di partenza è stato lo sviluppo di un sistema di object detection basato su YOLOv8 con bounding box orientate, addestrato su un dataset costruito ad hoc a partire da video reali di crash test. I modelli addestrati, nelle varianti Small, Medium e Large, hanno raggiunto prestazioni eccellenti e sostanzialmente equivalenti, con una precisione superiore al 99% e un mAP@50-95 intorno all'87%, indicando che il dataset disponibile satura la capacità estrattiva già a partire dalla configurazione più leggera. La scelta del modello YOLOv8m-OBb come candidato definitivo ha rappresentato il miglior compromesso tra accuratezza geometrica e costo computazionale.

La pipeline completa, dall'analisi video alla ricostruzione della simulazione fisica in PyBullet, ha dimostrato di essere in grado di riprodurre il comportamento dinamico reale dei carichi in modo soddisfacente. La campagna di fine-tuning condotta su 103 video ha evidenziato che il 64.1% dei casi raggiunge una perdita inferiore all'unità normalizzata, con il 26.2% che scende al di sotto di 10^{-2} . L'analisi della sensibilità parametrica ha identificato il coefficiente di attrito k_f e il numero di avvolgimenti della pellicola N_w come i fattori dominanti nel determinare la qualità della calibrazione, mentre la rigidità k_s si è rivelata marginalmente influente. L'eterogeneità dei risultati tra video riflette la varietà intrinseca delle configurazioni sperimentali e le difficoltà legate alla qualità del segnale visivo in condizioni di illuminazione critica.

Nel complesso, il sistema sviluppato rappresenta uno strumento concreto per ridurre il numero di test fisici necessari durante la fase di progettazione dell'imballaggio, consentendo all'operatore di esplorare scenari alternativi in simulazione prima di procedere con la realizzazione del prototipo fisico.

Lavori futuri

Nonostante i risultati incoraggianti, il sistema presenta margini di miglioramento su più fronti. Sul versante del rilevamento video, le prestazioni del tracker YOLO risultano ancora sensibili alle condizioni di illuminazione e al contrasto: tecniche di preprocessing adattivo o una data augmentation più aggressiva in fase di addestramento potrebbero aumentare la robustezza del sistema negli scenari visivamente più critici, riducendo la coda di video con perdita elevata osservata nei risultati.

Dal punto di vista della simulazione fisica, il modello attuale della pellicola estensibile adotta un approccio a parametri concentrati che, pur risultando efficace nella maggioranza dei casi, non cattura fenomeni locali come lacerazioni parziali o interazioni ad alta frequenza tra colli adiacenti. Un modello a elementi finiti o un approccio ibrido potrebbe migliorare la fedeltà della simulazione negli scenari dinamicamente più complessi. In modo analogo, la grid search parallela attuale esplora uno spazio discreto predefinito: la sostituzione con tecniche di ottimizzazione bayesiana o algoritmi evolutivi permetterebbe di esplorare spazi continui più ampi in tempi comparabili, con maggiori probabilità di trovare configurazioni ottimali nei paesaggi di perdita più pronunciati.

Infine, il dataset corrente copre una gamma specifica di prodotti e configurazioni di carico: l'estensione a tipologie di pallet differenti, imballaggi in carta, carichi asimmetrici, strutture a più strati, aprirebbe la strada a una generalizzazione più ampia del sistema. A più lungo termine, l'integrazione del framework in un flusso di lavoro industriale completo potrebbe consentire all'operatore non solo di testare configurazioni esistenti in simulazione, ma di ricevere indicazioni automatiche sulla disposizione ottimale dei colli o sul numero di giri di pellicola consigliato, chiudendo il ciclo tra analisi sperimentale e progettazione dell'imballaggio.

Bibliografia

- [1] ACMI S.p.A. ACMI S.p.A. - Packaging and End-of-Line Solutions, 2026.
- [2] Dr. Mohd Ansari and Dushyant Singh. Human detection techniques for real time surveillance: a comprehensive survey. *Multimedia Tools and Applications*, 80:1–50, 03 2021.
- [3] Herbert Bay, Tinne Tuytelaars, and Luc Van Gool. SURF: Speeded Up Robust Features. In *European Conference on Computer Vision*, pages 404–417. Springer, 2006.
- [4] Thierry Bouwmans. Traditional and Recent Approaches in Background Modeling for Foreground Detection: An Overview. *Computer Science Review*, 11-12:31–66, 2014.
- [5] Oriol Corcoll Andreu. Semantic Image Cropping, 07 2021.
- [6] Erwin Coumans. Bullet Physics Engine. In *ACM SIGGRAPH 2013 Courses*, 2013.
- [7] Erwin Coumans and Yunfei Bai. PyBullet Quickstart Guide, 2021.
- [8] Farzaneh Dadrass Javan, Farhad Samadzadegan, Mehrnaz Gholamshahi, and Farnaz Ashatari Mahini. A Modified YOLOv4 Deep Learning Network for Vision-Based UAV Recognition. *Drones*, 6(7), 2022.
- [9] Navneet Dalal and Bill Triggs. Histograms of Oriented Gradients for Human Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, volume 1, pages 886–893, 2005.
- [10] European Parliament and Council of the European Union. Directive 2014/45/EU on periodic roadworthiness tests for motor vehicles and their trailers, 2014. Official Journal of the European Union, L 127, pp. 51–128.
- [11] European Safe Logistics Association (EUMOS). EUMOS Quality and Safety Standards, 2026.

- [12] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation, 2014.
- [13] Sepp Hochreiter. The Vanishing Gradient Problem During Learning Recurrent Neural Nets and Problem Solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6(2):107–116, 1998.
- [14] J. D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
- [15] International Telecommunication Union. ITU-R Recommendation BT.601-7: Studio Encoding Parameters of Digital Television for Standard 4:3 and Wide-Screen 16:9 Aspect Ratios, 2011.
- [16] R. E. Kalman. A New Approach to Linear Filtering and Prediction Problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.
- [17] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet Classification with Deep Convolutional Neural Networks, 2012.
- [18] H. W. Kuhn. The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.
- [19] Roderic S. Lakes. *Viscoelastic Materials*. Cambridge University Press, 1 edition, 2009.
- [20] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep Learning. *Nature*, 521(7553):436–444, 2015.
- [21] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common Objects in Context. In *European Conference on Computer Vision (ECCV)*, 2014.
- [22] David G. Lowe. Distinctive Image Features from Scale-Invariant Keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.
- [23] Melvin Mooney. A Theory of Large Elastic Deformation. *Journal of Applied Physics*, 11(9):582–592, 1940.
- [24] Xuebin Qin, Zichen Zhang, Chenyang Huang, Masood Dehghan, Osmar R. Zaiane, and Martin Jagersand. U2-Net: Going deeper with nested U-structure for salient object detection. *Pattern Recognition*, 106:107404, 2020.

- [25] J. N. Reddy. *An Introduction to the Finite Element Method*. McGraw-Hill Education, 4 edition, 2019.
- [26] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You Only Look Once: Unified, Real-Time Object Detection, 2016.
- [27] Ronald Rivlin. Large Elastic Deformations of Isotropic Materials. IV. Further Developments of the General Theory. *Philosophical Transactions of the Royal Society of London. Series A*, 241(835):379–397, 1948.
- [28] Maxim Tkachenko, Mikhail Malyuk, Andrey Holmanyuk, and Nikolai Liubimov. Label Studio: Data labeling software, 2020–2025.
- [29] Ultralytics. Ultralytics YOLOv8, 2023.
- [30] Huo Yingge, Imran Ali, and Kang-Yoon Lee. Deep Neural Networks on Chip - A Survey, 02 2020.

Ringraziamenti

Giunto al termine di questo percorso di studi, sento il bisogno di esprimere il mio più profondo e sincero ringraziamento alla Prof.ssa Eleonora Iotti, relatrice di questa tesi, per la costante disponibilità, l'attenzione puntuale e la straordinaria dedizione dimostrate lungo tutto il percorso di ricerca e di scrittura. La sua guida sempre presente, i suoi consigli estremamente lucidi e la passione trasmessa nel seguire questo lavoro hanno rappresentato un riferimento fondamentale, permettendomi di affrontare ogni fase del progetto con crescente consapevolezza, metodo e maturità.

Ringrazio sentitamente il corpo docente e l'Università degli Studi di Parma, che in questi anni mi hanno fornito strumenti, conoscenze e opportunità imprescindibili per la mia crescita accademica e personale. Ogni insegnamento, diretto e indiretto, ha contribuito a costruire il percorso che mi ha condotto fino a questo traguardo.

Un ringraziamento speciale va alla mia famiglia, che ha rappresentato in ogni momento un punto fermo, stabile e insostituibile della mia vita.

Ai miei genitori, Anna e Massimo, va la mia più profonda gratitudine. Il loro sostegno non è mai stato soltanto presente, ma costante e silenziosamente determinante: fatto di sacrifici affrontati con discrezione, di incoraggiamenti mai invadenti ma sempre nel momento giusto, e di una fiducia incondizionata che mi ha permesso di non sentirmi mai solo nelle scelte più importanti. A loro devo non solo questo traguardo, ma il modo stesso in cui ho imparato ad affrontare il percorso per raggiungerlo. Il loro esempio rimane il riferimento più solido su cui ho costruito ogni passo.

A mio fratello Luca va un ringraziamento ancora più profondo e personale. La sua presenza è stata una costante reale e concreta, capace di andare oltre il semplice sostegno familiare: è stato ascolto nei momenti di confusione, lucidità quando serviva ordine, e confronto sincero quando era necessario rimettere a fuoco la direzione. Nei passaggi più impegnativi del percorso, il suo contributo è stato determinante nel mantenermi centrato, motivato e lucido. Gran parte di questo risultato è anche il frutto della sua vicinanza, spesso silenziosa ma sempre decisiva.

A Roberta, per aver condiviso con me una parte importante di questo percorso, offrendo supporto, incoraggiamento e presenza nei momenti in cui era più necessario, contribuendo a mantenermi motivato e determinato nel proseguire.

Ai miei nonni Iole e Silvio, va un ringraziamento colmo di affetto e riconoscenza. Il loro amore, sempre autentico e mai ostentato, ha rappresentato negli anni una presenza costante e rassicurante, capace di dare equilibrio e continuità anche nei momenti più complessi. Più ancora del sostegno quotidiano, porto con me il loro esempio di vita: fatto di semplicità, dignità e forza silenziosa, che ha lasciato un segno profondo nel mio modo di essere e di affrontare le difficoltà. Il loro supporto discreto, ma sempre presente, è stato un punto di riferimento solido e duraturo, che continua a vivere in ciò che sono diventato.

Un ringraziamento speciale va a Michele, Filippo e Simone, compagni di viaggio insostituibili, con i quali ho condiviso non solo momenti di studio e di crescita, ma anche esperienze che resteranno indelebili. La loro amicizia, il supporto reciproco e la capacità di alleggerire anche i momenti più impegnativi hanno reso questo percorso infinitamente più ricco e umano.

Un pensiero va a tutti i compagni di corso, per le risate, la collaborazione spontanea e tutte quelle piccole esperienze quotidiane che, sommandosi, hanno reso questo cammino più leggero e memorabile.

Un ringraziamento particolare agli amici della Skill 5.0. Nell'ultimo anno questo sport ha avuto un impatto profondo sulla mia vita, insegnandomi disciplina, costanza e una mentalità orientata al miglioramento continuo. Allenarmi insieme a persone capaci di trasmettere passione, supporto reciproco e spirito di squadra ha rappresentato un'esperienza formativa non solo dal punto di vista sportivo, ma soprattutto umano. Porterò sempre con me ciò che questo ambiente mi ha lasciato e insegnato.

Infine, un sincero grazie a tutte le altre persone che, in modi e forme diverse, hanno condiviso con me questo percorso di crescita. Amici, conoscenti e figure incontrate lungo questi anni hanno contribuito, anche solo con un gesto, una parola o una presenza discreta, a rendere questo cammino più ricco e significativo. Ognuno di loro, a suo modo, ha lasciato un segno che porto con me con gratitudine.

Grazie di cuore a tutti.

Marco

